


**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE GOIÁS**
**CAMPUS ANÁPOLIS**
**BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**

Wagner Oliveira de Araujo

# MIGRAÇÃO DE SISTEMAS LEGADOS DE ANÁLISE QUÍMICA: DE JAVA DESKTOP PARA PYTHON WEB

Anápolis
2026

Wagner Oliveira de Araujo

# MIGRAÇÃO DE SISTEMAS LEGADOS DE ANÁLISE QUÍMICA: DE JAVA DESKTOP PARA PYTHON WEB

Trabalho de Conclusão de Curso II do Bacharelado em Ciência da Computação do Instituto Federal de Educação, Ciência e Tecnologia de Goiás – Campus Anápolis para a aprovação da disciplina de TCC II.

Orientador: Prof.º Alexandre Bellezi Jose, Me.

Coorientador: Prof.º Eder Silva de Brito, Dr.

Anápolis

2026

Wagner Oliveira de Araujo

**MIGRAÇÃO DE SISTEMAS LEGADOS DE ANÁLISE QUÍMICA: DE JAVA DESKTOP PARA PYTHON WEB**

Este Trabalho de Conclusão de Curso II foi julgado adequado para obtenção do Título de "bacharel" e aprovado em sua forma final pelo Bacharelado em Ciência da Computação.

Anápolis, 20 de novembro de 2026.

---

Prof.º Hugo Vinicius Leão e Silva, Dr.
Coordenador do Curso

**Banca Examinadora:**

---

Prof.º Alexandre Bellezi Jose, Me.
Orientador

---

Prof.º Luiz Fernando Loja, Dr.
Avaliadora Interna
Instituto Federal de Goiás (IFG)

---

Prof.ª Maria de Oliveira Vaz dos Santos, Ma.
Avaliadora Interna
Instituto Federal de Goiás (IFG)

*Se você for receptivo e humilde, a matemática o conduzirá pela mão. Repetidas vezes, quando estive perdido sem saber como prosseguir, bastou-me esperar até sentir que a matemática me guiava pela mão. Ela me conduziu por um caminho inesperado, um caminho onde novas perspectivas se abrem, um caminho que leva a um território novo, onde é possível estabelecer uma base de operações, a partir da qual se pode observar os arredores e planejar o progresso futuro.*

(Paul Dirac)

## RESUMO

A calibração multivariada é essencial para determinar a composição elementar de aço-ligas a partir de dados espectrométricos. Este trabalho investigou, em caráter exploratório, a aplicação das técnicas de reamostragem *bootstrap*, *bagging* e *subagging* incorporadas à Regressão Linear Múltipla (RLM) para a estimativa quantitativa dos elementos Manganês (Mn), Molibdênio (Mo), Cromo (Cr), Níquel (Ni) e Ferro (Fe). Paralelamente, conduziu-se a modernização de um sistema legado monolítico em Java *Desktop* para uma aplicação *Web* em Python e Streamlit, adotando arquitetura desacoplada em três módulos de responsabilidade única. A validação do sistema reescrito e a fidelidade a lógica matemática original foram asseguradas por uma suíte de 92 testes automatizados desenvolvida com `pytest`, englobando 21 testes unitários, 60 testes de caixa-preta baseados nos arquivos do sistema legado e 11 testes de carga e estresse com matrizes de alta dimensão. Os resultados experimentais demonstraram que a eficiência das técnicas de *ensemble* varia conforme o analito: a RLM clássica obteve menor *Root-Mean-Square Error of Prediction* (RMSEP) para o Molibdênio e o Cromo; o RLM-*bagging* obteve menor erro para o Ferro; e o RLM-*subagging* apresentou melhor desempenho para o Manganês e o Níquel (neste último, com desempenho numericamente equivalente ao da RLM clássica, com diferença de 0,25% no (RMSEP). Em termos de eficiência computacional, o RLM-*subagging* reduziu o tempo de processamento em relação ao RLM-*bagging* em quatro dos cinco analitos. A suíte de testes confirmou a equivalência numérica estrita para os métodos determinísticos e a estabilidade da aplicação sob exigência computacional elevada, comprovando a viabilidade da modernização arquitetural sem perda de acurácia científica.



**Palavras-chave**: Técnicas de reamostragem. Validação de Modelos. *Bagging*. *Subagging*. Quimiometria.

## ABSTRACT

Multivariate calibration is essential for determining the elemental composition of alloy steels from spectrometric data. This study explored the application of bootstrap resampling, bagging, and subagging techniques incorporated into Multiple Linear Regression (MLR) for the quantitative estimation of Manganese (Mn), Molybdenum (Mo), Chromium (Cr), Nickel (Ni), and Iron (Fe). Concurrently, a legacy monolithic Java Desktop system was modernized into a Python and Streamlit Web application, adopting a decoupled architecture structured into three single-responsibility modules. The validation of the rewritten system and its fidelity to the original algorithmic core were ensured by a suite of 92 automated tests developed with pytest, encompassing 21 unit tests, 60 black-box tests based on legacy system outputs, and 11 load and stress tests with high-dimensional matrices. Experimental results demonstrated that the efficiency of ensemble techniques varies depending on the analyte: classical MLR achieved a lower Root-Mean-Square Error of Prediction (RMSEP) for Molybdenum and Chromium; MLR-bagging yielded the lowest error for Iron; and MLR-subagging exhibited superior performance for Manganese and Nickel (in the latter, with performance numerically equivalent to classical MLR, showing a 0.25% difference in RMSEP). In terms of computational efficiency, MLR-subagging reduced processing time compared to MLR-bagging across four of the five analytes. The test suite confirmed strict numerical equivalence for deterministic methods and application stability under high computational demand, proving the feasibility of architectural modernization without loss of scientific accuracy.



**Keywords**: Resampling. Multivariate Calibration. Bagging. Subagging. Chemometrics.

## LISTA DE FIGURAS


Figura 1 – Exemplo do método *bootstrap*. . . . . 19
Figura 2 – Processo de amostragem aleatória. . . . . 22
Figura 3 – Reamostragem com agregação de modelos de regressão. . . . . 25
Figura 4 – Regressão Linear Múltipla via pseudoinversa (`BaggingOptimized.java`). 31
Figura 5 – Ensemble de Bagging/Subagging (`BaggingOptimized.java`). . . . . 31
Figura 6 – Espectros médios dos analitos analisados. . . . . 33
Figura 7 – RMSEP com RLM para Mn e Fe. . . . . 37
Figura 8 – RMSEP em função do número de replicações *bootstrap* para RLM-*bagging*. 38
Figura 9 – RMSEP para o Mn e Fe com RLM-*bagging*. . . . . 39
Figura 10 – RMSEP em função das replicações *bootstrap* para RLM-*subagging*. . . . . 41
Figura 11 – RMSEP do Mn e Fe com *subagging*. . . . . 42
Figura 12 – Tempo de execução dos algoritmos RLM, RLM-*bagging* e RLM-*subagging*. 44
Figura 13 – RMSEP para todas as técnicas. . . . . 46
Figura 14 – Tempo de execução dos algoritmos RLM-*bagging* e RLM-*subagging* com parâmetros otimizados por validação cruzada 5-*fold*. . . . . 48
Figura 15 – Fluxograma do processo de execução da análise. . . . . 54
Figura 16 – Organização lógica em camadas dentro de um único arquivo — Sistema legado monolítico (Java Desktop). . . . . 60
Figura 17 – Arquitetura em camadas — Sistema modernizado (Python/Streamlit). . . . . 60
Figura 18 – Interface do sistema modernizado: (a) aba Configuração; (b) aba Resultados; (c) aba Gráficos. . . . . 63
Figura 19 – Diagrama de classes. . . . . 79
Figura 20 – Diagrama de atividades. . . . . 80
Figura 21 – Diagrama de sequência. . . . . 81
Figura 22 – Diagrama de atividade Java PlantUML . . . . . 82
Figura 23 – Diagrama de atividade Python PlantUML. . . . . 83
Figura 24 – Diagrama de classes (módulos) PlantUML. . . . . 84
Figura 25 – Diagrama de sequência PlantUML. . . . . 85

## LISTA DE QUADROS

Quadro 1 – Características comparativas entre *bagging* e *subagging*. . . . . 28
Quadro 2 – Dimensões dos conjuntos de dados para diferentes elementos. . . . . 34
Quadro 3 – Inventário das funções do sistema. . . . . 50
Quadro 4 – Entradas necessárias para execução do sistema. . . . . 51
Quadro 5 – Saídas esperadas do sistema. . . . . 51
Quadro 6 – Requisitos essenciais do sistema. . . . . 52
Quadro 7 – Requisitos desejáveis do sistema. . . . . 52
Quadro 8 – Requisitos extras para fases futuras. . . . . 53
Quadro 9 – Fluxos alternativos e tratamento de exceções. . . . . 54
Quadro 10 – Pontos críticos de usabilidade e estratégias de mitigação. . . . . 55
Quadro 11 – Requisitos de performance do sistema. . . . . 55
Quadro 12 – Requisitos de usabilidade do sistema. . . . . 56
Quadro 13 – Requisitos de segurança do sistema. . . . . 56
Quadro 14 – Requisitos de escalabilidade do sistema. . . . . 57

## LISTA DE TABELAS


Tabela 1 – O coeficiente de variação. . . . . 21
Tabela 2 – RMSEP por analito obtido via RLM para amostras de aço-ligas. . . . . 37
Tabela 3 – RMSEP da RLM e RLM-*bagging* para cada elemento de aço-ligas. . . . . 40
Tabela 4 – RMSEP por elemento de aço-ligas comparando RLM e RLM-*subagging*. . . 41
Tabela 5 – Comparação do RMSEP por elemento para diferentes técnicas e proporções de calibração. . . . . 42
Tabela 6 – Configuração de desempenho superior e RMSEP mínimo. . . . . 43
Tabela 7 – Desempenho comparativo das técnicas de calibração multivariada para elementos químicos em aço-liga. . . . . 47
Tabela 8 – Paridade *Bagging* – Cr. . . . . 66
Tabela 9 – Paridade *Bagging* – Fe. . . . . 86
Tabela 10 – Paridade *Bagging* – Mn. . . . . 87
Tabela 11 – Paridade *Bagging* – Ni. . . . . 88
Tabela 12 – Paridade *Bagging* – Mo. . . . . 89
Tabela 13 – Paridade *RLM* – Cr. . . . . 90
Tabela 14 – Paridade *RLM* – Mo. . . . . 91
Tabela 15 – Paridade *RLM* – Fe. . . . . 92
Tabela 16 – Paridade *RLM* – Ni. . . . . 93
Tabela 17 – Paridade *RLM* – Mn. . . . . 94
Tabela 18 – Paridade *Subagging* – Cr. . . . . 95
Tabela 19 – Paridade *Subagging* – Fe. . . . . 96
Tabela 20 – Paridade *Subagging* – Mo. . . . . 97
Tabela 21 – Paridade *Subagging* – Mn. . . . . 98
Tabela 22 – Paridade *Subagging* – Ni. . . . . 99

## LISTA DE ABREVIATURAS E SIGLAS

| | |
|---|---|
| AWT | *Abstract Window Toolkit* |
| EJML | *Efficient Java Matrix Library* |
| IDE | *Integrated Development Environment* |
| JAVA SE | *Java Platform, Standard Edition* |
| MAE | *Mean Absolute Error* |
| MoSCoW | *Must, Should, Could, Won't have* |
| MQO | Método dos Mínimos Quadrados Ordinários |
| MVP | *Minimum Viable Product* |
| RLM | Regressão Linear Múltipla |
| RMSEP | *Root-Mean-Square Error of Prediction* |
| $RMSEP_{CV}$ | *Root-Mean-Square Error of Prediction in Cross-Validation* |
| $R^2$ | Coeficiente de Determinação |
| SQR | Soma dos Quadrados dos Resíduos |
| SVD | *Singular Value Decomposition* |
| UML | *Unified Modeling Language* |
| WCAG | *Web Content Accessibility Guidelines* |

## LISTA DE SÍMBOLOS

| | |
|---|---|
| $y$ | Variável dependente (concentração do analito) |
| $y^i$ | Concentração do analito para a i-ésima amostra |
| $\gamma$ | Função estatística aplicada às amostras *bootstrap* |
| $\mathbf{x}_{al}$ | Vetor de observações de uma amostra aleatória |
| $t(\mathbf{d})$ | Função ou método avaliativo aplicado à distribuição de probabilidade d |
| $x^*$ | Amostra *bootstrap* individual |
| $\widehat{\theta}^*$ | Estimativa *bootstrap* do parâmetro $\theta$ |
| $m$ | Número de replicações *bootstrap* |
| $\widehat{se}$ | Erro padrão estimado (*estimated standard error*) |
| $\Delta$ | Parâmetro que mede o tamanho da distribuição empírica estimada usando amostras *bootstrap* |
| $\mathbf{u}$ | Amostra aleatória formada por n unidades selecionadas |
| $\mathbf{v}$ | População finita de referência |
| $x_i$ | Resposta instrumental em um canal analítico para a i-ésima amostra |
| $\mathbf{x}$ | Vetor de dados de dimensão 1 por J |
| $x_{al}$ | Variável aleatória escalar (resultado de um experimento aleatório) |
| $f_k$ | k-ésima frequência probabilística |
| $\mathbf{f}$ | Vetor de frequências probabilísticas |
| BI$(n, p)$ | Distribuição binomial de tamanho n e probabilidade de sucesso p |
| $\mu$ | Média populacional |
| $\sigma^2$ | Variância populacional |
| $N(\mu, \sigma^2)$ | Distribuição normal com média $\mu$ e variância $\sigma$ ao quadrado |
| $E(x)$ | Valor esperado (expectativa) de uma variável aleatória x |
| $E_{\mathbf{f}}(x)$ | Expectativa de x sob o vetor de frequências f |
| $y_{cal}$ | Variável aleatória de calibração |
| $z_{cal}$ | Variável aleatória auxiliar de calibração |
| $g(y_{cal})$ | Função g aplicada sobre y de calibração |
| $h(z_{cal})$ | Função h aplicada sobre z de calibração |
| $\mathbf{d}$ | Função de distribuição de probabilidade |
| $\widehat{\mathbf{d}}$ | Função de distribuição empírica |
| $\mathbf{X}_{cal}$ | Matriz de respostas instrumentais em J canais analíticos para as M amostras de calibração |
| $\mathbf{Y}_{cal}$ | Vetor de concentrações do analito para as M amostras de calibração |
| $\mathbf{X}_{teste}$ | Matriz de respostas instrumentais teste para os elementos Mn, Mo, Cr, Ni e Fe |
| $\mathbf{y}_{teste}$ | Vetor de respostas instrumentais teste para os elementos Mn, Mo, Cr, Ni e Fe |

**SUMÁRIO**


**1 INTRODUÇÃO . . . . . 14**
**2 REVISÃO DA LITERATURA . . . . . 17**
**2.1 Regressão Linear Múltipla . . . . . 17**
2.1.1 Estimação dos Coeficientes . . . . . 17
2.1.2 Premissas e Métodos Robustos . . . . . 18
**2.2 A Técnica de *Bootstrap* . . . . . 18**
2.2.1 *Bootstrap* . . . . . 19
2.2.2 Estimativa *Bootstrap* . . . . . 19
2.2.3 O Número de Replicações *Bootstrap m* . . . . . 20
**2.3 Reamostragem Aleatória . . . . . 22**
**2.4 Teoria da Probabilidade . . . . . 22**
**2.5 Função de Distribuição Empírica . . . . . 24**
**2.6 Princípio *Plug-in* . . . . . 25**
**2.7 Técnicas Clássicas de Reamostragem . . . . . 25**
2.7.1 *Bagging* . . . . . 26
2.7.2 *Subagging* . . . . . 27
2.7.3 Comparação entre *Bagging* e *Subagging* . . . . . 28
**2.8 Fundamentos de Engenharia de Migração e Qualidade de Software 29**
2.8.1 Teste de Software . . . . . 29
**3 MATERIAIS E MÉTODOS . . . . . 30**
**3.1 Fragmento do Código-Fonte Java — Lógica Matemática . . . . . 30**
**3.2 Amostras de Aço-Ligas . . . . . 32**
**3.3 Protocolo de Validação e Seleção de Hiperparâmetros . . . . . 34**
**3.4 Medida da Raiz do Erro Quadrático Médio de Predição . . . . . 35**
**4 ANÁLISE EXPERIMENTAL DOS ALGORITMOS QUIMIOMÉTRICOS 37**
**4.1 *Bagging* . . . . . 38**
**4.2 *Subagging* . . . . . 40**
**4.3 *Bagging* $\times$ *Subagging* . . . . . 45**
**5 REQUISITOS FUNCIONAIS . . . . . 50**
**5.1 Levantamento de Requisitos Funcionais . . . . . 50**
5.1.1 Inventário das Funcionalidades Existentes . . . . . 50
5.1.1.1 Funções do Sistema . . . . . 50
5.1.1.2 Entradas Necessárias (*Inputs*) . . . . . 51
5.1.1.3 Saídas Esperadas (*Outputs*) . . . . . 51
5.1.2 Classificação por Prioridade . . . . . 52
5.1.2.1 Essenciais (*Must Have*) . . . . . 52
5.1.2.2 Desejáveis (*Should Have*) . . . . . 52

5.1.2.3 Extras (*Could Have*) . . . 53
5.1.3 Fluxos de Uso . . . 53
5.1.3.1 Fluxo Principal: Execução de Análise Completa . . . 53
5.1.3.2 Fluxos Alternativos e Tratamento de Exceções . . . 54
5.1.3.3 Pontos Críticos de Usabilidade . . . 55
5.1.4 Requisitos Não Funcionais . . . 55
5.1.4.1 Performance . . . 55
5.1.4.2 Usabilidade . . . 56
5.1.4.3 Segurança . . . 56
5.1.4.4 Escalabilidade . . . 57
**5.2 Considerações Finais do Levantamento . . . 57**
**6 ENGENHARIA DE SOFTWARE E MODERNIZAÇÃO . . . 58**
**6.1 Engenharia de Requisitos com Sommerville . . . 58**
6.1.1 Elicitação . . . 58
6.1.2 Análise . . . 58
6.1.3 Especificação . . . 58
6.1.4 Validação . . . 59
**6.2 Arquitetura Modular em Python/Streamlit . . . 59**
**7 RESULTADOS E DISCUSSÕES . . . 61**
**7.1 Avaliação Preditiva e Eficiência Computacional . . . 61**
**8 INTERFACE E EXPERIÊNCIA DO USUÁRIO CIENTÍFICA . . . 62**
**8.1 Design da Interface e Fluxo de Trabalho . . . 62**
**9 DESENVOLVIMENTO ITERATIVO POR SPRINTS E VALIDAÇÃO POR TESTES AUTOMATIZADOS . . . 64**
**9.1 Metodologia Ágil: Desenvolvimento por Sprints . . . 64**
9.1.1 *Sprint* 1 — Camada de Leitura e Tratamento de Dados . . . 64
9.1.2 *Sprint* 2 — Algoritmos Científicos . . . 64
9.1.3 *Sprint* 3 — Interface Reativa . . . 64
9.1.4 *Sprint* 4 — Otimização com *Cache* . . . 64
**9.2 Suíte de Testes Automatizados com pytest . . . 65**
9.2.1 Testes Unitários (21 testes) . . . 65
9.2.2 Testes de Caixa-Preta (60 testes) . . . 65
9.2.3 Testes de Carga e *Stress* (11 testes) . . . 66
9.2.4 Resultados Consolidados . . . 66
**10 CONCLUSÃO . . . 68**
**10.1 Limitações e Trabalhos Futuros . . . 68**
**REFERÊNCIAS . . . 70**
**APÊNDICE A – DIAGRAMAS UML . . . 79**
**A.1 Diagrama de Classes — Java . . . 79**

**A.2 Diagrama de Atividade — Java . . . . . 80**
**A.3 Diagrama de Sequência — Java . . . . . 81**
**A.4 Diagrama de Atividades — Java (PlantUML) . . . . . 82**
**A.5 Diagrama de Atividades — Streamlit . . . . . 83**
**A.6 Diagrama de Classes (Módulos) — Streamlit . . . . . 84**
**A.7 Diagrama de Sequência — Streamlit . . . . . 85**
**APÊNDICE B – TABELAS DE PARIDADE AMOSTRA-A-AMOSTRA — JAVA $\times$ PYTHON . . . . . 86**
**APÊNDICE C – RESULTADO DOS TESTES . . . . . 100**

# 1 INTRODUÇÃO

A calibração multivariada desempenha um papel fundamental em diversas áreas da análise química e industrial, permitindo a estimação de concentrações a partir de dados instrumentais multivariados. No contexto específico da análise de aço-ligas, a determinação precisa da composição elementar (Mn, Cr, Mo, Ni, Fe) é crucial para garantir a qualidade e as propriedades desejadas do material.

No entanto, a complexidade dos dados multivariados provenientes de métodos instrumentais de análise exige o uso de técnicas de calibração multivariada para obter estimativas com erro de predição quantificável, expresso pela *Root-Mean-Square Error of Prediction* (RMSEP), pelo *Mean Absolute Error* (MAE) e pelo Coeficiente de Determinação ($R^2$).

Este estudo investiga, em caráter exploratório, a aplicação de técnicas de reamostragem (*bootstrap*), *bagging* e *subagging* na Regressão Linear Múltipla (RLM) para a calibração multivariada de amostras de aço-ligas. A pesquisa busca preencher a lacuna de conhecimento sobre a aplicação dessas técnicas de *ensemble* nesse contexto específico, fornecendo subsídios empíricos exploratórios para a quimiometria e exemplificando a modernização de sistemas computacionais de apoio à análise de aço-ligas (1, 2, 3).

Como otimizar a calibração multivariada de amostras de aço-ligas para obter previsões mais precisas dos elementos Mn, Cr, Mo, Ni e Fe, considerando a complexidade inerente à análise de dados multivariados, a necessidade de alta precisão e a exigência de uma arquitetura de *software* modular e verificável por testes automatizados para sustentar essa análise?

Este estudo contribui em dois eixos: (a) a aplicação de *bagging* e *subagging* à calibração de aço-ligas; (b) a migração de um sistema legado verificada por testes automatizados. Essa estratégia evidencia, de forma reprodutível, como preservar a corretude de uma lógica matemática crítico durante a reescrita completa de uma arquitetura de *software*.

A originalidade da pesquisa é dupla. No eixo quimiométrico, reside na aplicação de técnicas de *bagging* e *subagging*, ainda pouco exploradas na análise de aço-ligas.

No eixo de engenharia de *software*, reside na condução de uma reescrita caixa preta em três estágios (*Desktop* Java, *Web* Vaadin/Spring, Python/Streamlit), com arquitetura final desacoplada em três módulos de responsabilidade única e validação por 92 testes automatizados, que verificam a equivalência numérica do núcleo determinístico (a Regressão Linear Múltipla) e a consistência estatística dos métodos estocásticos entre o sistema legado e o sistema reescrito (4, 5).

Dado o conjunto amostral reduzido — característica inerente à aquisição dos dados espectrométricos utilizados —, os resultados devem ser interpretados como evidências preliminares do comportamento dessas técnicas nesse domínio, e não como afirmações definitivas de superioridade geral de um método sobre outro.

Analisar e comparar o desempenho das técnicas de *bagging* e *subagging* aplicadas à Regressão Linear Múltipla na calibração multivariada de dados de amostras de aço-ligas, in-

vestigando, em caráter exploratório, tanto o comportamento preditivo quanto a eficiência computacional de cada abordagem, a partir de uma arquitetura de *software* desenvolvida de forma incremental e validada a cada etapa da migração.

Esta pesquisa tem como objetivos específicos:

**a)** Aplicar as técnicas de reamostragem *bootstrap*, *bagging* e *subagging* na calibração de dados espectrométricos de amostras de aço-ligas, empregando a Regressão Linear Múltipla (RLM) como modelo base, visando a estruturação de um fluxo analítico reprodutível e padronizado (6, 7).

**b)** Realizar uma comparação sistemática entre os algoritmos RLM clássica, RLM-*bagging* e RLM-*subagging*, por meio da avaliação criteriosa da acurácia preditiva e da eficiência computacional de cada abordagem na análise quantitativa dos elementos Mn, Cr, Mo, Ni e Fe.

**c)** Reaproveitar, em *Java Platform, Standard Edition* (JAVA SE) 17, o sistema *Desktop* de calibração multivariada com RLM-*bagging* e RLM-*subagging* — desenvolvido na disciplina de Programação Orientada a Objetos, com arquitetura monolítica (em um único arquivo), biblioteca *Efficient Java Matrix Library* (EJML)[1] e interface gráfica Swing/*Abstract Window Toolkit* (AWT) —, refinando suas estruturas de dados e tratando inconsistências nas métricas de avaliação do algoritmo para servir como referência de validação e base de código legada.

**d)** Desenvolver e disponibilizar um protótipo *Web* reescrito em Python 3.12 e Streamlit, adotando arquitetura modular desacoplada em três módulos de responsabilidade única (`app.py`, `components.py` e `models.py`), que automatize o fluxo analítico de calibração multivariada por RLM, RLM-*bagging* e RLM-*subagging*, permitindo o carregamento de matrizes de calibração e teste, a configuração de hiperparâmetros, a execução de validação cruzada 5-*fold* por *grid search* e a geração automática de relatórios com métricas preditivas (RMSEP, MAE, $R^2$) e gráficos interativos Plotly.

**e)** Garantir a equivalência comportamental e a qualidade do *software* reescrito por meio da especificação de requisitos formais e da implementação de uma suíte de 92 testes automatizados com `pytest`, abrangendo testes unitários, testes de caixa-preta baseados em saídas do sistema legado e testes de estresse com matrizes sintetizadas de alta dimensão.

Esse protocolo de testes de caixa-preta (Seção 9.2) funciona como validador de equivalência entre o sistema legado e o sistema reescrito em dois níveis: equivalência numérica estrita para a RLM determinística e consistência estatística, dentro de tolerâncias explícitas, para o *bagging* e o *subagging*, cujas sequências pseudoaleatórias diferem entre as duas linguagens.

[1] A biblioteca *Efficient Java Matrix Library* (EJML) é uma biblioteca de álgebra linear de alto desempenho para Java, projetada para manipulação eficiente de matrizes densas e esparsas.

Este trabalho está organizado em dez seções. A Seção 1 apresenta a introdução, o problema de pesquisa e os objetivos. A Seção 2 reúne a revisão da literatura sobre a Regressão Linear Múltipla, o *bootstrap*, o *bagging* e o *subagging*. A Seção 3 descreve os materiais e métodos, incluindo as amostras de aço-ligas e o protocolo de validação. A Seção 4 apresenta a análise experimental dos algoritmos quimiométricos.

A Seção 5 levanta os requisitos funcionais da aplicação. A Seção 6 trata da engenharia de *software* e da modernização. A Seção 7 discute a avaliação preditiva e a eficiência computacional, e a Seção 8 trata da interface e da experiência do usuário.

A Seção 9 aborda o desenvolvimento por *sprints* e a validação por testes automatizados. Por fim, a Seção 10 apresenta a conclusão, as limitações e os trabalhos futuros.

# 2 REVISÃO DA LITERATURA

Esta seção apresenta a Regressão Linear Múltipla (RLM) e a técnica *bootstrap*. Adicionalmente, são abordados conceitos como a teoria da probabilidade, distribuição empírica, princípio *plug-in* e técnicas clássicas de reamostragem, fundamentais para a compreensão.

## 2.1 Regressão Linear Múltipla

A RLM é uma técnica estatística empregada para modelar a relação linear entre uma variável dependente (resposta) ($y$) e duas ou mais variáveis independentes (preditoras) ($\mathbf{X}$). O modelo, que inclui um termo de intercepto ($\beta_0$) e um termo de erro ($\varepsilon$), é formalizado como:

$$y = \beta_0 + x_1\beta_1 + x_2\beta_2 + \ldots + x_n\beta_n + \varepsilon \tag{1}$$

onde $\beta_j \in \mathbb{R}$ representa o coeficiente angular de cada preditor $x_j$, para $j = 1, 2, \ldots, n$.

Dispondo de um conjunto de $M$ dados de calibração $\{(x^i, y^i), \quad i = 1, 2, \ldots, M\}$, o modelo pode ser expresso na forma matricial:

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon \tag{2}$$

onde o vetor de resposta $\mathbf{y}$ e a matriz de calibração $\mathbf{X}$ (incluindo a coluna de uns para o intercepto) são definidos por:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^1 & x_2^1 & \ldots & x_n^1 \\ 1 & x_1^2 & x_2^2 & \ldots & x_n^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^M & x_2^M & \ldots & x_n^M \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y^1 \\ y^2 \\ \vdots \\ y^M \end{bmatrix} \tag{3}$$

O vetor de coeficientes populacionais é $\beta = [\beta_0, \beta_1, \ldots, \beta_n]^{\mathbf{T}}$.

### 2.1.1 Estimação dos Coeficientes

O procedimento de estimação é conhecido como Método dos Mínimos Quadrados Ordinários (MQO). O MQO busca o vetor de coeficientes estimados $\widehat{\beta}$ que minimiza a Soma dos Quadrados dos Resíduos (SQR).

A melhor estimativa $\widehat{\beta}$ no sentido de mínimos quadrados é dada pela solução da pseudo-inversa de Moore-Penrose (8, 3):

$$\widehat{\beta} = \mathbf{X}^{+}\mathbf{y} \triangleq (\mathbf{X}^{\mathbf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathbf{T}}\mathbf{y} \tag{4}$$

onde $\mathbf{X}^{+}$ é a pseudo-inversa da matriz $\mathbf{X}$.

### 2.1.2 Premissas e Métodos Robustos

A validade das estimativas obtidas pelo MQO depende do cumprimento das premissas clássicas da regressão linear. Em dados espectrais, a colinearidade entre variáveis e a presença de outliers violam com frequência esses pressupostos (9, 10).

Métodos robustos tornam-se indispensáveis quando há violação das premissas da regressão linear múltipla, especialmente na presença de *outliers* (9, 10). A verificação da normalidade dos resíduos, em particular, é crucial e discutida em detalhes em estudos anteriores (9, 10).

## 2.2 A Técnica de *Bootstrap*

A inferência estatística estabelece propriedades para a população $\mathbf{X}$ a partir da análise de uma amostra aleatória dessa população (8, 11, 12). O método *bootstrap* é um procedimento computacional que fornece a medida do nível de precisão da inferência estatística, como por exemplo, média, desvio padrão e erro padrão (13, 14, 15, 16, 17).

A ideia básica do *bootstrap* é inferir estatisticamente as propriedades de interesse a partir da análise de amostras aleatórias extraídas da população original sem a necessidade de conhecer a distribuição de dados a priori, o que dispensa a suposição prévia de uma distribuição paramétrica dos dados (2, 3).

Quando o parâmetro populacional de interesse é desconhecido, procura-se estimá-lo por reamostragem aleatória. Inicialmente, considera-se a **população** $\mathbf{x} = [x_1, x_2, \ldots, x_N]$. A partir dela, seleciona-se uma **amostra** $\mathbf{X} = [x_1, x_2, \ldots, x_n]$, formada por $n$ elementos independentes retirados da população.

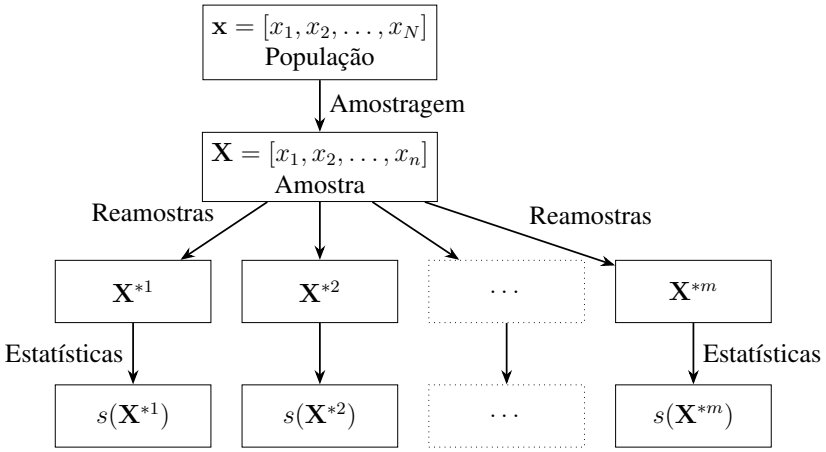
Para avaliar a variabilidade das estimativas, aplica-se o método de ***bootstrap***, que consiste em gerar $m$ **reamostras** $\mathbf{X}^{*1}, \mathbf{X}^{*2}, \ldots, \mathbf{X}^{*m}$, obtidas por sorteio com reposição a partir da amostra original. Formalmente, cada reamostra é definida como

$$\mathbf{X}^{*i} \sim \textit{Bootstrap}(\mathbf{X}), \quad i = 1, 2, \ldots, m,$$

onde $\textit{Bootstrap}(\mathbf{X})$ denota a distribuição empírica gerada pela amostra.

Em seguida, calcula-se uma estatística $s(\mathbf{X}^{*i})$ para cada reamostra, gerando uma distribuição empírica das estimativas. Esse processo é ilustrado na Figura 1.

Figura 1 – Exemplo do método *bootstrap*.




Fonte: Wagner Oliveira de Araujo.

A Figura 1 ilustra o procedimento de *bootstrap* em quatro etapas. O primeiro passo (primeiro retângulo) representa a população completa, denotada por $\mathbf{x} = [x_1, x_2, \ldots, x_N]$, da qual é extraída uma amostra através do processo de amostragem. O segundo passo (segundo retângulo) corresponde ao conjunto de dados observados, denotado por $\mathbf{X} = [x_1, x_2, \ldots, x_n]$.

O terceiro passo (terceiro conjunto de retângulos) corresponde à extração de amostras *bootstrap*, obtidas por reamostragem com reposição a partir de $\mathbf{X}$. O quarto passo (quarto conjunto de retângulos) mostra as replicações *bootstrap*, nas quais uma função estatística $s(\gamma)$ é recalculada $m$ vezes, sendo $m$ o número total de replicações.

Nas seções subsequentes, são discutidos conceitos fundamentais relacionados a amostras aleatórias e probabilísticas, bem como a distribuição empírica e o princípio *plug-in*. A apresentação desses tópicos tem como objetivo fornecer a base teórica necessária para a compreensão dos procedimentos de inferência estatística realizados a partir de amostras *bootstrap*.

### 2.2.1 *Bootstrap*

Uma amostra aleatória $\mathbf{x}_{al} = [x_{al1}, \ldots, x_{aln}]$ é extraída da distribuição de probabilidade $\mathbf{D}$. O parâmetro de interesse $\theta = t(\mathbf{d})$ é desconhecido e será estimado com base na amostra observada $\mathbf{x}_{al}$, resultando no estimador $\widehat{\theta} = t(\mathbf{x}_{al})$.

### 2.2.2 Estimativa *Bootstrap*

O método *bootstrap* necessita inicialmente das amostras *bootstrap* para depois aplicar as replicações *bootstrap*, como foi visto na Figura 1. Suponha a distribuição empírica $\widehat{\mathbf{d}}$, com probabilidade $\frac{1}{n}$ em cada valor de $x_i$, tendo $i = 1, 2, \ldots, n$. A amostra *bootstrap* é definida como sendo a amostra aleatória de tamanho $n$ extraída de $\widehat{\mathbf{d}}$, representada por $x^* = [x_1^*, x_2^*, \ldots, x_n^*]$ (18). As amostras *bootstrap* são obtidas por reamostragem com reposição, podendo haver repetições dos elementos da amostra original $[x_1, x_2, \ldots, x_n]$.

As replicações *bootstrap* são representadas por

$$\widehat{\theta} = s(x^*), \tag{5}$$

onde $s(\gamma)$ é a função estatística a ser aplicada sobre as amostras *bootstrap*. Por exemplo, se $s(x)$ for a média amostral $\overline{x}$, então $s(x^*)$ é a média da amostra *bootstrap*

$$\overline{x^*} = \frac{1}{n}\sum_{i=1}^{n} x_i^*. \tag{6}$$

**Algoritmo 1**: Estimativa das replicações *bootstrap*.

**Entrada**: Amostra original $x = (x_1, x_2, \ldots, x_n)$, estatística de interesse $\theta = s(x)$, número de replicações *bootstrap* $m$

**Saída**: Estimativa *bootstrap* $\widehat{\theta}^*$, erro padrão *bootstrap*, intervalo de confiança *bootstrap*

1 **início**
2 Selecione a quantidade de amostras independentes $m$, $\{x^{*1}, x^{*2}, \ldots, x^{*m}\}$, tendo tamanho $m$ e podendo haver repetições de $x$.
3 Avaliar a replicação *bootstrap* para amostra *bootstrap*, $\widehat{\theta}^*(b) = s(x^{*b}) \quad b = 1, 2, \ldots, m.$
4 **fim**

Fonte: Wagner Oliveira de Araujo.

O Algoritmo 1 (19) mostra a descrição do procedimento para utilizar a estimativa *bootstrap*, replicando o estimador $\widehat{\theta} = s(x)$ a partir dos dados $x$ observados.

### 2.2.3 O Número de Replicações *Bootstrap m*

A estimativa *bootstrap* ideal é obtida através de uma boa escolha do número de replicações $m = \infty$, com o $\widehat{se}_\infty$ igual à estimativa *plug-in* $se_{\widehat{\mathbf{d}}}(\widehat{\theta}^*)$ (19). O número de replicações *bootstrap* influencia diretamente o tempo computacional gasto ao processar as replicações, principalmente se a função $s(x)$ do estimador $\widehat{\theta} = s(x)$ for muito complexa. Portanto, se não for possível obter infinitas replicações, deve-se utilizar o maior número possível de replicações.

É necessário que estimadores apresentem bom comportamento: baixo erro de predição e pequeno desvio padrão. A estimativa *bootstrap* para o erro padrão geralmente apresenta erro de predição relativamente pequeno. A estimativa *bootstrap* ideal $\widehat{se}_\infty$ possui baixa variabilidade entre estimativas com erro de predição quase nulo de $se_{\widehat{\mathbf{d}}}(\widehat{\theta^*})$, pelo menos em um sentido assintótico.

Esta propriedade decorre do fato de que $\widehat{se}_m$ é a estimativa *plug-in* de $se_{\widehat{\mathbf{d}}}(\widehat{\theta^*})$. Não é difícil mostrar que $\widehat{se}_m$ sempre possui desvio padrão maior que $\widehat{se}_\infty$ (19). A questão que se coloca é: **o quanto maior**?

Uma resposta aproximada, porém satisfatória, pode ser encontrada utilizando o coeficiente de variação de $\widehat{se}_m$ em relação à sua expectativa. O aumento da variabilidade decorrente da interrupção do processo após $m$ replicações *bootstrap*, em vez de permitir que $m$ tenda ao infinito, resulta em um aumento do coeficiente de variação:

$$cv(\widehat{se}_m) = \sqrt{[cv(\widehat{se}_\infty)]^2 + \frac{E(\Delta) + 2}{4m}}. \tag{7}$$

O parâmetro $\Delta$ mede o tamanho da distribuição de $\widehat{\theta}^*$: $\Delta$ é zero para a distribuição normal; na prática, $\Delta$ geralmente é maior que 10. O coeficiente de variação na Equação (7) incorpora tanto a variabilidade devida à reamostragem *bootstrap* quanto a variabilidade devida à amostragem da população original.

A estimativa ideal $\widehat{se}_\infty = se_{\widehat{\mathbf{d}}}(\widehat{\theta}^*)$ não é perfeita (19). Pode haver variabilidade estatisticamente significativa na estimativa $se_{\mathbf{d}}(\widehat{\theta})$ devido à variabilidade de $\widehat{\mathbf{d}}$ em relação a $\mathbf{d}$. Por exemplo, se $\{x_1, x_2, \ldots, x_n\}$ for uma amostra aleatória de uma distribuição normal e $\widehat{\theta} = \overline{x}$, então $cv(\widehat{se}_\infty) = \frac{1}{\sqrt{2n}}$, resultando em $0,22$ para $n = 10$.

A Equação (7) tem uma consequência prática importante: dados os valores típicos de $cv(\widehat{se}_\infty)$ e $\Delta$ encontrados na prática, $cv(\widehat{se}_m)$ não será muito maior que $cv(\widehat{se}_\infty)$ quando $m \geq 200$.

Tabela 1 – O coeficiente de variação.

| | m[a]→ | | | | |
|---|---|---|---|---|---|
| | 25 | 50 | 100 | 200 | $\infty$ |
| $cv(\widehat{se}_\infty)$[b] 0,25 | 0,29 | 0,27 | 0,26 | 0,25 | 0,25 |
| ↓ 0,20 | 0,24 | 0,22 | 0,21 | 0,21 | 0,20 |
| 0,15 | 0,21 | 0,18 | 0,17 | 0,16 | 0,15 |
| 0,10 | 0,17 | 0,14 | 0,12 | 0,11 | 0,10 |
| 0,05 | 0,15 | 0,11 | 0,09 | 0,07 | 0,05 |
| 0,00 | 0,14 | 0,10 | 0,07 | 0,05 | 0,00 |

Fonte: EFRON, B.; TIBSHIRANI, R. J. *An Introduction to the Bootstrap*, USA: Chapman & Hall/CRC, 1993, p.53.
[a] Número de amostras *bootstrap*.
[b] Coeficiente de variação.

A Tabela 1 compara $cv(\widehat{se}_m)$ com $cv(\widehat{se}_\infty)$ para várias escolhas de $m$, assumindo $\Delta = 0$. Para $cv(\widehat{se}_\infty) \geq 0{,}10$, com $m = 100$ o $cv(\widehat{se}_m)$ excede o ideal em no máximo $0{,}02$ (Tabela 1). Efron (19) sugere duas regras práticas através da sua experiência:

a) Mesmo um número pequeno de replicações *bootstrap*, como $m = 25$, é geralmente informativo; $m = 50$ é frequentemente suficiente para uma boa estimativa de $se_{\mathbf{d}}(\widehat{\theta})$.

b) Segundo Efron e Tibshirani (19), raramente são necessárias mais de 200 replicações para o erro padrão; para intervalos de confiança.

A Tabela 1 mostra a variação de $\widehat{se}_m$ em função do coeficiente de variação da estimativa ideal $\widehat{se}_\infty$ e do número de amostras *bootstrap* $m$, obtidos da Equação (7) assumindo $\Delta = 0$.

## 2.3 Reamostragem Aleatória

O processo de reamostragem é realizado a partir da seleção aleatória de números inteiros $\{j_1, j_2, \ldots, j_n\}$, cada um com probabilidade $1/n$. Uma amostra aleatória de tamanho $n$ é definida pela seleção aleatória de $n$ unidades $\{u_1, u_2, \ldots, u_n\}$ extraídas da população $\mathbf{u}$ (ver Figura 2), onde $\mathbf{u}$ é a população de números inteiros selecionados para gerar variáveis aleatórias.

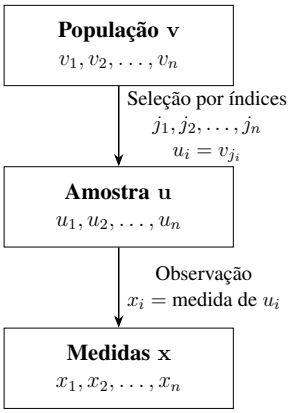
A população $\mathbf{v}$ é formada pelos valores selecionados de $\mathbf{u}$ de acordo com os valores de $j$, sendo constituída por $\{v_1, v_2, \ldots, v_n\}$. Esta população é criada por um dispositivo (como um software) responsável por gerar as amostras aleatórias. Por meio dos valores $\{j_1, j_2, \ldots, j_n\}$ selecionados, pode-se determinar quais membros de $\mathbf{v}$ serão escolhidos para a amostra aleatória:

$$u_1 = v_{j_1}, \quad u_2 = v_{j_2}, \quad \ldots, \quad u_n = v_{j_n}.$$

Sendo $x_i$ uma medida da unidade $u_i$ para $i = 1, 2, \ldots, n$, o conjunto de dados observados será

$$\mathbf{x} = \{x_1, x_2, \ldots, x_n\}. \tag{8}$$

Figura 2 – Processo de amostragem aleatória.




Fonte: Wagner Oliveira de Araujo.

Se as unidades da amostra $\{u_1, u_2, \ldots, u_n\}$ são observadas, obtêm-se as medidas correspondentes $\{x_1, x_2, \ldots, x_n\}$, que formam o vetor de dados $\mathbf{x}$.

## 2.4 Teoria da Probabilidade

A partir de uma amostra aleatória $\mathbf{x}_{al}$, deve-se inferir a distribuição de probabilidade da população completa $\mathbf{X} = [X_1, X_2, \ldots, X_n]$, onde $X_i$ representa a variável aleatória associada à $i$-ésima observação. A teoria da probabilidade segue o raciocínio oposto: a partir do conhecimento da distribuição de probabilidade da população $\mathbf{X}$, deduzem-se as propriedades das amostras aleatórias e das estatísticas calculadas a partir delas.

Seja $x_{al}$ o resultado da rolagem de um dado não viciado. Por exemplo, no lançamento de um dado cúbico de 6 faces, onde todas as faces possuem probabilidades iguais de ocorrência,

obtém-se como resultado 1, 2, 3, 4, 5 ou 6. Em notação probabilística, tem-se

$$\Pr\{x_{al} = k\} = \frac{1}{6}, \quad \text{para } k = 1, 2, 3, 4, 5, 6. \tag{9}$$

A quantidade aleatória que possua a probabilidade de ser selecionada como $x_{al}$ é chamada de *variável aleatória*. Na teoria da probabilidade, a notação utilizada para definir a frequência $f_k$ é dada por

$$f_k = \Pr\{x_{al} = k\}. \tag{10}$$

Na rolagem do dado não viciado, tem-se $f_k = \frac{1}{6}$ para $k = 1, 2, \ldots, 6$. A distribuição de probabilidade para a variável aleatória $x_{al}$ será denotada por $\mathbf{f}$. Aqui, $\mathbf{f}$ é o vetor de frequências

$$\mathbf{f} = (f_1, f_2, \ldots, f_6) = \left(\frac{1}{6}, \frac{1}{6}, \ldots, \frac{1}{6}\right). \tag{11}$$

Podem-se encontrar também amostras com variáveis aleatórias $x_{al}$ na forma da distribuição binomial de tamanho $n$ e probabilidade de sucesso $p$, denotada

$$x_{al} \sim \text{BI}(n, p). \tag{12}$$

A frequência é representada por

$$f_k = \binom{n}{k} p^k (1-p)^{n-k}, \quad \text{para } k = 0, 1, 2, \ldots, n, \tag{13}$$

sendo $n$ um inteiro positivo, $p$ um número entre 0 e 1, e $\binom{n}{k}$ o coeficiente binomial definido por

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}. \tag{14}$$

A probabilidade pode ser definida também como no exemplo da distribuição normal (ou gaussiana). Uma variável aleatória de valor real $x_{al}$, tendo a distribuição normal com média $\mu$ e variância $\sigma^2$, é descrita por

$$x_{al} \sim N(\mu, \sigma^2). \tag{15}$$

A probabilidade de $x_{al}$ pertencer a um conjunto $A \subseteq \mathbb{R}$ é dada por

$$\Pr\{x_{al} \in A\} = \int_A \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right) dx. \tag{16}$$

Existem outros problemas com dimensões maiores envolvendo integrais similares à Equação (16), mas não será necessário utilizar estas distribuições contínuas para o *bootstrap* neste trabalho.

A expectativa da variável aleatória de valor real $x_{al}$, escrita por $E(x)$, é o valor médio sobre os possíveis resultados de $x_{al}$. Assim, para a distribuição binomial:

$$E(x_{al}) = \sum_{k=0}^{n} k \binom{n}{k} p^k (1-p)^{n-k}, \quad \text{para } x_{al} \sim \text{BI}(n, p), \tag{17}$$

e para a distribuição normal:

$$E(x_{al}) = \int_{-\infty}^{\infty} x \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right) dx, \quad \text{para } x_{al} \sim N(\mu, \sigma^2). \tag{18}$$

A expectativa, como a teoria da média, é muito ampla e inclui casos de variáveis aleatórias sem valor real definido.

Pode-se obter a variância de $x_{al}$, representada por $\sigma^2$. Tendo $\mu = E_{\mathbf{f}}(x)$ para $x_{al}$ sendo uma variável aleatória de valor real, a variância é definida como o valor esperado de $(x_{al} - \mu)^2$. Em outras palavras, $\sigma^2$ é o quadrado médio das distâncias da variável aleatória $x_{al}$ em relação à sua expectativa $\mu$ (6):

$$\sigma^2 = E_{\mathbf{f}}[(x_{al} - \mu)^2]. \tag{19}$$

A variância de $x_{al} \sim N(\mu, \sigma^2)$ é igual a $\sigma^2$, e a variância de $x_{al} \sim \text{BI}(n, p)$ é igual a $np(1-p)$. O desvio padrão da variável aleatória é definido pela raiz quadrada destas variâncias.

Para que duas variáveis aleatórias $y_{cal}$ e $z_{cal}$ sejam independentes, deve-se ter

$$E[g(y_{cal})h(z_{cal})] = E[g(y_{cal})] \cdot E[h(z_{cal})] \tag{20}$$

para todas as funções $g$ e $h$. A independência implica que o resultado aleatório de $y_{cal}$ não afeta o resultado aleatório de $z_{cal}$, e vice-versa.

## 2.5 Função de Distribuição Empírica

Suponha que uma amostra aleatória de tamanho $n$, dada por $\mathbf{x} = \{x_1, x_2, \ldots, x_n\}$, seja obtida de uma função de distribuição de probabilidade $\mathbf{d}$. A função de distribuição empírica $\widehat{\mathbf{d}}$ é definida como a distribuição discreta que atribui probabilidade $\frac{1}{n}$ a cada valor observado $x_i$, para $i = 1, 2, \ldots, n$.

Em outras palavras, a distribuição empírica $\widehat{\mathbf{d}}$ é construída a partir do espaço amostral de $\mathbf{x}$, atribuindo a seguinte probabilidade empírica a um evento $A$:

$$\widehat{\Pr}\{A\} = \frac{\#\{x_i \in A\}}{n}, \tag{21}$$

onde $\#\{x_i \in A\}$ denota o número de observações $x_i$ que pertencem ao conjunto $A$.

Em casos onde ocorrem repetições de valores na amostra, para evitar atribuir probabilidade $\frac{1}{n}$ a cada ocorrência repetida individualmente, utiliza-se $\widehat{\mathbf{d}}$ com as frequências relativas observadas. Para valores distintos $k$ no espaço amostral, define-se

$$\widehat{f}_k = \frac{\#\{x_i = k\}}{n}, \tag{22}$$

onde $\#\{x_i = k\}$ representa o número de observações na amostra que são iguais a $k$. Assim, $\widehat{f}_k$ é a frequência relativa (ou probabilidade empírica) do valor $k$ na amostra.

## 2.6 Princípio *Plug-in*

O princípio *plug-in* constitui uma abordagem fundamental para estimação de parâmetros estatísticos, permitindo estimar um parâmetro de interesse $\theta$ por meio de uma função avaliativa aplicada à distribuição empírica dos dados (20).

Formalmente, seja $\theta = t(F)$ um parâmetro definido como função da distribuição populacional $F$. O estimador *plug-in* é obtido substituindo $F$ pela distribuição empírica $\hat{F}_n$:

$$\hat{\theta} = t(\hat{F}_n), \tag{23}$$

onde $n$ denota o tamanho da amostra.

Para o caso particular em que $\theta$ representa a expectativa populacional $\mathbb{E}_F(X)$, o estimador *plug-in* corresponde à média amostral:

$$\hat{\theta} = \mathbb{E}_{\hat{F}_n}(X) = \frac{1}{n}\sum_{i=1}^{n} x_i = \bar{x}. \tag{24}$$

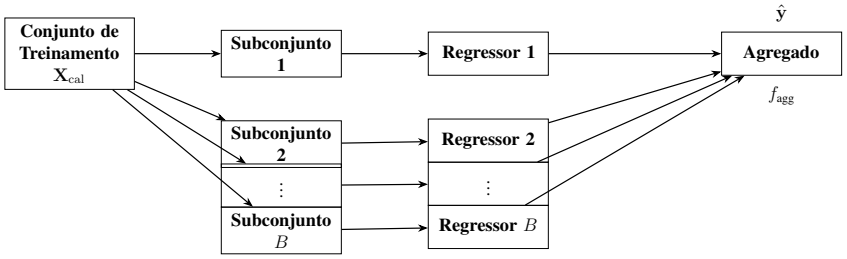
É importante destacar que o método *bootstrap* constitui uma aplicação direta do princípio *plug-in*, podendo ser adotado como método de estimação para diversos modelos estatísticos, incluindo modelos de regressão (20, 21).

## 2.7 Técnicas Clássicas de Reamostragem

Nesta seção, revisam-se duas técnicas fundamentais de reamostragem: *bagging* e *subbagging*. Ambas utilizam amostras *bootstrap*, como detalhado na Seção 2, baseando-se na distribuição empírica das probabilidades.

As técnicas de reamostragem são particularmente úteis em cenários com limitação amostral ou quando os dados são contaminados por ruídos instrumentais. A ideia central consiste em combinar as predições de múltiplos modelos de regressão em um único preditor agregado, processo conhecido como *ensemble* (22, 23).

Figura 3 – Reamostragem com agregação de modelos de regressão.




Fonte: Wagner Oliveira de Araujo.

A Figura 3 ilustra o fluxo de reamostragem e agregação: o conjunto de calibração $\mathbf{X}_{\text{cal}}$ é reamostrado em $B$ subconjuntos, cada um utilizado para calibrar um regressor independente.

As predições individuais são subsequentemente combinadas no estágio de agregação mediante uma função $f_{\text{agg}}$.

Técnicas *ensemble* integram múltiplos modelos para aprimorar o desempenho preditivo final (22). Neste trabalho, emprega-se a média aritmética como função de agregação, com todos os regressores utilizando Regressão Linear Múltipla (RLM), caracterizando um *ensemble* homogêneo (24, 25, 26).

A distinção fundamental entre *bagging* e *subagging* reside no esquema de amostragem: com e sem reposição, respectivamente. As Seções 2.7.1 e 2.7.2 detalham estas técnicas.

### 2.7.1 *Bagging*

A técnica *bagging* (*Bootstrap Aggregating*) foi proposta por Breiman (27) com o objetivo de gerar múltiplos conjuntos de calibração por meio de reamostragem *bootstrap*, combinando posteriormente os resultados obtidos. Esta abordagem visa reduzir a variância do preditor final através da agregação (27, 28, 29, 30, 7).

Breiman demonstrou que o *bagging* proporciona ganhos expressivos de desempenho em problemas de regressão, especialmente quando o modelo é instável – isto é, quando pequenas alterações no conjunto de calibração resultam em mudanças drásticas nas predições. Na decomposição do erro de predição, o *bagging* atua principalmente na redução da componente de variância do preditor.

A técnica também se mostra eficaz em regressões multivariadas, particularmente em problemas de seleção de amostras de calibração em modelos de Regressão Linear Múltipla (RLM) (31, 2). Variações na seleção de variáveis, decorrentes de flutuações nos conjuntos de calibração, são suavizadas pelo cálculo da média dos modelos resultantes (32).

**Algoritmo 2**: Estimativa *bagging*.

**Entrada**: Amostra original $\mathbf{x} = (x_1, x_2, \ldots, x_n)$ com $n$ observações, estatística de interesse $\theta = s(\mathbf{x})$, número de réplicas *bootstrap* $B$

**Saída**: Estimativa *bagging* $\hat{\theta}^*_{\text{bag}}$, distribuição das estimativas *bootstrap* $\{\hat{\theta}^*(1), \ldots, \hat{\theta}^*(B)\}$

**início**

  Definir o número de réplicas *bootstrap* $B$

  **para** $b = 1$ ***até*** $B$ **faça**

    Gerar amostra *bootstrap* $\mathbf{x}^{*(b)}$ de tamanho $n$ com reposição a partir de $\mathbf{x}$

    Calcular a estatística para a réplica $b$:

    $\hat{\theta}^*(b) = s(\mathbf{x}^{*(b)})$

  **fim**

  Agregar as réplicas via média aritmética:

$$\hat{\theta}^*_{\text{bag}} = \frac{1}{B} \sum_{b=1}^{B} \hat{\theta}^*(b)$$

**fim**

Fonte: Wagner Oliveira de Araujo.

O Algoritmo 2 descreve o procedimento para aplicação do *bagging*. Inicialmente, define-se o número $B$ de réplicas *bootstrap* a serem geradas. Para cada réplica $b = 1, \ldots, B$, extrai-se uma amostra $\mathbf{x}^{*(b)}$ de tamanho $n$ com reposição da amostra original, aplica-se a função $s(\cdot)$ para obter $\hat{\theta}^*(b)$, e finalmente agrega-se os resultados através da média aritmética (33).

No *bagging* tradicional proposto por Breiman (34), cada amostra *bootstrap* possui o mesmo tamanho da amostra original. Neste trabalho, a implementação do *bagging* utilizando Regressão Linear Múltipla (RLM) é denominada **RLM-*bagging***.

### 2.7.2 *Subagging*

A técnica *subagging* (*subsample aggregating*) baseia-se na reamostragem sem reposição, diferindo do *bagging* tradicional por não permitir repetições de observações dentro das amostras (23, 35, 36, 37, 38, 39, 40, 41, 42, 43). Cada modelo é construído de maneira similar ao *bagging*, porém utilizando subamostras de tamanho reduzido $k < n$.

**Algoritmo 3**: Estimativa *subagging*.

**Entrada**: Amostra original $\mathbf{x} = (x_1, x_2, \ldots, x_n)$ com $n$ observações, estatística de interesse $\theta = s(\mathbf{x})$, número de réplicas $B$, tamanho das subamostras $k$ (onde $k \leq n$)

**Saída**: Estimativa subagging $\hat{\theta}^*_{\text{subag}}$, conjunto de estimativas $\{\hat{\theta}^*(1), \ldots, \hat{\theta}^*(B)\}$

**início**
  Definir o tamanho das subamostras $k$ (tipicamente $k < n$)
  Definir o número de réplicas $B$
  **para** $b = 1$ ***até*** $B$ **faça**
    Gerar subamostra $\mathbf{x}^{*(b)}$ de tamanho $k$ **sem reposição** a partir de $\mathbf{x}$
    Calcular a estatística para a subamostra: $\hat{\theta}^*(b) = s(\mathbf{x}^{*(b)})$
  **fim**
  Agregar as réplicas via média aritmética:

$$\hat{\theta}^*_{\text{subag}} = \frac{1}{B} \sum_{b=1}^{B} \hat{\theta}^*(b)$$

**fim**

Fonte: Wagner Oliveira de Araujo.

O Algoritmo 3 descreve o procedimento para aplicação do *subagging*. Inicialmente, definem-se os parâmetros $k$ (tamanho das subamostras) e $B$ (número de réplicas). Para cada iteração $b = 1, \ldots, B$, gera-se uma subamostra $\mathbf{x}^{*(b)}$ de tamanho $k$ **sem reposição** a partir dos dados originais, aplica-se a função $s(\cdot)$, e finalmente agrega-se os resultados através da média aritmética.

A ausência de reposição implica que cada observação aparece no máximo uma vez por subamostra, diferenciando fundamentalmente esta técnica do *bagging*. Esta característica torna

o *subagging* particularmente sensível à escolha do tamanho das subamostras $k$ e do número de réplicas $B$. A relação ótima entre estes parâmetros, bem como uma análise comparativa entre *bagging* e *subagging*, são discutidas na Seção 2.7.3.

Bühlmann (35) introduziu o subagging como uma otimização do *bagging* para subamostras de dimensão $m < n$. A generalização para regressão de alta dimensionalidade foi formalizada por Ing e Lai (44). Análises teóricas recentes (39, 36) determinam condições necessárias e suficientes para a convergência e eficiência assintótica[1] desses métodos.

Neste trabalho, a implementação do *subagging* utilizando Regressão Linear Múltipla (RLM) é denominada **RLM-*subagging***.

### 2.7.3 Comparação entre *Bagging* e *Subagging*

Embora *bagging* e *subagging* compartilhem o princípio de agregação de múltiplos modelos, diferem fundamentalmente no mecanismo de reamostragem empregado. O Quadro 1 sintetiza as principais distinções entre estas técnicas, destacando suas características operacionais e implicações para a variância dos estimadores resultantes.

Quadro 1 – Características comparativas entre *bagging* e *subagging*.

| **Característica** | ***Bagging*** | ***Subagging*** |
|---|---|---|
| Tipo de amostragem | Com reposição | Sem reposição |
| Tamanho das réplicas | $n$ | $k \leq n$ |
| Repetição de observações | Permitida | Não permitida |
| Diversidade entre réplicas | Moderada | Alta |
| Variância do estimador | Redução moderada | Redução potencialmente maior |
| Sensibilidade a parâmetros | Baixa | Alta ($k$ e $B$) |

Fonte: Wagner Oliveira de Araujo.

Como mostra o Quadro 1, a ausência de reposição no *subagging* resulta em maior diversidade entre as réplicas, uma vez que cada subamostra contém um conjunto distinto de observações. Esta característica pode proporcionar maior redução de variância, mas torna a técnica mais sensível à escolha adequada dos parâmetros $k$ (tamanho das subamostras) e $B$ (número de réplicas).

A escolha entre *bagging* e *subagging* depende fundamentalmente das características do problema em questão. O *bagging* é mais robusto e requer menos ajuste de parâmetros, sendo preferível quando se dispõe de amostras de tamanho moderado. Por outro lado, o *subagging* pode ser vantajoso em cenários com amostras grandes ou quando há interesse em controlar explicitamente o tamanho das subamostras para reduzir o custo computacional (35).

Nas aplicações com Regressão Linear Múltipla (RLM) desenvolvidas neste trabalho, ambas as técnicas serão avaliadas empiricamente, permitindo uma análise comparativa de seus desempenhos preditivos em diferentes configurações amostrais.

[1] Assintótica refere-se ao comportamento de algo quando uma variável tende ao infinito (ou a um limite extremo).

## 2.8 Fundamentos de Engenharia de Migração e Qualidade de Software

Assim como a calibração multivariada, a modernização de sistemas legados possui fundamentação teórica própria. Sommerville (45) estrutura a engenharia de requisitos em quatro fases interdependentes — elicitação, análise, especificação e validação —, oferecendo o arcabouço metodológico adotado neste trabalho para decompor as responsabilidades do sistema Java legado em módulos independentes, evitando que a reescrita repetisse o acoplamento entre lógica de negócio, persistência e interface presente na implementação original.

A estratégia de migração adotada — reescrita caixa preta — extrai os requisitos do comportamento observável de um sistema legado, sem reaproveitar seu código-fonte (46). Diferentemente da reescrita caixa-branca, que preserva estruturas internas do sistema original, essa abordagem trata o legado exclusivamente como especificação funcional, permitindo à equipe redesenhar a arquitetura livremente, desde que a equivalência comportamental da lógica matématica seja formalmente demonstrada em cada etapa da migração.

A verificação dessa equivalência comportamental exige um critério de avaliação independente da implementação (47). Testes de caixa-preta comparam entradas e saídas entre o sistema original e o sistema reescrito sem inspecionar o código-fonte legado, estratégia adequada quando o objetivo — como neste trabalho — é substituir integralmente a plataforma de execução preservando o comportamento matemático. Essa fundamentação sustenta a suíte de 92 testes automatizados que compara Java e Python, apresentada adiante.

### 2.8.1 Teste de Software

O teste de software verifica se um sistema se comporta conforme especificado, revelando defeitos antes da operação (47). Neste trabalho, empregam-se o teste unitário, que exercita funções isoladas; o teste de caixa-preta, que confronta entradas e saídas sem inspecionar o código interno (47, 48); e o teste de carga e *stress*, que mede tempo e memória sob grandes volumes de dados. A automação com *frameworks* como o `pytest` (49) permite repetir essas verificações a cada alteração. Na migração, o teste de caixa-preta serve para verificar a equivalência entre o sistema legado e o reescrito, pois compara apenas o comportamento observável.

# 3 MATERIAIS E MÉTODOS

Esta seção apresenta os dados utilizados na aplicação dos algoritmos de reamostragem, bem como as medidas estatísticas empregadas para comparação entre os métodos. O objetivo é determinar a concentração de Manganês (Mn), Molibdênio (Mo), Cromo (Cr), Níquel (Ni) e Ferro (Fe) a partir de espectros reamostrados pelos métodos *bagging* e *subagging*.

Para a implementação computacional, utilizou-se a linguagem de programação Java (JavaSE-17) em conjunto com o ambiente de desenvolvimento integrado Eclipse *Integrated Development Environment* (IDE) (versão 2025-09). As operações matriciais exigidas pelos algoritmos de calibração multivariada, incluindo o cômputo da pseudo-inversa de Moore-Penrose, as reamostragens *bootstrap* e a agregação dos modelos de *ensemble*, foram implementadas com o suporte da biblioteca *Efficient Java Matrix Library*, `org.ejml.simple.SimpleMatrix` (EJML). Esta biblioteca fornece as operações de álgebra linear utilizadas no trabalho, como multiplicação matricial e pseudoinversa de Moore-Penrose (50).

A visualização gráfica dos resultados foi realizada pela biblioteca JFreeChart[1], que possibilitou a geração dos gráficos de predição e comparação de desempenho deste trabalho. A interface gráfica de usuário, para entrada de parâmetros e exibição de resultados durante os experimentos, foi desenvolvida com os *frameworks* nativos Java Swing e AWT. O gerenciamento dos arquivos de entrada e saída, incluindo leitura das matrizes espectrométricas e exportação dos resultados, foi realizado pelas bibliotecas padrão `java.io`.

Durante o desenvolvimento deste trabalho, o modelo de linguagem de inteligência artificial Qwen3.7-Plus foi utilizado como ferramenta de assistência para: revisão ortográfica, sugestão de estruturação de seções, tradução de resumos, refatoração de trechos de código (Java/Pthyton), correção de erros código LaTeX (51, 52). A validação factual, a análise crítica dos dados espectrométricos e a redação final das conclusões foram realizadas integralmente pelo autor, garantindo a integridade acadêmica e a autoria do trabalho.

## 3.1 Fragmento do Código-Fonte Java — Lógica Matemática

A classe `BaggingOptimized` (que estende `JFrame`) concentra, entre os componentes de interface gráfica, a lógica matemática responsável pelos três modelos de regressão avaliados neste trabalho. A Figura 4 apresenta o método `performRLM`, que implementa a Regressão Linear Múltipla simples via pseudoinversa de Moore-Penrose.

[1]Versões: EJML 0.43.1; JFreeChart 1.5.4. Código em: https://github.com/woa2009/MigracaoJava

Figura 4 – Regressão Linear Múltipla via pseudoinversa (`BaggingOptimized.java`).

```
private SimpleMatrix performRLM(SimpleMatrix Xcal, SimpleMatrix Ycal,
                                SimpleMatrix Xteste) {
    SimpleMatrix b = pseudoInverse(Xcal).mult(Ycal);
    return Xteste.mult(b);
}
```

Fonte: Wagner Oliveira de Araujo.

A Figura 5 apresenta o método `performBagging`, que implementa tanto o *Bagging* (reamostragem com reposição) quanto o *Subagging* (reamostragem sem reposição, com fração configurável), controlados pelo parâmetro booleano `withReplacement`. Cada um dos `numBags` modelos é ajustado sobre uma reamostra distinta e a predição final é obtida pela média aritmética das predições individuais.

Figura 5 – Ensemble de Bagging/Subagging (`BaggingOptimized.java`).

```
private SimpleMatrix performBagging(SimpleMatrix Xcal, SimpleMatrix Ycal,
        SimpleMatrix Xteste, int numBags, boolean withReplacement,
        double sampleFraction) {
    int n = Xcal.numRows();
    int sampleSize = withReplacement ? n : (int) (n * sampleFraction);
    List<SimpleMatrix> models = new ArrayList<>();
    Random rand = new Random(42); // semente fixa para reprodutibilidade

    for (int bag = 0; bag < numBags; bag++) {
        SimpleMatrix Xbag = new SimpleMatrix(sampleSize, Xcal.numCols());
        SimpleMatrix Ybag = new SimpleMatrix(sampleSize, 1);

        if (withReplacement) {
            // Bagging: amostragem com reposicao (bootstrap completo)
            for (int i = 0; i < sampleSize; i++) {
                int idx = rand.nextInt(n);
                copyRow(Xcal, idx, Xbag, i);
                Ybag.set(i, 0, Ycal.get(idx, 0));
            }
        } else {
            // Subagging: amostragem sem reposicao (subconjunto aleatorio)
            List<Integer> indices = new ArrayList<>();
            for (int i = 0; i < n; i++) indices.add(i);
            Collections.shuffle(indices, rand);
            for (int i = 0; i < sampleSize; i++) {
                int idx = indices.get(i);
                copyRow(Xcal, idx, Xbag, i);
                Ybag.set(i, 0, Ycal.get(idx, 0));
            }
```

```
        }

        // Ajusta o modelo RLM para esta amostra bootstrap
        SimpleMatrix b_bag = pseudoInverse(Xbag).mult(Ybag);
        models.add(b_bag);
    }

    // Agrega as predicoes de todos os modelos pela media aritmetica
    SimpleMatrix Ypred = new SimpleMatrix(Xteste.numRows(), 1);
    for (SimpleMatrix b : models) {
        SimpleMatrix pred = Xteste.mult(b);
        Ypred = Ypred.plus(pred.scale(1.0 / numBags));
    }
    return Ypred;
}
```

Fonte: Wagner Oliveira de Araujo.

Essa mesma alternativa — reamostragem com ou sem reposição controlada por um único parâmetro booleano — é preservada na reimplementação Python (Seção 6), constituindo um dos pontos de verificação dos testes de caixa-preta descritos na Seção 9.2.

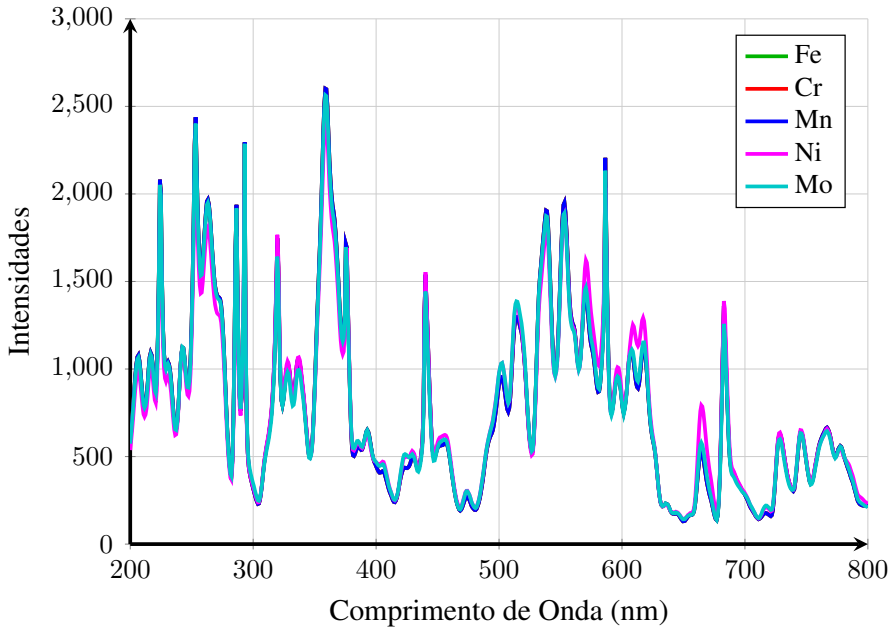
## 3.2 Amostras de Aço-Ligas

Os dados foram obtidos por meio de um espectrômetro de emissão atômica em plasma com acoplamento indutivo sequencial convertido para um sistema de múltiplos canais de detecção baseado em um arranjo linear de 1024 fotodiodos. O aparelho instrumental faz uso das propriedades de absorção e emissão de plasma, como detalhado em (53).

A espectroscopia de emissão atômica é responsável pela conversão dos componentes da amostra em átomos ou íons elementares, podendo excitá-los a vários estados eletrônicos. A rápida relaxação das espécies excitadas possibilita a produção de espectros visíveis que serão analisados quantitativamente e qualitativamente, com vantagens sobre outros métodos clássicos. Por exemplo, a baixa interferência entre elementos da mesma amostra, e o registro de espectros para vários elementos no mesmo conjunto de excitação (1).

Os espectros obtidos por fontes de plasma contêm centenas a milhares de frequências (723 variáveis por amostra neste trabalho) (Figura 6). Logo, melhoram a análise qualitativa. Porém, dificulta a análise quantitativa pela grande quantidade de variáveis geradas por amostra motivando o emprego de calibração multivariada (54, 55).

Figura 6 – Espectros médios dos analitos analisados.




Fonte: Wagner Oliveira de Araujo.

As matrizes de respostas instrumentais e concentração das amostras de aço-ligas são organizadas em dois conjuntos de dados: calibração (33,3 – 45,9%) e teste (54,1 – 66,7%), como segue (Quadro 2). Cada amostra é representada por 723 variáveis espectrais, resultando em uma razão de aproximadamente 42 preditores por observação no conjunto de calibração. Esse regime de alta dimensionalidade — inerente à natureza dos dados espectrométricos de plasma — é tratado pela pseudo-inversa de Moore-Penrose (Equação 4), que fornece solução única mesmo em sistemas subdeterminados.

Na reimplementação em Python (Seção 6), os coeficientes são obtidos pela função `np.linalg.lstsq` da biblioteca NumPy (56, 57). Essa função resolve o problema de mínimos quadrados, isto é, encontra o vetor $\widehat{\boldsymbol{\beta}}$ que minimiza a norma euclidiana do resíduo:

$$\widehat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 . \tag{25}$$

Internamente, a função utiliza uma rotina da biblioteca LAPACK baseada na decomposição em valores singulares *Singular Value Decomposition* (SVD), (58, 57), que fatora a matriz de calibração como $\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathbf{T}}$. A solução é então calculada por $\widehat{\boldsymbol{\beta}} = \mathbf{V}\boldsymbol{\Sigma}^{+}\mathbf{U}^{\mathbf{T}}\mathbf{y}$, em que $\boldsymbol{\Sigma}^{+}$ inverte os valores singulares não nulos e descarta aqueles inferiores a um limiar proporcional ao maior valor singular (parâmetro `rcond`, cujo valor padrão deriva da precisão da máquina).

No presente trabalho, as matrizes de calibração possuem 17 amostras e 723 variáveis, de modo que o sistema é subdeterminado: existem infinitas soluções com resíduo nulo. Nesse caso, `np.linalg.lstsq` retorna, entre todas elas, a de menor norma euclidiana $\|\boldsymbol{\beta}\|_2$, que coincide com $\mathbf{X}^{+}\mathbf{y}$, a solução da pseudoinversa de Moore-Penrose (Equação 4). Portanto, as duas abordagens são matematicamente equivalentes; a diferença é que `lstsq` obtém o resultado

diretamente da decomposição, sem formar explicitamente a matriz $\mathbf{X}^{+}$, como faz o método `pseudoInverse()` da biblioteca EJML no sistema Java.

Ressalta-se, contudo, que conjuntos de calibração reduzidos são uma característica recorrente em análises de aço-ligas por espectrometria, como documentado na literatura (53, 54), e que os resultados obtidos neste estudo devem ser interpretados como evidências exploratórias do comportamento das técnicas investigadas nesse contexto específico, não sendo diretamente generalizáveis a outras composições ou regimes amostrais sem validação adicional.

Quadro 2 – Dimensões dos conjuntos de dados para diferentes elementos.

| Elemento | Calibração | Teste |
|---|---|---|
| Mn | $\mathbf{Xcal}_{Mn_{(17\times723)}}$, $\mathbf{Ycal}_{Mn_{(17\times1)}}$ | $\mathbf{Xteste}_{Mn_{(34\times723)}}$, $\mathbf{yteste}_{Mn_{(34\times1)}}$ |
| Mo | $\mathbf{Xcal}_{Mo_{(17\times723)}}$, $\mathbf{Ycal}_{Mo_{(17\times1)}}$ | $\mathbf{Xteste}_{Mo_{(21\times723)}}$, $\mathbf{yteste}_{Mo_{(21\times1)}}$ |
| Cr | $\mathbf{Xcal}_{Cr_{(17\times723)}}$, $\mathbf{Ycal}_{Cr_{(17\times1)}}$ | $\mathbf{Xteste}_{Cr_{(34\times723)}}$, $\mathbf{yteste}_{Cr_{(34\times1)}}$ |
| Ni | $\mathbf{Xcal}_{Ni_{(17\times723)}}$, $\mathbf{Ycal}_{Ni_{(17\times1)}}$ | $\mathbf{Xteste}_{Ni_{(20\times723)}}$, $\mathbf{yteste}_{Ni_{(20\times1)}}$ |
| Fe | $\mathbf{X}_{calFe_{(17\times723)}}$, $\mathbf{Y}_{calFe_{(17\times1)}}$ | $\mathbf{X}_{testeFe_{(34\times723)}}$, $\mathbf{y}_{testeFe_{(34\times1)}}$ |

Fonte: Wagner Oliveira de Araujo.

## 3.3 Protocolo de Validação e Seleção de Hiperparâmetros

A seleção dos hiperparâmetros dos algoritmos RLM-*bagging* e RLM-*subagging* — respectivamente, o número de replicações bootstrap $m$ e a fração de subamostragem $k$ — foi conduzida mediante validação cruzada *5-fold* aplicada exclusivamente ao conjunto de calibração, garantindo que o conjunto de teste permanecesse estritamente independente durante todo o processo de otimização.

O procedimento adotado seguiu as etapas descritas a seguir. Primeiramente, o conjunto de calibração foi particionado em cinco subconjuntos (*folds*) de tamanho aproximadamente igual, após embaralhamento aleatório com semente fixa (`seed` = 42) para assegurar a reprodutibilidade dos resultados. Quando o número de amostras de calibração não é divisível por cinco, o último *fold* absorve as amostras remanescentes, podendo conter uma ou duas amostras adicionais em relação aos demais *folds*.

Em seguida, para cada combinação de hiperparâmetros avaliada, realizou-se uma busca em grade (*grid search*) sobre os conjuntos de valores $m \in \{10, 20, 30, 40, 50\}$ e $k \in \{0{,}50;\ 0{,}60;\ 0{,}70;\ 0{,}80\}$.

A avaliação do modelo foi realizada por validação cruzada *k-fold* dividida em cinco etapas. A cada iteração, utilizaram-se quatro partições para treinamento e uma para validação, repetindo-se o ciclo até que todos os grupos fossem avaliados.

O desempenho foi quantificado pelo *Root-Mean-Square Error of Prediction in Cross-Validation* ($\text{RMSEP}_{\text{CV}}$) médio (Equação 26), onde $K = 5$ representa o total de *folds*, $n_k$ a quantidade de amostras na partição $k$, $y_i^{(k)}$ o valor de referência e $\hat{y}_i^{(k)}$ a predição da $i$-ésima

amostra.

$$\text{RMSEP}_{\text{CV}} = \frac{1}{K}\sum k = 1^{K}\sqrt{\frac{1}{n_k}\sum_{i=1}^{n_k}\left(\hat{y}_i^{(k)} - y_i^{(k)}\right)^2} \tag{26}$$

Os hiperparâmetros ótimos $m^*$ e $k^*$ foram definidos como a combinação que minimizou o $\text{RMSEP}_{\text{CV}}$. Após a seleção, os modelos finais foram retreinados utilizando a totalidade do conjunto de calibração com os parâmetros ótimos identificados. A avaliação final de desempenho foi realizada uma única vez sobre o conjunto de teste independente, reportando-se as métricas RMSEP, MAE e $R^2$, definidas nas Equações 27, 28 e 29, respectivamente.

## 3.4 Medida da Raiz do Erro Quadrático Médio de Predição

Para avaliar e comparar a acurácia preditiva dos modelos de regressão linear múltipla (RLM) clássica, RLM com *bagging* (RLM-*bagging*) e RLM com *subagging* (RLM-*subagging*), utilizou-se um conjunto de três métricas complementares: o RMSEP[2], o MAE[3] e o coeficiente de determinação $R^2$. A adoção conjunta dessas métricas proporciona uma caracterização mais abrangente do desempenho preditivo, uma vez que cada uma captura aspectos distintos dos erros de predição (59, 31).

O RMSEP fornece uma medida agregada da magnitude dos erros entre os valores previstos por um modelo e os valores reais observados em um conjunto de dados independente (geralmente um conjunto de teste ou validação) (59, 31). Ao elevar os erros ao quadrado antes de calcular a média e, posteriormente, extrair a raiz quadrada, o RMSEP atribui maior peso aos erros maiores, penalizando modelos com grandes discrepâncias entre as predições e a realidade.

Essa característica o torna particularmente útil em situações onde grandes erros são mais indesejáveis, sendo adotado como critério primário de seleção do melhor modelo neste trabalho (24, 60, 17, 59).

Obtidos os coeficientes de regressão para cada uma das técnicas avaliadas, calculou-se a concentração do elemento de interesse. O RMSEP objetiva minimizar a soma quadrada da diferença entre a concentração predita e a concentração observada, como expresso na Equação (27):

$$\text{RMSEP} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\widehat{y}_i - y_i)^2} \tag{27}$$

onde $\widehat{y}_i$ é a $i$-ésima concentração predita pelo modelo de regressão, $y_i$ é a concentração observada (real) e $n$ é o número de amostras no conjunto de validação.

O MAE complementa o RMSEP ao quantificar a magnitude média dos erros de predição sem elevar os resíduos ao quadrado, atribuindo peso igual a todos os desvios independentemente de sua magnitude (61). Isso o torna mais robusto à presença de valores discrepantes e permite verificar a consistência dos erros: quando o RMSEP supera consideravelmente o MAE, há

[2]Raiz do Erro Quadrático Médio de Predição de Predição
[3]Erro Médio Absoluto

indício de erros grandes e isolados nas predições; quando ambos são próximos, os erros tendem a ser homogeneamente distribuídos. O MAE é dado pela Equação (28):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\widehat{y}_i - y_i| \tag{28}$$

onde $\widehat{y}_i$, $y_i$ e $n$ mantêm as mesmas definições apresentadas na Equação (27). Assim como o RMSEP, o MAE é expresso na mesma unidade da variável resposta, com valores menores indicando maior acurácia preditiva.

O coeficiente de determinação $\text{R}^2$ avalia a proporção da variância total da variável resposta que é explicada pelo modelo preditivo, sendo definido pela Equação (29):

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(\widehat{y}_i - y_i)^2}{\sum_{i=1}^{n}(\bar{y} - y_i)^2} \tag{29}$$

onde $\bar{y}$ representa a média dos valores de referência no conjunto de teste. O $\text{R}^2$ assume valores no intervalo $(-\infty,\, 1]$: $\text{R}^2 = 1$ indica ajuste perfeito; $\text{R}^2 = 0$ equivale ao desempenho de um modelo que prediz apenas a média da variável resposta; e valores negativos indicam desempenho inferior a essa referência trivial (62).

Na calibração multivariada de aço-ligas, valores de $\text{R}^2 > 0{,}99$ são frequentemente reportados como indicativos de elevada capacidade preditiva (59, 31, 60). Contudo, o $\text{R}^2$ isoladamente pode ser enganoso em conjuntos de teste pequenos ou com distribuição não uniforme de concentrações, razão pela qual sua interpretação é sempre realizada em conjunto com o RMSEP e o MAE neste trabalho.

# 4 ANÁLISE EXPERIMENTAL DOS ALGORITMOS QUIMIOMÉTRICOS

Esta seção apresenta os resultados e discute as técnicas de *bootstrap*, *bagging* e *subagging* aplicadas à regressão linear múltipla. Ela avalia a Raiz do Erro Quadrático Médio de Predição (RMSEP) e compara os tempos de execução dos algoritmos RLM, RLM-*bagging* e RLM-*subagging*.

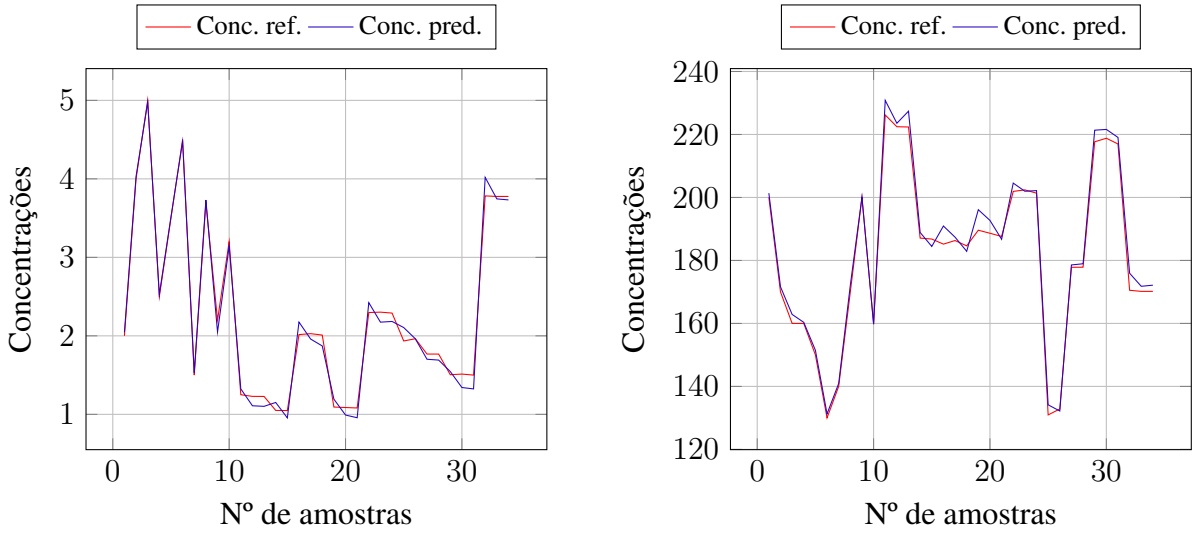
Tabela 2 – RMSEP por analito obtido via RLM para amostras de aço-ligas.

| **Elementos** | **RMSEP (RLM)** |
|---|---|
| Mn | 0,1051 |
| Mo | 0,2633 |
| Cr | 1,0798 |
| Ni | 1,6043 |
| Fe | 2,7513 |

Fonte: Wagner Oliveira de Araujo.

A Tabela 2 mostra o RMSEP obtido com a RLM para todas as amostras de aço-ligas.

Figura 7 – RMSEP com RLM para Mn e Fe.




(a) RMSEP com RLM para Mn.

(b) RMSEP com RLM para Fe.

Fonte: Wagner Oliveira de Araujo.

A Figura 7 exibe o comportamento do RMSEP para os elementos Mn e Fe. A Figura 7a detalha o RMSEP do manganês (Mn) e a Figura 7b, o do ferro (Fe). O sinal vermelho representa a concentração de referência do Mn e do Fe e o sinal azul a concentração predita obtida com a RLM. A Tabela 2 mostra que, para a RLM, o Mn apresenta o menor RMSEP (0,1051) e o Fe o maior (2,7513).

Na comparação com parâmetros otimizados (Tabela 7), o RLM-*subagging* obteve RMSEP menor que o RLM-*bagging* em quatro dos cinco analitos (Mn, Mo, Cr e Ni), e a RLM

clássica obteve o menor RMSEP entre as três técnicas em dois analitos (Mo e Cr). Isso indica que a técnica mais adequada depende do analito.

A escolha dos elementos ferro e manganês para esta análise baseia-se em fundamentos científicos bem estabelecidos na metalurgia. O ferro é o elemento matriz que define a própria natureza do material, uma vez que sem ferro não existe aço. O manganês, por sua vez, é o único elemento de liga historicamente considerado essencial para a produção de qualquer aço comercial, atuando como desoxidante e modificador fundamental das propriedades mecânicas.

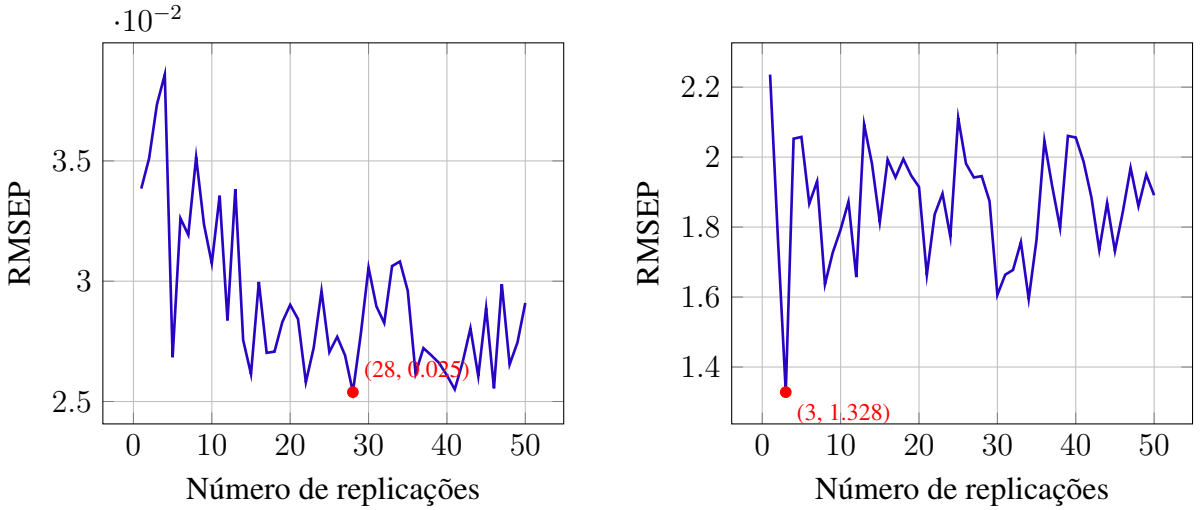
Os demais elementos (Cr, Mo, Ni) são importantes para propriedades específicas, mas não são fundamentais para a definição básica e produção do aço como o ferro e o manganês. Mn e Fe foram escolhidos como analitos ilustrativos nas figuras desta seção; os resultados dos cinco analitos são apresentados na Tabela 7.

## 4.1 *Bagging*

Os resultados obtidos com a técnica *bagging* foram produzidos com a implementação do Algoritmo 2. Contudo, como visto na Subseção 2.7.1 da Seção 2.7, o algoritmo RLM-*bagging* depende do parâmetro $m$ que define o número de replicações *bootstrap*.

Foi conduzido um experimento para avaliar a sensibilidade do *bagging* em função do número de replicações, variando de 1 a 50. O procedimento experimental implementou o RLM-*bagging* com um algoritmo iterativo que modifica sistematicamente o número de replicações utilizadas.

Figura 8 – RMSEP em função do número de replicações *bootstrap* para RLM-*bagging*.




(a) RMSEP em função do número de replicação para o Mn.

(b) RMSEP em função do número de replicação para o Fe.

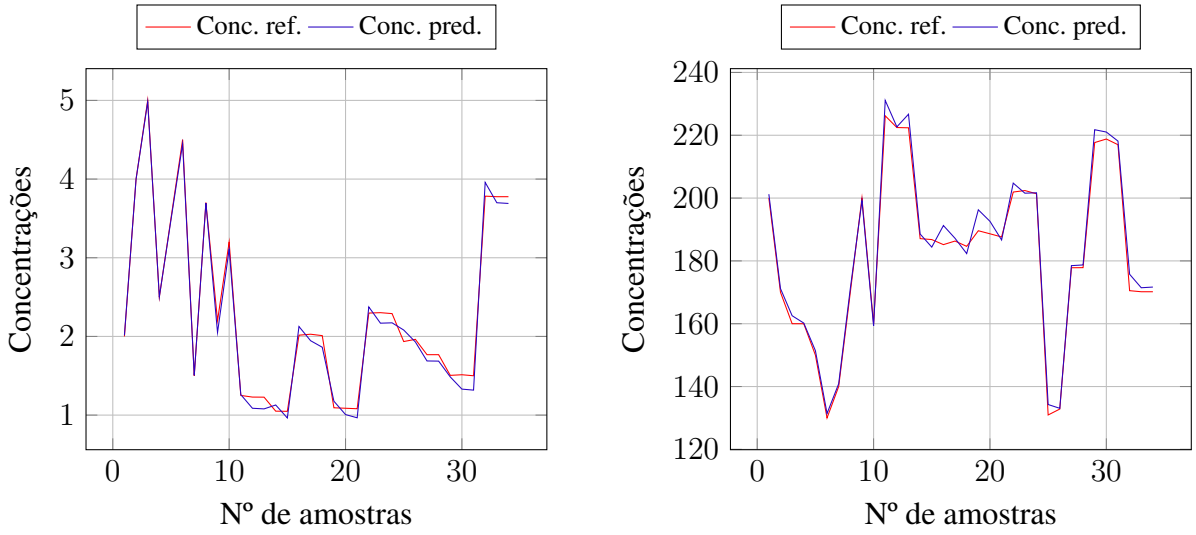
Fonte: Wagner Oliveira de Araujo.

A Figura 8 mostra o RMSEP da RLM em função das replicações *bootstrap*. A Figura 8a mostra o comportamento do RLM-*bagging* para o Mn e a Figura 8b mostra o RMSEP do RLM-

*bagging* para o Fe. Para o Mn, o RMSEP tende a se estabilizar entre $0{,}026$ e $0{,}030$ a partir de 15–20 replicações. Para o Fe, o RMSEP oscila entre $1{,}3$ e $2{,}2$ em todo o intervalo de 1 a 50 replicações, sem estabilização observável.

Duas hipóteses são compatíveis com esse comportamento: o Fe pode exigir mais de 50 replicações, ou sua predição pode ser mais difícil que a do Mn; o experimento não permite distingui-las. Os valores da Figura 8 foram obtidos com o conjunto de calibração completo, em análise de sensibilidade interna, e podem diferir dos valores do conjunto de teste independente reportados nas Tabelas 3 e 7.

Figura 9 – RMSEP para o Mn e Fe com RLM-*bagging*.




(a) Predição do Mn com RLM-*bagging*.

(b) Predição do Fe com RLM-*bagging*.

Fonte: Wagner Oliveira de Araujo.

A Figura 9 mostra o comportamento do RMSEP com RLM-*bagging*. A Figura 9a mostra o RMSEP com RLM-*bagging* do Mn e a Figura 9b mostra o RMSEP com RLM-*bagging* do Fe.

Observa-se desempenho variável do RLM-*bagging* em relação aos resultados da técnica RLM mostrados na Tabela 2. O RLM-*bagging* apresentou melhorias para Mn (redução de 5,4%) e Fe (redução de 3,3%), enquanto demonstrou desempenho inferior para Mo (aumento de 30,8%), Cr (aumento de 12,4%) e Ni (aumento de 4,3%). Os valores de RMSEP obtidos com o RLM-*bagging* foram $(0{,}0994)$ para o Mn e $(2{,}6597)$ para o Fe, como apresentado na Tabela 3.

A Tabela 3 mostra a comparação do erro do RLM-*bagging* em relação ao erro da RLM. O RLM-*bagging* apresentou melhorias apenas para Mn (redução de 5,42%) e Fe (redução de 3,33%), enquanto para Mo, Cr e Ni o erro aumentou em 30,76%, 12,44% e 4,32%, respectivamente. Em média, o erro do RLM-*bagging* foi 7,75% maior do que o da RLM sem reamostragem, evidenciando um desempenho inferior para a maioria dos analitos avaliados no estudo.

Analisados em conjunto, esses resultados indicam que a aplicação do *bagging* não resultou em uma melhoria consistente no desempenho preditivo do modelo de regressão linear múltipla para este conjunto de dados. O aumento da variância introduzido pelo processo de

Tabela 3 – RMSEP da RLM e RLM-*bagging* para cada elemento de aço-ligas.

| **Elementos** | **RLM** | **RLM-*bagging*** [a] |
|---|---|---|
| Mn | 0,1051 | 0,0994 |
| Mo | 0,2633 | 0,3443 |
| Cr | 1,0798 | 1,2141 |
| Ni | 1,6043 | 1,6736 |
| Fe | 2,7513 | 2,6597 |

Fonte: Wagner Oliveira de Araujo.
[a] Dezessete *bootstrap* utilizados. Parâmetros definidos sem otimização formal; ver Tabela 7 para resultados com validação cruzada 5-*fold*.

reamostragem comprometeu a acurácia global dos estimadores. Dessa forma, a RLM clássica manteve-se como a abordagem mais precisa e estável para a maioria dos elementos químicos considerados nesta análise experimental específica.

## 4.2 *Subagging*

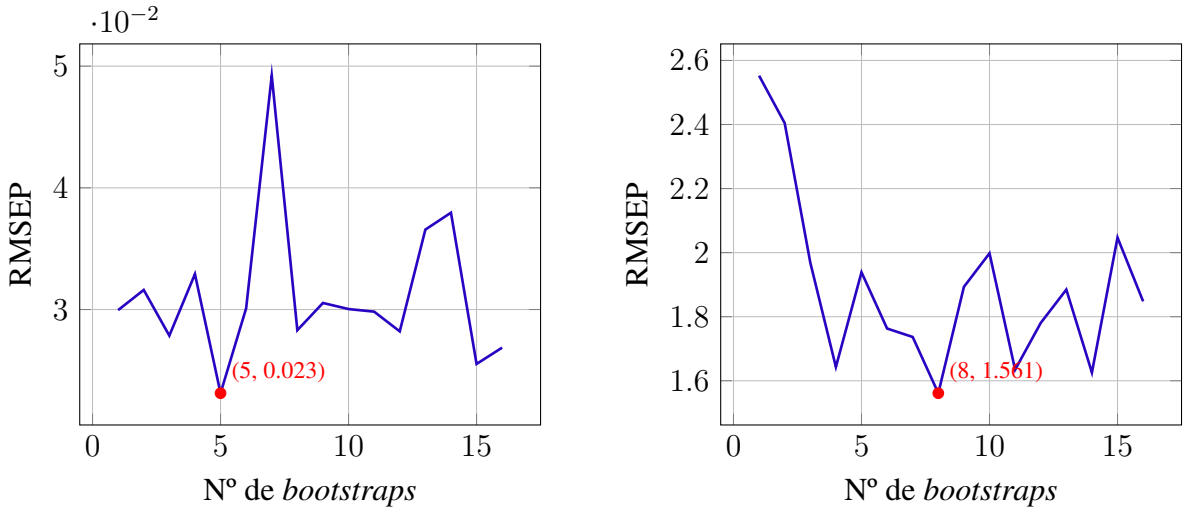
A seção *Subagging* apresenta os resultados obtidos por meio da aplicação da Regressão Linear Múltipla (RLM) sobre subconjuntos gerados via *bootstrap*, de forma semelhante ao *bagging*, porém sem reposição de amostras. O algoritmo *subagging* exige a definição de um parâmetro adicional, além do número de replicações. Enquanto no *bagging* o tamanho das amostras *bootstrap* permanece constante e igual ao do conjunto original, no *subagging* esse tamanho é variável, exigindo uma análise criteriosa da sua influência.

Para analisar a sensibilidade do *subagging* em relação a essas variações operacionais, o estudo conduz um experimento sistemático utilizando 16 amostras *bootstrap*, permitindo avaliar o comportamento do modelo sob diferentes configurações de subamostragem. Essa variação controlada busca identificar o impacto direto da redução do tamanho amostral sobre a acurácia preditiva da regressão. Os resultados detalhados desse processo de avaliação e a respectiva resposta preditiva da RLM são apresentados e comparados na Figura 10.

Observa-se nas Figuras 10a e 10b que o *subagging* apresenta sensibilidade ao número de amostras *bootstrap* utilizadas no algoritmo. Para a predição do Mn, o número ideal de amostras *bootstrap* é 5, resultando em RMSEP de 0,023. Para a predição do Fe, o número ideal de amostras *bootstrap* também é 8, com RMSEP de 1,561, observando-se uma ligeira piora acima desse valor.

A Tabela 4 mostra o erro do RLM-*subagging* comparado com o erro da RLM tradicional. O RLM-*subagging* apresentou melhorias para três dos cinco analitos testados: Mn (redução de 7,33%), Ni (redução de 0,25%) e Fe (redução de 0,77%). Para Mo e Cr, houve aumento do erro de 7,14% e 7,58%, respectivamente. Em média, considerando todos os analitos, o RLM-*subagging* apresentou desempenho praticamente equivalente ao da RLM tradicional, com variação média de apenas 1,27% no RMSEP.

Figura 10 – RMSEP em função das replicações *bootstrap* para RLM-*subagging*.




(a) Erros encontrados para cada replicação do Mn variando a quantidade de amostras *bootstrap*.

(b) Erros encontrados para cada replicação do Fe variando a quantidade de amostras *bootstrap*.

Fonte: Wagner Oliveira de Araujo.

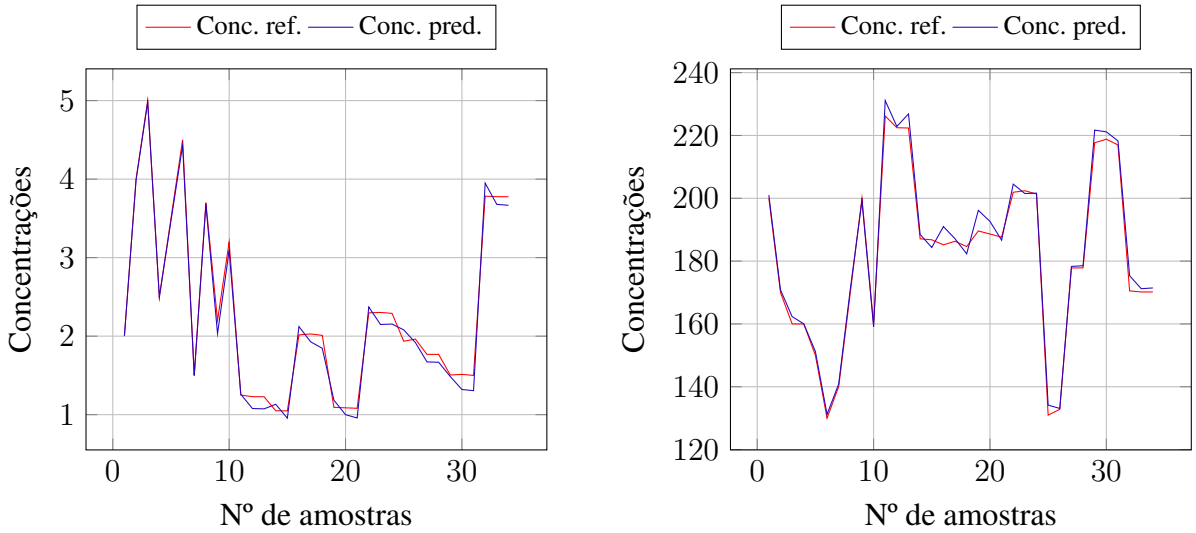
Tabela 4 – RMSEP por elemento de aço-ligas comparando RLM e RLM-*subagging*.

| **Elemento** | **RLM** | **RLM-*subagging*** |
|---|---|---|
| Mn | 0,1051 | 0,0974 [a] |
| Mo | 0,2633 | 0,2821 [b] |
| Cr | 1,0798 | 1,1616 [c] |
| Ni | 1,6043 | 1,6003 [d] |
| Fe | 2,7513 | 2,7302 [e] |

Fonte: Wagner Oliveira de Araujo.
[a] Cinco *bootstrap* utilizados.
[b] Dezesseis *bootstrap* utilizados.
[c] Doze *bootstrap* utilizados.
[d] Dez *bootstrap* utilizados.
[e] Oito *bootstrap* utilizados.

A Figura 11 mostra o comportamento do RMSEP com RLM-*subagging*. Nas Figuras 11a e 11b, é possível observar as predições dos elementos Mn e Fe.

Figura 11 – RMSEP do Mn e Fe com *subagging*.




(a) RMSEP do Mn com RLM-*subagging*.

(b) RMSEP do Fe com RLM-*subagging*.

Fonte: Wagner Oliveira de Araujo.

Tabela 5 – Comparação do RMSEP por elemento para diferentes técnicas e proporções de calibração.

| | **RLM** | | **RLM-*bagging*** | | **RLM-*subagging*** | |
|---|---|---|---|---|---|---|
| **Elemento** | **50%** | **80%** | **50%** | **80%** | **50%** | **80%** |
| Mn | 0,2029 | 0,1119 | 0,1742 | 0,1142 | 0,2205 | 0,1108 |
| Mo | 1,5651 | 0,2546 | 0,7338 | 0,3337 | 0,5944 | 0,2809 |
| Cr | 0,7215 | 1,0978 | 1,4512 | 1,1661 | 1,8984 | 1,1807 |
| Ni | 28,4381 | 1,5865 | 28,1847 | 1,3281 | 28,0398 | 1,3724 |
| Fe | 3,2962 | 2,5795 | 5,9944 | 2,5367 | 7,2990 | 2,4871 |

Fonte: Wagner Oliveira de Araujo.

Em geral, a utilização da proporção de calibração de 80% resultou em menor RMSEP em 14 das 15 combinações analito $\times$ técnica avaliadas no estudo. Para RLM-*bagging* e RLM-*subagging*, a proporção de 80% foi superior em todos os cinco elementos considerados. Para a RLM clássica, a proporção de 80% apresentou melhor desempenho em quatro dos cinco elementos (Mn, Mo, Ni e Fe), demonstrando a relevância da dimensão do conjunto de treinamento para a estimativa dos parâmetros do modelo.

A única exceção observada foi para o Cromo (Cr), onde a proporção de 50% de calibração produziu menor RMSEP ($0{,}7215$ contra $1{,}0978$). Contudo, a prevalência do conjunto de treinamento maior para a maioria dos cenários é compatível com a expectativa teórica da literatura de que conjuntos de calibração mais amplos favorecem a generalização e a estabilidade preditiva do modelo. Dessa forma, a expansão da base de treinamento consolidou-se como o arranjo preferencial para o protocolo analítico.

Quanto ao melhor desempenho absoluto por elemento, o RLM-*subagging* com 80% de

calibração obteve o menor RMSEP para Mn ($0{,}1108$) e Fe ($2{,}4871$), a RLM com 80% obteve o menor erro para Mo ($0{,}2546$), o RLM-*bagging* com 80% para Ni ($1{,}3281$), e a RLM com 50% para Cr ($0{,}7215$).

A técnica que atingiu o menor RMSEP (Tabela 6) varia em função do elemento, destacando que não há uma solução única para todos os casos:

Tabela 6 – Configuração de desempenho superior e RMSEP mínimo.

| **Elemento** | **Técnica de Desempenho Superior** | **Menor RMSEP** | **Proporção** (%) |
|---|---|---|---|
| Mn | RLM | 0,1108 | 80 |
| Mo | RLM | 0,2546 | 80 |
| Cr | RLM | **0,7215** | 50 |
| Ni | RLM | 1,3281 | 80 |
| Fe | RLM-*subagging* | **2,4871** | 80 |

Fonte: Wagner Oliveira de Araujo.
**Nota**:Resultados baseados na Tabela 5 (parâmetros sem otimização formal por validação cruzada). Para comparação com parâmetros otimizados, ver Tabela 7.

**a) RLM (Clássico):** Obteve a melhor performance para Mo ($0{,}2546$ com 80% de calibração) e Cr ($0{,}7215$ com 50% de calibração). Para estes elementos, a relação preditiva é relativamente simples e linear, não requerendo a complexidade adicional dos métodos de *ensemble*.

**b) RLM-*bagging*:** Demonstrou superioridade para Ni, atingindo $1{,}3281$ com 80% de calibração, superando a RLM ($1{,}5865$) e o RLM-*subagging* ($1{,}3724$).

**c) RLM-*subagging*:** Obteve o melhor desempenho para Mn ($0{,}1108$ com 80% de calibração) e Fe ($2{,}4871$ com 80% de calibração). Estes resultados indicam que estes elementos se beneficiam da agregação de modelos com subamostragem para reduzir a variância das predições.

As técnicas de ensemble (RLM-*bagging* e RLM-*subagging*) são construídas através da agregação de $m$ modelos individuais, gerados por *bootstrap*. O RMSEP final é determinado pela média das previsões desses $m$ modelos. O desafio na aplicação destas técnicas é otimizar o parâmetro $m \in \{5, 10, 20, 50\}$ para encontrar o ponto onde a redução da variância é maximizada sem desperdiçar recursos computacionais.

Os menores RMSEP reportados na Tabela 5 para as técnicas de *ensemble* são resultados diretos da otimização de $m$ dentro do conjunto testado:

**a)** Para Mn e Fe, onde o RLM-*subagging* foi o vencedor absoluto, os valores ótimos de $m$ foram $m = 5$ e $m = 8$, respectivamente. Estes valores relativamente baixos sugerem que poucos modelos agregados já foram suficientes para estabilizar e reduzir o erro preditivo,

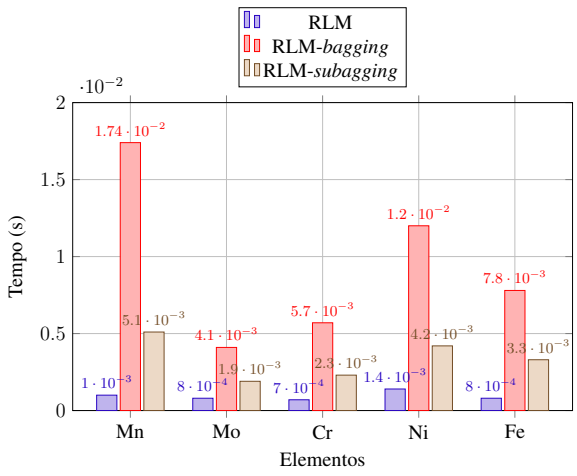
superando a performance do RLM clássico. Para Ni, o RLM-*bagging* com $m = 17$ obteve o menor RMSEP (1,3281).

**b)** Para Mo e Cr, nenhuma técnica de *ensemble* superou a RLM clássica, independentemente do valor de $m$ utilizado. Isso indica que, para estes elementos, a agregação de modelos introduz mais variância do que reduz, e a relação preditiva linear simples é mais adequada.

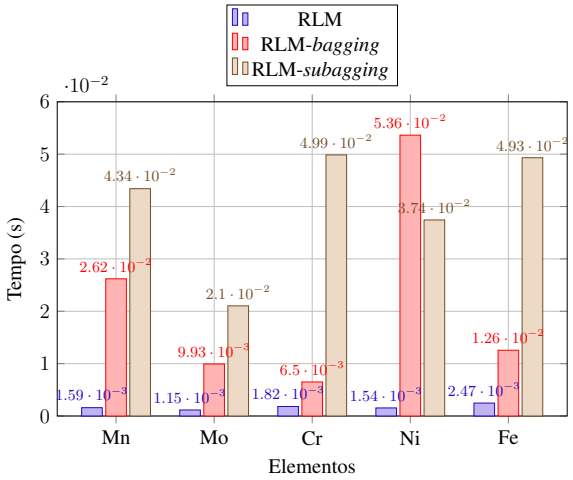
Em síntese, a análise demonstra que a escolha ideal do modelo não é apenas uma questão da técnica, mas também da proporção de calibração e, criticamente, da otimização do número de replicações ($m$) nas abordagens de *ensemble* para cada elemento químico.

Figura 12 – Tempo de execução dos algoritmos RLM, RLM-*bagging* e RLM-*subagging*.

(a) 50% das amostras




(b) 80% das amostras




Fonte: Wagner Oliveira de Araujo.

A Figura 12 apresenta os tempos de execução dos três algoritmos avaliados — RLM clássica, RLM-*bagging* e RLM-*subagging* — para cada elemento químico (Mn, Mo, Cr, Ni e Fe), com 50% e 80% das amostras disponíveis.

A comparação entre os dois cenários mostra o crescimento do tempo de execução com o aumento do conjunto de treinamento. Para a RLM clássica, o aumento médio foi de aproximadamente 2,0 vezes, variando de 1,1 × (Ni) a 3,1 × (Fe). O RLM-*bagging* apresentou aumento médio de cerca de 2,2 vezes, variando de 1,1 × (Cr) a 4,5 × (Ni).

O maior aumento foi observado no RLM-*subagging*, com média de aproximadamente 13,0 vezes ao passar de 50% para 80% das amostras, variando de 8,5 × (Mn) a 21,7 × (Cr). Esse aumento é compatível com a sensibilidade do algoritmo de subamostragem ao tamanho do conjunto de treinamento e com a possível seleção de valores maiores de $m^*$ e $k^*$ nos conjuntos maiores; os valores selecionados em cada cenário não são reportados nesta seção.

Em termos absolutos, com 80% das amostras, o RLM-*bagging* foi mais rápido que o RLM-*subagging* em quatro dos cinco elementos (Mn, Mo, Cr e Fe). A soma dos tempos do RLM-*subagging* (0,201 s) foi 1,85 vez maior do que a do RLM-*bagging* (0,109 s). Em contrapartida,

ao reduzir a base de treinamento para 50% das amostras, o RLM-*subagging* superou a velocidade do RLM-*bagging* nos cinco elementos analisados, demonstrando sensibilidade ao volume de dados de entrada.

Esse resultado indica que, neste conjunto de dados, a vantagem temporal do RLM-*subagging* não se manteve com o aumento do tamanho amostral, variando de acordo com a proporção de calibração adotada. Essa limitação computacional deve ser considerada na escolha da técnica em aplicações operacionais com conjuntos de dados maiores. Estudos futuros podem investigar o comportamento de escala dessa abordagem em matrizes espectrométricas de maior dimensão, avaliando o impacto no tempo de processamento do modelo.

## 4.3 *Bagging* × *Subagging*

Os valores de RMSEP apresentados na (Tabela 7) diferem dos reportados na (Tabela 3) porque esta última apresenta resultados com número fixo de *bootstraps* sem otimização, enquanto a (Tabela 7) adota parâmetros otimizados por validação cruzada 5-*fold* e incorpora as três técnicas comparativamente com métricas ampliadas.

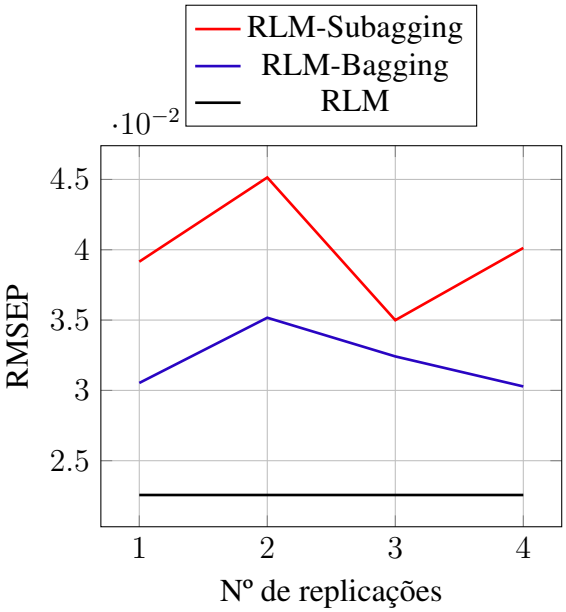
Com a implementação do *bagging* e do *subagging*, observou-se melhora das predições obtidas com a RLM para alguns analitos específicos. O RLM-*bagging* apresentou melhor desempenho para Fe, enquanto o RLM-*subagging* foi superior para Mn e Ni. Para Cr e Mo, a RLM clássica manteve o menor RMSEP. Entre as técnicas *ensemble*, o RLM-*subagging* superou o RLM-*bagging* no Cr, demonstrando um comportamento preditivo diferenciado e dependente da estrutura espectral de cada elemento analisado no estudo.

Contudo, nenhum dos dois métodos *ensemble* superou a RLM clássica para o Cromo, indicando que essas abordagens não garantem melhorias universais de acurácia. Dessa forma, é fundamental comparar o desempenho do RLM-*bagging* e do RLM-*subagging* considerando as características específicas de cada analito. Avaliações criteriosas por elemento garantem a seleção do modelo mais adequado para a matriz de aço-liga, assegurando a confiabilidade das estimativas analíticas geradas pelo sistema computacional de calibração multivariada.

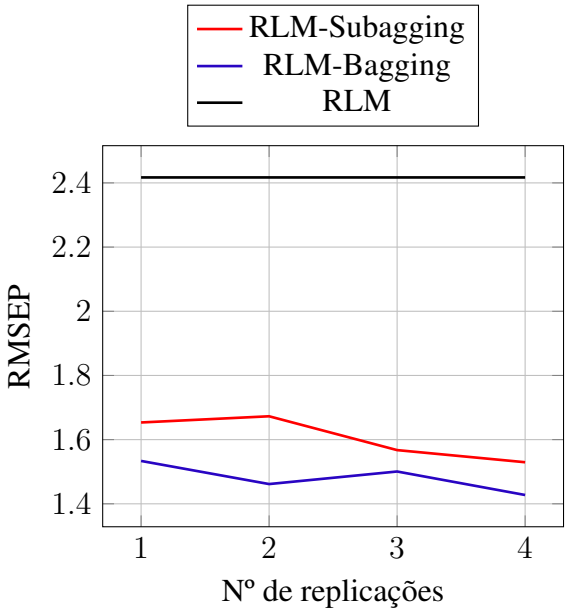
A variação do RMSEP em relação à RLM difere entre os analitos: de $-5{,}4\%$ (Mn) a $+30{,}8\%$ (Mo) para o RLM-*bagging* (Tabela 3) e de $-7{,}3\%$ (Mn) a $+7{,}6\%$ (Cr) para o RLM-*subagging* (Tabela 4). Para avaliar comparativamente o desempenho dessas técnicas, foi feita uma predição com o RLM-*bagging* e RLM-*subagging*, utilizando o mesmo conjunto de amostras *bootstrap*, no intuito de comparar os erros de predição das duas técnicas apresentadas na Figura 13.

A RLM clássica gera uma única predição, enquanto tanto o RLM-*bagging* quanto o RLM-*subagging* utilizam múltiplas reamostragens para produzir predições agregadas. Na análise comparativa, o RLM-*subagging* foi avaliado com diferentes quantidades de amostras *bootstrap*, gerando um conjunto de 16 predições, sendo uma para cada quantidade de *bootstrap* testada. A Tabela 7 apresenta os resultados dessa comparação direta entre as técnicas, utilizando o mesmo conjunto de dados de validação.

Figura 13 – RMSEP para todas as técnicas.




(a) RMSEP obtido com as três técnicas aplicadas ao Mn.




(b) RMSEP obtido com as três técnicas aplicadas ao Fe.

Fonte: Wagner Oliveira de Araujo.

Observa-se que a RLM clássica mantém desempenho superior para Mo e Cr, enquanto o RLM-*subagging* apresenta melhor resultado para Mn e Ni, e o RLM-*bagging* para Fe. Entre as técnicas *ensemble*, o RLM-*subagging* apresenta melhor resultado apenas para Cr em relação ao RLM-*bagging*, sem contudo superar a RLM clássica nesse elemento. Os resultados desta comparação são também apresentados nas Figuras 13a e 13b.

Os resultados comparativos entre as três técnicas avaliadas para todos os analitos são apresentados na Tabela 7. Diferentemente da análise anterior, os parâmetros $m^*$ e $k^*$ de cada método foram determinados por validação cruzada 5-*fold* aplicada exclusivamente ao conjunto de calibração, garantindo que o conjunto de teste não influenciasse a seleção dos hiperparâmetros e assegurando a comparabilidade entre os métodos.

Os resultados revelam que não existe uma técnica universalmente superior para todos os analitos considerados no estudo. Para o Manganês (Mn), o RLM-*subagging* ($k^* = 80\%$, $m^* = 10$) apresentou o menor RMSEP ($0{,}0966$), representando uma redução de 8,1% no erro em relação à RLM clássica, além de demonstrar alta eficiência computacional ao ser 4,0 vezes mais rápido do que o RLM-*bagging*, constituindo-se na opção mais vantajosa para a calibração desse elemento.

Para o Ferro (Fe), o RLM-*bagging* ($m^* = 30$) obteve o melhor desempenho preditivo (RMSEP = $2{,}6406$), alcançando uma melhoria de 4,0% sobre o modelo clássico de regressão. Por sua vez, para o Níquel (Ni), o RLM-*subagging* ($k^* = 60\%$, $m^* = 10$) apresentou resultado marginalmente superior (RMSEP = $1{,}6003$), com uma diferença de apenas 0,25% em relação à RLM clássica, mas garantindo um tempo de execução 6,3 vezes menor do que o RLM-*bagging*.

Tabela 7 – Desempenho comparativo das técnicas de calibração multivariada para elementos químicos em aço-liga.

| **Elemento** | **Técnica** | **RMSEP** | **MAE** | **R²** | **Tempo** (ms) | **Parâmetros** | |
|---|---|---|---|---|---|---|---|
| | | | | | | $k^*$ (%)[b] | $m^*$[c] |
| Mn | RLM | 0,1051 | 0,0887 | 0,9907 | 2,4000 | — | — |
| | RLM-*bagging* | 0,1072 | 0,0908 | 0,9903 | 68,2000 | — | 30 |
| | RLM-*subagging*[a] | **0,0966** | **0,0805** | **0,9922** | 17,1000 | 80 | 10 |
| Mo | RLM[a] | **0,2633** | **0,1952** | **0,9882** | 2,4000 | — | — |
| | RLM-*bagging* | 0,3388 | 0,2722 | 0,9805 | 67,5000 | — | 30 |
| | RLM-*subagging* | 0,2879 | 0,2201 | 0,9859 | 46,8000 | 80 | 30 |
| Cr | RLM[a] | **1,0798** | **0,8654** | **0,9938** | 2,5000 | — | — |
| | RLM-*bagging* | 1,2089 | 0,9830 | 0,9922 | 44,5000 | — | 20 |
| | RLM-*subagging* | 1,1294 | 0,9010 | 0,9932 | 46,8000 | 80 | 30 |
| Ni | RLM | 1,6043 | 1,3256 | 0,9837 | 2,3000 | — | — |
| | RLM-*subagging*[a] | **1,6003** | **1,2409** | **0,9838** | 10,6000 | 60 | 10 |
| | RLM-*bagging* | 1,7085 | 1,4122 | 0,9815 | 66,6000 | — | 30 |
| Fe | RLM | 2,7513 | 2,1865 | 0,9892 | 13,5000 | — | — |
| | RLM-*bagging*[a] | **2,6406** | **2,0246** | **0,9900** | 97,2000 | — | 30 |
| | RLM-*subagging* | 2,6509 | 2,0578 | 0,9900 | 41,2000 | 80 | 30 |

Fonte: Wagner Oliveira de Araujo.
Nota: Sinal convencional utilizado:
— Não se aplica dado numérico.
[a] Melhor desempenho em RMSEP. Conjunto de teste único para avaliação final.
[b] Fração de subamostra $k^*$ em porcentagem otimizados por validação cruzada 5-*fold* aplicada exclusivamente ao conjunto de calibração (*seed* = 42) *bootstrap*).
[c] Parâmetros $m^*$ número de replicações *bootstrap*.

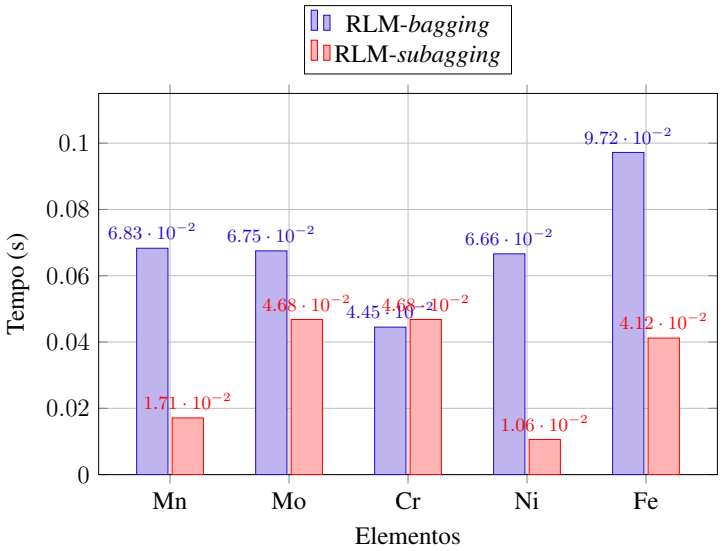
Em contrapartida, para o Molibdênio (Mo) e o Cromo (Cr), a RLM clássica demonstrou desempenho superior (RMSEP de 0,2633 e 1,0798, respectivamente), indicando que a complexidade adicional introduzida pelas estratégias de *ensemble* não resultou em menor RMSEP para esses analitos. Esses resultados são compatíveis com uma relação preditiva aproximadamente linear para Mo e Cr, hipótese não testada diretamente neste trabalho, em que a agregação de modelos não trouxe ganho de RMSEP.

A análise do custo computacional evidencia que o RLM-*subagging* configura-se como alternativa balanceada entre precisão e eficiência, sendo consistentemente mais rápido que o RLM-*bagging* para todos os elementos avaliados. Observa-se ainda que o parâmetro $k^* = 80\%$ revelou-se ótimo para o *subagging* em quatro dos cinco elementos, sugerindo que subamostras maiores favorecem a estabilidade do modelo neste contexto específico de dados espectrométricos de aço-ligas.

A Figura 14 apresenta os tempos de execução dos algoritmos RLM-*bagging* e RLM-*subagging* para cada elemento, utilizando os parâmetros $m^*$ e $k^*$ determinados por validação cruzada 5-*fold*. Diferentemente da análise anterior, os parâmetros agora variam por elemento — refletindo configurações otimizadas por validação cruzada —, o que torna a comparação de tempos diretamente atrelada ao custo real de cada configuração de melhor desempenho preditivo.

Em quatro dos cinco elementos avaliados (Mn, Mo, Ni e Fe), o RLM-*subagging* foi

Figura 14 – Tempo de execução dos algoritmos RLM-*bagging* e RLM-*subagging* com parâmetros otimizados por validação cruzada 5-*fold*.




Fonte: Wagner Oliveira de Araujo.

mais rápido que o RLM-*bagging*. A razão média entre os tempos (*bagging*/*subagging*) foi 3,0, variando de 0,95 (Cr) a 6,3 (Ni); a soma dos tempos dos cinco elementos é 52,8% menor no *subagging* (162,5 ms contra 344,0 ms).

As maiores diferenças ocorreram para o Ni (6,3 vezes mais rápido, com $k^* = 60\%$ e $m^* = 10$) e para o Mn (4,0 vezes mais rápido, com $k^* = 80\%$ e $m^* = 10$). Para Mo e Fe, as razões foram de 1,4 e 2,4, respectivamente. A menor duração do *subagging* é compatível com o uso de subconjuntos menores ($k^* \leq 80\%$ das amostras), que reduzem a dimensão de cada problema de mínimos quadrados; este trabalho não isolou esse efeito experimentalmente.

O Cromo (Cr) é o único elemento em que o RLM-*subagging* foi mais lento que o RLM-*bagging* (46,8 ms contra 44,5 ms; diferença de 2,3 ms, ou 5,2%). Esse resultado é consistente com os parâmetros selecionados por validação cruzada (menor $\text{RMSEP}_{\text{CV}}$): $m^* = 20$ para o *bagging* e $m^* = 30$ com $k^* = 80\%$ para o *subagging*.

Para o Fe, o RLM-*bagging* obteve o menor RMSEP (2,6406), apresentando desempenho superior ao modelo clássico de regressão (63). Para o Cr, apesar da sobreposição espectral com linhas do Fe e do Ni, nenhuma técnica de *ensemble* superou a RLM clássica (RMSEP = 1,0798). O comportamento diferenciado entre esses analitos reflete suas características espectrais individuais no conjunto de dados, demonstrando que a resposta ao processo de reamostragem varia conforme o elemento químico considerado.

Para o Mo, a RLM clássica também obteve o menor RMSEP (0,2633), reforçando que a introdução de modelos *ensemble* não garantiu ganho de acurácia preditiva para todos os elementos. Por outro lado, para o Mn, o RLM-*subagging* alcançou o menor RMSEP (0,0966), representando uma redução de 8,1% no erro em relação à RLM. A verificação das causas físicas subjacentes e a decomposição formal de viés e variância constituem tópicos sugeridos para

investigações futuras.

Em relação à RLM clássica, o custo computacional dos *ensembles* é maior: o RLM-*bagging* foi, em média, 22,1 vezes mais lento e o RLM-*subagging* até 19,5 vezes mais lento (Mo). Comparado ao RLM-*bagging*, o RLM-*subagging* obteve menor RMSEP em quatro elementos (Mn, Mo, Cr e Ni) e menor tempo em quatro (Mn, Mo, Ni e Fe), superando-o simultaneamente nos dois critérios em Mn, Mo e Ni.

# 5 REQUISITOS FUNCIONAIS

Esta seção apresenta o levantamento estruturado dos requisitos funcionais da aplicação *Regressão Espectral com Métodos de Ensemble (Bagging & Subagging)*, desenvolvida na plataforma Streamlit. A análise segue a estrutura proposta em quatro seções: Inventário de funcionalidades, Classificação por prioridade, Fluxos de uso e Requisitos não funcionais.

## 5.1 Levantamento de Requisitos Funcionais

### 5.1.1 Inventário das Funcionalidades Existentes

#### 5.1.1.1 Funções do Sistema

No Quadro 3 resume as funcionalidades identificadas na aplicação, categorizadas por tipo de operação.

Quadro 3 – Inventário das funções do sistema.

| **Categoria** | **Funcionalidade** | **Descrição Técnica** |
|---|---|---|
| Cálculos | RLM Simples (*Baseline*) | Regressão Linear Múltipla padrão sem técnicas de *ensemble* |
| Cálculos | RLM + *Bagging* | *Ensemble* com amostragem *com* reposição (*bootstrap*) |
| Cálculos | RLM + *Subagging* + CV[a] | *Ensemble* com amostragem *sem* reposição + otimização por validação cruzada 5-*Fold* |
| Configuração | Parâmetro *m* (*Bags*) | Define número de amostras/*ensembles* para o método *Bagging* |
| Configuração | Parâmetro *k* (*Subagging*) | Define fração ou quantidade para amostragem no *Subagging* |
| Visualização | Métricas de Desempenho | Exibição de $R^2$, RMSEP, MAE calculados no conjunto de teste |
| Visualização | Gráficos Comparativos | *Scatter plots* (previsto $\times$ real), resíduos e comparação entre modelos |
| Organização | Abas de Navegação | Separação lógica: “Arquivos”, “Resultados”, “Gráficos” |
| Exportação | Métricas (.csv) e log (.txt) | `st.download_button` na aba Resultados |

Fonte: Wagner Oliveira de Araujo.
[a] *Cross Validation* - Validação Cruzada.

#### 5.1.1.2 Entradas Necessárias (*Inputs*)

No Quadro 4 detalha os dados e parâmetros exigidos para execução da aplicação.

Quadro 4 – Entradas necessárias para execução do sistema.

| **Tipo** | **Nome** | **Formato** | **Obrig.** | **Descrição** |
|---|---|---|---|---|
| Arquivo | `Xcal` | `.txt, .csv, .xlsx` | Sim | Matriz de espectros de calibração |
| Arquivo | `Ycal` | `.txt, .csv, .xlsx` | Sim | Vetor de valores de referência |
| Arquivo | `Xteste` | `.txt, .csv, .xlsx` | Sim | Matriz de espectros para predição |
| Arquivo | `Yteste` | `.txt, .csv, .xlsx` | Sim | Vetor de valores reais para validação |
| Parâmetro | $m$ | Inteiro $\geq 1$ | Condic.[a] | Usado apenas nos métodos de *ensemble* |
| Parâmetro | $k$ | Inteiro $\geq 1$ | Condic.[a] | Usado apenas no método *Subagging* |
| Configurado | Validação Cruzada | Fixo: 5-*Fold* | Auto | Parâmetro interno de otimização |

[a] Condicional: habilitados apenas quando o usuário seleciona métodos de ensemble.
Fonte: Wagner Oliveira de Araujo.

#### 5.1.1.3 Saídas Esperadas (*Outputs*)

No Quadro 5 apresenta os resultados gerados pelo sistema após processamento.

Quadro 5 – Saídas esperadas do sistema.

| **Tipo** | **Conteúdo** | **Formato de Exibição** | **Destinatário** |
|---|---|---|---|
| Numérico | $R^2$, RMSEP, MAE (conjunto de teste) | Quadro ou *cards* destacados | Usuário analista |
| Gráfico | Previsto $\times$ Real | *Scatter plot* com reta de ajuste | Usuário analista |
| Gráfico | Resíduos da Predição | Gráfico de dispersão ou histograma | Usuário analista |
| Comparativo | Desempenho: RLM $\times$ *Bagging* $\times$ *Subagging* | Quadro comparativa ou gráfico de barras | Usuário analista |
| *Feedback* | Mensagens de erro/validação | Alertas em destaque | Usuário final |
| *Status* | Indicador de processamento | *Spinner* ou barra de progresso | Usuário final |

Fonte: Wagner Oliveira de Araujo.

### 5.1.2 Classificação por Prioridade

Os requisitos foram classificados utilizando a técnica *Must, Should, Could, Won't have* (MoSCoW)[1](64) adaptada, conforme descrito nas seções seguintes.

#### 5.1.2.1 Essenciais (*Must Have*)

O Quadro 6 lista os requisitos essenciais, indispensáveis à versão mínima, pois garantem a validação dos dados e avaliação dos modelos.

Quadro 6 – Requisitos essenciais do sistema.

| **ID** | **Requisito** | **Justificativa** |
|---|---|---|
| E01 | *Upload* e validação dos 4 arquivos de entrada | Dados são pré-requisito para qualquer cálculo estatístico |
| E02 | Execução da RLM Simples (*baseline*) | Referência obrigatória para comparação de desempenho dos *ensembles* |
| E03 | Cálculo de métricas básicas ($R^2$, RMSEP, MAE) | Mínimo necessário para avaliação quantitativa da qualidade do modelo |
| E04 | Validação de consistência dimensional (X $\times$ Y) | Previne erros de execução e geração de resultados inválidos |
| E05 | Exibição clara dos resultados numéricos | Permite decisão técnica fundamentada em evidências quantitativas |

Fonte: Wagner Oliveira de Araujo.

#### 5.1.2.2 Desejáveis (*Should Have*)

O Quadro 7 lista os requisitos desejáveis, que ampliam usabilidade e reprodutibilidade, mas não são indispensáveis à versão mínima.

Quadro 7 – Requisitos desejáveis do sistema.

| **ID** | **Requisito** | **Benefício** |
|---|---|---|
| D01 | Gráficos interativos (*zoom*, *tooltip*, *download*) | Facilita exploração visual detalhada e apresentação de resultados |
| D02 | Exportação em PDF e PNG (CSV e TXT já implementados) | Permite documentação, compartilhamento e reprodutibilidade |
| D03 | *Tooltips* explicativos para *m*, *k*, *Bagging*/*Subagging* | Reduz curva de aprendizado para usuários não especialistas |
| D04 | Pré-processamento básico (normalização, escalonamento) | Melhora robustez dos modelos com dados espectrais reais |
| D05 | Histórico de execuções na sessão | Permite comparar configurações sem reenviar arquivos |

Fonte: Wagner Oliveira de Araujo.

[1] MoSCoW é uma técnica de priorização de requisitos dividida em quatro categorias: *Must have* (obrigatórios), *Should have* (importantes, mas não críticos), *Could have* (desejáveis) e *Won't have (this time)* (excluídos desta versão). A letra "o" é adicionada apenas para facilitar a pronúncia.

#### 5.1.2.3 Extras (*Could Have*)

O Quadro 8 apresenta os requisitos extras, destinados a fases futuras, como autenticação, integrações externas e *dashboards*, condicionados à validação.

Quadro 8 – Requisitos extras para fases futuras.

| ID | Requisito | Cenário de Uso |
|---|---|---|
| X01 | Autenticação de usuários e projetos privados | Ambientes corporativos ou de pesquisa colaborativa |
| X02 | Integração com Interface de Programação de Aplicações externas (ex: KNIME[a], scripts Python) | Pipelines automatizadas de quimiometria |
| X03 | Suporte a formatos adicionais (.mat) | Flexibilidade para ingestão de dados de diferentes fontes |
| X04 | Otimização automática de hiperparâmetros | Dispensa o usuário de informar *m* e *k* manualmente |
| X05 | *Dashboard* comparativo multi-rodada | Análise estatística de estabilidade dos modelos *ensemble* |

Fonte: Wagner Oliveira de Araujo.
[a] *Konstanz Information Miner* (KNIME) (65) é uma plataforma de código aberto voltada para a ciência de dados, análise e automação de processos.

### 5.1.3 Fluxos de Uso

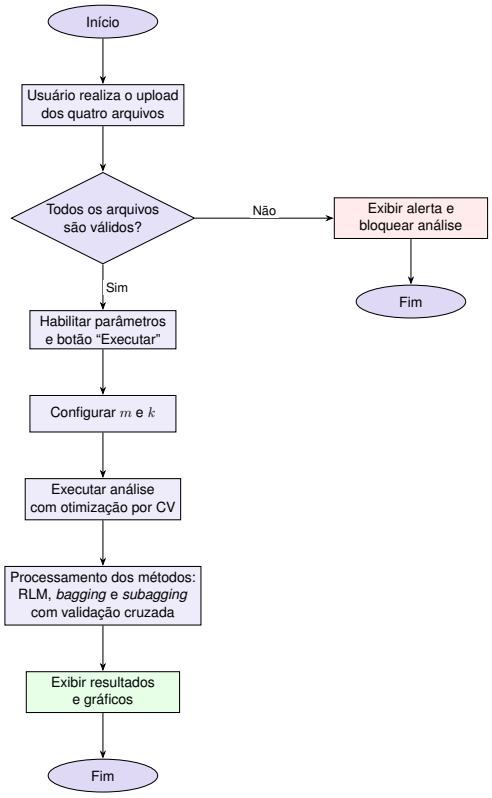
#### 5.1.3.1 Fluxo Principal: Execução de Análise Completa

A Figura 15 apresenta o fluxograma do fluxo principal de interação do usuário com o sistema, ilustrando as etapas percorridas desde o acesso à aplicação Streamlit até a obtenção dos resultados finais. O fluxo tem início com o *upload* dos quatro arquivos de entrada — as matrizes `Xcal` e `Xteste` e os vetores `Ycal` e `Yteste` —, seguido pela configuração dos hiperparâmetros $m$ (número de *bags*) e $k$ (fração de subamostragem) na aba **Configuração**.

Uma vez validados os arquivos e os parâmetros informados, o usuário aciona o botão de execução, que dispara o processamento em `models.py`: primeiro a otimização por validação cruzada 5-*fold* (`find_optimal_parameters()`) e, em seguida, a avaliação final dos três modelos — RLM Simples, RLM + *Bagging* e RLM + *Subagging* — sobre o conjunto de teste independente (`run_full_analysis()`).

Ao término do processamento, os resultados numéricos (RMSEP, MAE, $R^2$ e tempo de execução) tornam-se disponíveis na aba **Resultados**, enquanto os gráficos comparativos de valores reais *versus* preditos e o gráfico de dispersão são renderizados na aba **Gráficos**, encerrando o ciclo de interação representado no fluxograma.

Figura 15 – Fluxograma do processo de execução da análise.




Fonte: Wagner Oliveira de Araujo.

#### 5.1.3.2 Fluxos Alternativos e Tratamento de Exceções

O Quadro 9 apresenta exceções, como arquivo inválido ou parâmetros incorretos, detalhando a resposta do sistema e a ação esperada.

Quadro 9 – Fluxos alternativos e tratamento de exceções.

| **Cenário** | **Gatilho** | **Comportamento do Sistema** | **Ação do Usuário** |
|---|---|---|---|
| Arquivo inválido | Upload com formato/dimensão incorreta | Alerta vermelho + destaque no campo problemático | Corrigir arquivo e reenviar |
| Parâmetros fora do intervalo | Inserir $m$ ou $k \leq 0$ ou excessivamente alto | Validação em tempo real + mensagem orientativa | Ajustar para valores válidos |
| *Timeout* de processamento | Dados muito grandes ou carga elevada | Mensagem: "Processamento demorado..." | Otimizar entrada ou repetir depois |
| Navegação sem executar | Usuário muda de aba antes de clicar | Mantém estado dos uploads (persistência) | Retornar e concluir análise |

Fonte: Wagner Oliveira de Araujo.

#### 5.1.3.3 Pontos Críticos de Usabilidade

O Quadro 10 identifica pontos críticos de usabilidade, como upload múltiplo e parâmetros abstratos, associando riscos a estratégias de mitigação.

Quadro 10 – Pontos críticos de usabilidade e estratégias de mitigação.

| **Ponto** | **Risco** | **Sugestão de Mitigação** |
|---|---|---|
| Upload múltiplo obrigatório | Esquecer um dos 4 arquivos e travar o fluxo | *Checklist* visual + botão habilitado apenas quando todos estiverem OK |
| Parâmetros *m* e *k* abstratos | Inserir valores inadequados e obter resultados ruins | *Sliders* com valores recomendados + *tooltip* prático |
| Processamento sem *feedback* claro | Achar que a aplicação travou | Barra de progresso com etapas descritivas |
| Gráficos sem legenda ou escala | Dificulta interpretação em telas menores | Legendas fixas, eixos nomeados, opção de ampliar |
| Resultados não exportáveis | Impede documentação externa | Botão "Baixar Relatório" com métricas + gráficos |

Fonte: Wagner Oliveira de Araujo.

### 5.1.4 Requisitos Não Funcionais

#### 5.1.4.1 Performance

O Quadro 11 define requisitos de performance, com limites de tempo para upload, cálculo, renderização gráfica e resposta da interface.

Quadro 11 – Requisitos de performance do sistema.

| **Requisito** | **Especificação** | **Critério de Aceite** |
|---|---|---|
| Tempo de *upload* | Arquivos $\leq 200$ MB | $< 30$ s em conexão 10 Mbps |
| Tempo de cálculo (RLM) | *Dataset* típico ($n \leq 500$, $p \leq 1000$) | $< 10$ s |
| Tempo de cálculo (*Ensemble*) | *Bagging*/*Subagging* com $m \leq 100$ | $< 2$ min com CV 5-*Fold* |
| Renderização de gráficos | Após conclusão do cálculo | $< 3$ s |
| Responsividade da Interface do Usuário | Interações (mudar aba, ajustar parâmetro) | $< 500$ ms de *feedback* visual |

Fonte: Wagner Oliveira de Araujo.

#### 5.1.4.2 Usabilidade

O Quadro 12 define requisitos de usabilidade, abrangendo acessibilidade, responsividade, clareza textual, consistência visual e orientação para recuperação de erros.

Quadro 12 – Requisitos de usabilidade do sistema.

| **Requisito** | **Especificação** | **Critério de Aceite** |
|---|---|---|
| Acessibilidade | Compatível com WCAG 2.1 Nível AA[a] | Contraste adequado, *labels*, navegação por teclado |
| Responsividade | Funcional em *desktop*, *tablet* e *mobile* ($\geq$ 768 px) | *Layout* adaptativo sem perda de funcionalidade crítica |
| Clareza textual | Mensagens, rótulos e erros em português claro | Teste com 3 usuários: $\geq$ 90% de acerto na interpretação |
| Consistência | Padrões visuais alinhados ao Streamlit | Botões, cores e ícones seguem guia de estilo |
| Recuperação de erro | Mensagens orientam a correção | Ex: "Ycal.txt tem 99 linhas, mas Xcal.txt tem 100..." |

[a] A *Web Content Accessibility Guidelines* (WCAG) 2.1 Nível AA é um padrão internacional de acessibilidade digital que estabelece critérios de desenvolvimento para garantir que softwares e sites sejam plenamente utilizáveis por pessoas com deficiências visuais, auditivas, motoras ou cognitivas (66).
*Nota:* Modificado para atender ao escopo do projeto.
Fonte: Wagner Oliveira de Araujo.

#### 5.1.4.3 Segurança

O Quadro 13 define requisitos de segurança, como isolamento de sessão, sanitização de arquivos, limite de tamanho e comunicação criptografada.

Quadro 13 – Requisitos de segurança do sistema.

| **Requisito** | **Especificação** | **Justificativa** |
|---|---|---|
| Isolamento de sessão | Dados não vazam entre usuários | Aplicação pública; dados podem ser propriedade intelectual |
| Sanitização de arquivos | Validar extensão, cabeçalho e conteúdo | Prevenção contra upload malicioso |
| Limite de tamanho | Máximo 200 MB por arquivo | Evita abuso de recursos computacionais |
| Logs de auditoria (opcional) | Registrar uploads/execuções (sem dados) | Rastreabilidade para suporte técnico |
| HTTPS[a] obrigatório | Todas as comunicações criptografadas | Proteção de integridade e confidencialidade |

Fonte: Wagner Oliveira de Araujo.
[a] *HyperText Transfer Protocol Secure* (HTTPs) (em português, Protocolo de Transferência de Hipertexto Seguro)

#### 5.1.4.4 Escalabilidade

O Quadro 14 define requisitos de escalabilidade, como usuários simultâneos, volume de dados, crescimento funcional e internacionalização, com metas definidas.

Quadro 14 – Requisitos de escalabilidade do sistema.

| **Cenário** | **Capacidade Atual** | **Meta** | **Estratégia** |
|---|---|---|---|
| Usuários simultâneos | ∼10–20 (*Streamlit Cloud free*) | 100+ usuários/h | Container com *load balancer* |
| Volume de dados | ∼500 amostras × 2k variáveis | 5k+ amostras × 10k var. | Processamento em *batch* / *Dask* |
| Crescimento funcional | Script único | Módulos *plug-and-play* | Arquitetura de *plugins* |
| Internacionalização | Português + Inglês | Espanhol | Arquivos *i18n*[a] |

Fonte: Wagner Oliveira de Araujo.
[a] Um arquivo i18n(67) é um arquivo de tradução utilizado para internacionalizar softwares, sites e aplicativos.

## 5.2 Considerações Finais do Levantamento

O levantamento apresentado estabelece uma base estruturada para o desenvolvimento e evolução da aplicação de regressão espectral. Destacam-se como prioridades imediatas:

**a)** Validação rigorosa das entradas (arquivos e parâmetros) para garantir integridade dos cálculos;

**b)** Implementação de mecanismos de *feedback* visual durante processamentos demorados;

**c)** Documentação clara da especificação do formato de dados (estrutura, delimitador e codificação dos arquivos `.txt`);

**d)** Inclusão de funcionalidade básica de exportação de resultados para viabilizar uso acadêmico e profissional.

Recomenda-se que os requisitos classificados como *Essenciais* (Quadro 6) sejam implementados na versão mínima viável *Minimum Viable Product* (MVP)[2] , enquanto os itens *Desejáveis* e *Extras* compõem o *backlog* para iterações futuras, sujeitos a validação com usuários finais e análise de custo-benefício.

Os requisitos não funcionais definidos nas Seções 5.1.4.1 a 5.1.4.4 devem ser considerados como restrições de projeto em todas as fases de desenvolvimento, assegurando que a aplicação seja não apenas funcional, mas também performática, segura e adaptável a cenários de crescimento.

[2] O *Minimum Viable Product* (MVP), ou Produto Mínimo Viável, é a versão mais enxuta de um produto que contém apenas as funcionalidades essenciais para que ele possa ser lançado e testado no mercado.

# 6 ENGENHARIA DE SOFTWARE E MODERNIZAÇÃO

## 6.1 Engenharia de Requisitos com Sommerville

Sommerville (45) estrutura a engenharia de requisitos em quatro fases interdependentes: elicitação, análise, especificação e validação. A reestruturação do sistema em três módulos — `models.py`, `components.py` e `app.py` — evidencia como cada fase foi aplicada para preservar as regras de negócio do legado Vaadin/Java sem reaproveitar seu código-fonte.

### 6.1.1 Elicitação

As entradas críticas do domínio de regressão espectral foram identificadas diretamente no sistema Java original: as matrizes `Xcal` e `Xteste`, os vetores `Ycal` e `Yteste`, e os hiperparâmetros *m* (número de *bags*) e *k* (fração de *subagging*). O `app.py` captura esses insumos por meio de `st.file_uploader` e `st.text_input`, incluindo suporte a decimal com vírgula — particularidade do padrão brasileiro — tratada nas funções `parse_matrix()` e `parse_vector()` do `models.py`.

### 6.1.2 Análise

A análise revelou que a lógica do `RegressionService.java` podia ser decomposta em responsabilidades distintas: *parse* de arquivos, algoritmos de regressão, métricas de avaliação e otimização por validação cruzada. Essa decomposição originou a estrutura modular adotada, onde `models.py` concentra toda a inteligência matemática, sem qualquer dependência de interface.

### 6.1.3 Especificação

As assinaturas de entrada e saída das principais funções do `models.py` foram documentadas com *type hints* [1] nas principais funções do `models.py`: `parse_matrix(file_obj) -> np.ndarray` e `parse_vector(file_obj) -> np.ndarray`, em que `file_obj` é um objeto de arquivo (*file-like*) que expõe `.name` e `.read()`, sendo a extensão (`.txt`, `.csv` ou `.xlsx`) usada para escolher o *parser*; `perform_rlm(Xcal, Ycal, Xteste: np.ndarray) -> np.ndarray`; `calc_metrics(actual, predicted) -> tuple[float, float, float]`. A função `run_full_analysis()` retorna um dicionário (`dict`) com as chaves `log`, `optimal`, `metrics` e `preds`, definindo a interface de comunicação entre a camada de negócio e a camada de apresentação.

[1] *Type hints* são anotações opcionais da linguagem Python que declaram os tipos esperados dos parâmetros e do valor de retorno de uma função (por exemplo, `x: np.ndarray` e `-> float`). Documentam a assinatura da função e permitem verificação estática por ferramentas externas, mas não são impostas pelo interpretador em tempo de execução.

### 6.1.4 Validação

A validação cruzada 5-*fold* implementada em `run_cross_validation()` opera como mecanismo de validação de equivalência, permitindo comparar o desempenho do sistema Python com os resultados históricos do sistema Java. O *log* estruturado gerado por `run_full_analysis()` documenta cada etapa — dimensões detectadas, $\text{RMSEP}_{\text{CV}}$ por combinação de hiperparâmetros, métricas finais e tempo de execução — possibilitando auditoria científica e rastreabilidade completa. Para assegurar a estabilidade dessa arquitetura, construiu-se uma suíte com 92 testes automatizados, composta por 21 testes unitários, 60 testes de caixa-preta e 11 testes de carga e estresse.

## 6.2 Arquitetura Modular em Python/Streamlit

A arquitetura do sistema modernizado adota o padrão de separação de responsabilidades em três camadas físicas distintas, representadas pelos arquivos `models.py`, `components.py` e `app.py`. Essa organização substitui a estrutura monolítica do sistema Java original, garantindo testabilidade isolada e desacoplamento entre a lógica de negócio e a interface com o usuário.

O módulo `models.py` atua como a lógica matemática do sistema, encapsulando todas as operações matemáticas e estatísticas sem qualquer importação de bibliotecas de interface gráfica. Ele contém as implementações da RLM via `np.linalg.lstsq`, do *bagging* e do *subagging* parametrizados, das funções de *parsing* de dados e das rotinas de validação cruzada 5-*fold* com *grid search*.

Ressalta-se que o sistema Java legado calcula os coeficientes da RLM pela pseudoinversa de Moore-Penrose explícita (`SimpleMatrix.pseudoInverse()`, biblioteca EJML), ao passo que a versão Python emprega `np.linalg.lstsq`, que resolve o problema de mínimos quadrados e retorna a solução de mínima norma. Para os sistemas subdeterminados deste trabalho ($17 \times 723$), ambas as rotinas convergem para a mesma solução teórica $\widehat{\boldsymbol{\beta}} = \mathbf{X}^{+}\mathbf{y}$, embora por caminhos numéricos distintos.

O módulo `components.py` responde pela fábrica de componentes visuais e geração de gráficos interativos utilizando a biblioteca Plotly. Ele transforma as matrizes de resultados e vetores de predição retornados pela camada de modelagem em objetos de figura prontos para renderização, isolando o código de visualização do fluxo principal da aplicação.

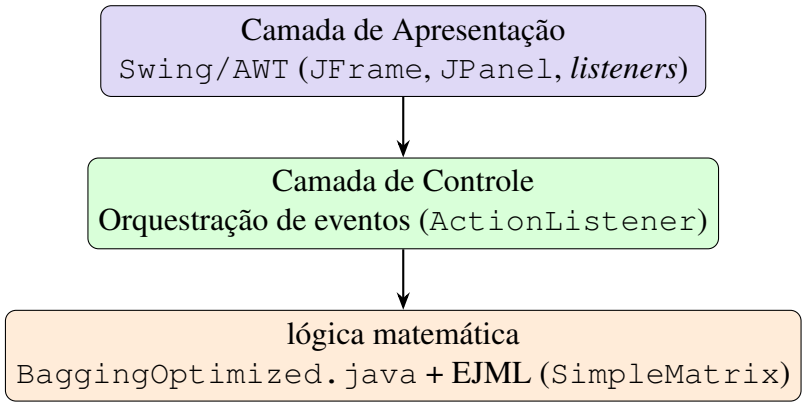
O módulo `app.py` funciona como controlador da interface e do estado da sessão (`st.session_state`). Ele captura as interações do usuário, invoca as funções puras de `models.py`, repassa os resultados para `components.py` e gerencia a exibição sequencial em abas. Essa divisão garante que alterações na camada visual não impactem os algoritmos científicos e vice-versa.

No Apêndice A, a Figura 17 apresenta as três camadas do sistema modernizado, em contraste com a organização monolítica do sistema Java original, ilustrada na Figura 16. O

diagrama de classes dos módulos (Figura 24) e os demais diagramas de arquitetura e *Unified Modeling Language* (UML) estão reunidos nesse mesmo apêndice.

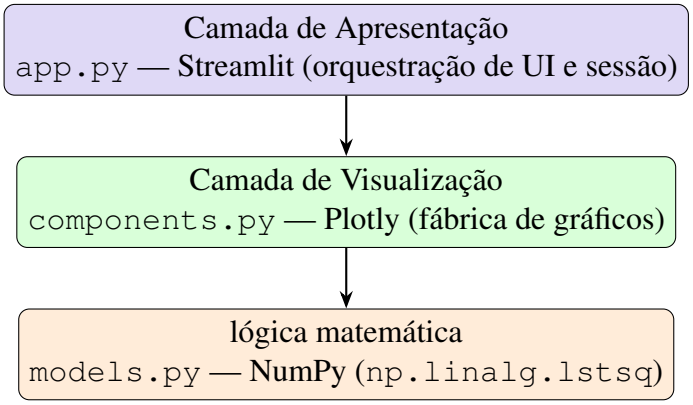
A Figura 16 mostra a organização lógica em camadas do sistema Java original, contida em um único arquivo, e a Figura 17 apresenta as três camadas físicas do sistema modernizado. O diagrama de classes dos módulos (Figura 24) e os demais diagramas UML estão no Apêndice A.

Figura 16 – Organização lógica em camadas dentro de um único arquivo — Sistema legado monolítico (Java Desktop).




Fonte: Wagner Oliveira de Araujo.

Figura 17 – Arquitetura em camadas — Sistema modernizado (Python/Streamlit).




Fonte: Wagner Oliveira de Araujo.

# 7 RESULTADOS E DISCUSSÕES

## 7.1 Avaliação Preditiva e Eficiência Computacional

A avaliação experimental do sistema modernizado foi conduzida utilizando os dados espectrométricos de cinco analitos (Mn, Mo, Cr, Ni e Fe) presentes em amostras de aço-ligas. Os modelos de Regressão Linear Múltipla (RLM), RLM-*bagging* e RLM-*subagging* foram comparados quanto à acurácia preditiva, mensurada pelas métricas RMSEP, MAE e $R^2$, e quanto ao tempo de processamento computacional.

Para o Molibdênio (Mo) e o Cromo (Cr), a RLM clássica obteve menor RMSEP, indicando que a introdução das estratégias de *ensemble* não proporcionou ganho de acurácia preditiva no conjunto de teste avaliado. Os resultados detalhados demonstram que a estrutura linear simples da RLM foi suficiente para capturar a variabilidade desses elementos sem introduzir a variância adicional inerente ao processo de reamostragem.

No caso do Ferro (Fe), o modelo RLM-*bagging* apresentou o menor erro de predição, enquanto para o Manganês (Mn) e o Níquel (Ni), o RLM-*subagging* obteve desempenho superior ou equivalente às demais abordagens. O desempenho diferenciado entre os analitos reflete suas características espectrais individuais no conjunto de dados. A verificação detalhada das causas físico-químicas subjacentes e a decomposição formal de viés e variância constituem tópicos sugeridos para investigações futuras.

Em termos de eficiência computacional, o RLM-*subagging* demonstrou tempos de execução sistematicamente inferiores aos do RLM-*bagging* em quatro dos cinco analitos avaliados. Essa redução do tempo de processamento é atribuída ao menor tamanho da amostra utilizada em cada submodelo durante o procedimento de *subagging* ($k < 1$), o que diminui o custo computacional do ajuste de matrizes por Mínimos Quadrados Ordinários (MQO).

# 8 INTERFACE E EXPERIÊNCIA DO USUÁRIO CIENTÍFICA

## 8.1 Design da Interface e Fluxo de Trabalho

A interface gráfica do sistema modernizado foi desenvolvida sobre o *framework* Streamlit, adotando princípios de usabilidade voltados para aplicações científicas e de análise de dados. A organização visual busca minimizar a poluição de tela e guiar o usuário de forma intuitiva pelas etapas do processo de calibração multivariada.

A aplicação é dividida em três abas sequenciais que espelham o fluxo de trabalho do usuário:

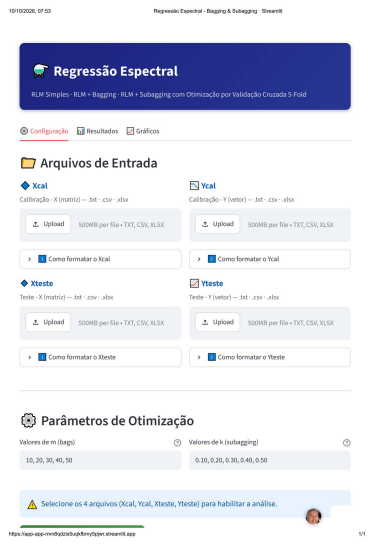
a) **Aba 1 (Configuração e Carga de Dados):** Destinada ao envio dos arquivos de calibração e teste nos formatos `.txt`, `.csv` ou `.xlsx`, e à definição dos hiperparâmetros de entrada (número de replicações *m* e fração de subamostragem *k*);

b) **Aba 2 (Resultados Numéricos e Logs):** Exibe as métricas de avaliação preditiva (RMSEP, MAE, $R^2$), a indicação dos parâmetros ótimos selecionados pela validação cruzada 5-*fold* e o *log* detalhado de execução;

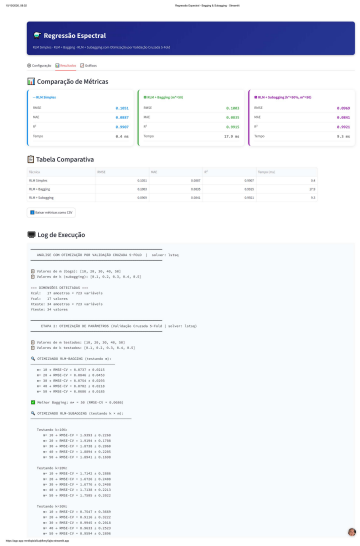
c) **Aba 3 (Visualização Gráfica):** Apresenta os gráficos interativos gerados via Plotly: três gráficos de linha comparando os valores reais e preditos por amostra (RLM, RLM-*bagging* e RLM-*subagging*) e um gráfico de dispersão Real $\times$ Predito com os três modelos sobrepostos e a reta ideal ($y = x$).

Essa estrutura em abas sequenciais Figura 18 organiza o *workflow* analítico segundo as etapas operacionais do usuário, reduzindo a dispersão visual e delimitando os passos de execução.

Figura 18 – Interface do sistema modernizado: (a) aba Configuração; (b) aba Resultados; (c) aba Gráficos.




(a) Configuração.




(b) Resultados.




(c) Gráficos.

Fonte: Wagner Oliveira de Araujo.

A Figura 25, apresentada no Apêndice A, mostra o diagrama de sequência com a troca de chamadas entre as abas da interface, o módulo `app.py` e a lógica matemática em `models.py`.

O ambiente de implantação viabiliza o acesso simultâneo via navegador sem necessidade de instalação local de dependências ou ambientes de execução Java, sujeitando-se aos limites de concorrência e memória da plataforma de hospedagem adotada.

Para visualizar a aplicação em funcionamento, o leitor pode acessá-la diretamente pelo navegador, sem instalar nenhum programa, por meio do endereço https://app-app-rmn8qdzis5uqkfbmy5pjwr.streamlit.app/, em que é possível enviar os arquivos, configurar os parâmetros e executar a análise.

# 9 DESENVOLVIMENTO ITERATIVO POR SPRINTS E VALIDAÇÃO POR TESTES AUTOMATIZADOS

## 9.1 Metodologia Ágil: Desenvolvimento por Sprints

O desenvolvimento do sistema Python/Streamlit seguiu uma abordagem iterativa e incremental, organizada em quatro *sprints* sucessivas, alinhada aos princípios do *framework* Scrum (68). Cada *sprint* produziu um incremento funcional e testável do sistema, permitindo validação contínua da equivalência com o sistema Java legado antes do avanço para a etapa seguinte.

### 9.1.1 *Sprint* 1 — Camada de Leitura e Tratamento de Dados

Esta etapa implementou as funções `parse_matrix()` e `parse_vector()`, responsáveis pela leitura unificada de arquivos `.txt`, `.csv` e `.xlsx`, com detecção automática de separador de coluna e normalização do decimal brasileiro. A validação ocorreu por meio de testes no console, sem envolvimento da interface Streamlit, utilizando os *datasets* reais *Ethanol-Level* e `CGL_nir`, confirmando as dimensões esperadas das matrizes.

### 9.1.2 *Sprint* 2 — Algoritmos Científicos

A lógica matemática foi codificada em `models.py` como funções puras, independentes de qualquer biblioteca de interface. Implementaram-se `perform_rlm()` via `np.linalg.lstsq`, `perform_bagging()` para *bagging* e *subagging* parametrizados, `calc_metrics()` para RMSEP, MAE e $R^2$, e `run_cross_validation()` para validação cruzada 5-*fold*, culminando na função `find_optimal_parameters()` de *grid search*.

### 9.1.3 *Sprint* 3 — Interface Reativa

Conectou-se `models.py` ao controlador `app.py`, implementando os quatro componentes `st.file_uploader`, a validação de parâmetros por meio de `parse_int_list()` e `parse_float_list()`, o *callback* de progresso e a persistência do resultado em `st.session_state`. Os quatro gráficos Plotly foram centralizados em `components.py`, garantindo que nenhuma lógica de negócio residisse na camada de apresentação.

### 9.1.4 *Sprint* 4 — Otimização com *Cache*

Avaliou-se a arquitetura resultante quanto à sua compatibilidade com os mecanismos de *cache* do Streamlit. Como `parse_matrix()`, `parse_vector()` e `run_full_-`

`analysis()` são funções puras, determinísticas e com entradas serializáveis, o decorador[1] `@st.cache_data` pode ser aplicado diretamente, podendo reduzir a latência em execuções repetidas; o ganho não foi medido.

## 9.2 Suíte de Testes Automatizados com pytest

Os conceitos de teste unitário, de caixa-preta e de carga e *stress* adotados nesta suíte, bem como o papel da automação, estão fundamentados na Seção 2.8.1. A suíte aplica esses conceitos à verificação da equivalência entre o sistema Java legado e o sistema Python reescrito.

A correção do sistema Python foi validada por meio de uma suíte de 92 testes automatizados com `pytest` (49), organizada em três classes complementares: testes unitários, testes de caixa-preta e testes de carga e *stress*. Essa estratificação permitiu verificar, respectivamente, a corretude matemática isolada, a fidelidade em relação ao sistema Java legado e o comportamento sob volumes de dados realistas. A saída da execução da suíte de testes automatizados, desenvolvida especificamente para este trabalho com `pytest` e executada no VS Code, é apresentada no Apêndice C.

### 9.2.1 Testes Unitários (21 testes)

Exercitaram cada função pura de `models.py` isoladamente, com matrizes de dimensão reduzida e valores determinísticos, por meio das *fixtures* `dados_simples` ($20 \times 5$) e `dados_medios` ($300 \times 50$). Cobriram-se oito testes de *parsing* de arquivos e treze testes de modelos de regressão, incluindo a recuperação exata de $\boldsymbol{\beta}$ em sistemas determinados e a reprodutibilidade do *bagging* sob a mesma semente aleatória.

### 9.2.2 Testes de Caixa-Preta (60 testes)

Compararam as métricas agregadas (RMSEP, MAE e $R^2$) calculadas pelo sistema Python com as registradas nos arquivos de saída do sistema Java original, sem acesso ao código-fonte legado, seguindo a estratégia de teste caixa preta (47). Doze testes parametrizados foram aplicados a cada um de cinco analitos (Mn, Mo, Cr, Ni, Fe). Para a RLM, que é determinística, adotou-se tolerância de $5 \times 10^{-4}$.

Embora a RLM seja determinística, Java (pseudoinversa explícita) e Python (`lstsq`) utilizam rotinas numéricas distintas, o que justifica a adoção de uma tolerância de $5 \times 10^{-4}$, e não de igualdade exata.

Para o *bagging* e o *subagging*, que são estocásticos e utilizam geradores pseudoaleatórios distintos em Python e em Java, adotaram-se tolerâncias de 0,15 para RMSEP e MAE e de $5 \times 10^{-3}$ para $R^2$, além da verificação de que $m^*$ e $k^*$ pertencem à grade de busca. Esses testes

[1] Em Python, um decorador é uma sintaxe especial (indicada pelo caractere @) utilizada para estender ou modificar o comportamento de funções ou métodos de forma declarativa, sem a necessidade de alterar diretamente o seu código-fonte.

atestam, portanto, equivalência numérica estrita para a RLM e consistência estatística para os métodos de *ensemble*, sem exigir a identidade das reamostragens.

### 9.2.3 Testes de Carga e *Stress* (11 testes)

Quantificaram os limites operacionais de tempo de execução e consumo de memória para matrizes espectrais de alta dimensionalidade (45), empregando as *fixtures* `matriz_grande` ($5.000 \times 723$) e `matriz_stress` ($15.000 \times 723$). O tempo foi medido com `time.perf_counter()` e o pico de alocação de memória com `tracemalloc`, registrando o tempo do *bagging* para 5, 10, 25 e 50 *bags* e o pico de memória da RLM.

Esses testes documentam o comportamento operacional do sistema; apenas alguns impõem limites (30 s para a RLM em $5.000 \times 723$; 2048 MB de memória), e a relação entre número de *bags* e tempo é reportada, mas não confirmada por um teste de validação.

### 9.2.4 Resultados Consolidados

A suíte completa foi executada em ambiente Windows com Python 3.12.10 e pytest 9.1.0, totalizando 92 testes aprovados, nenhuma falha e nenhum teste ignorado, em 75,25 segundos. Esse resultado evidencia três conclusões objetivas: correção matemática nos testes unitários, equivalência com o sistema Java legado nos níveis descritos no item (b) e execução sem erro de memória para matrizes de até $15.000 \times 723$ (cerca de 87 MB em precisão dupla).

Como ilustração da fidelidade amostra-a-amostra subjacente a esses testes agregados, a Tabela 8 apresenta a comparação completa entre Java e Python para o caso do *Bagging* aplicado ao Cromo (Cr); as tabelas equivalentes para os demais catorze pares analito×técnica encontram-se no Apêndice B.

Tabela 8 – Paridade *Bagging* – Cr.

| **Amostra** | **Y real** | **Y predito (Java)**[a] | **Y predito (Python)** | **Δ** |
|---|---|---|---|---|
| 1 | 30,000 | 30,1484 | 30,1484 | 0,0 |
| 2 | 40,000 | 39,9788 | 39,9788 | 0,0 |
| 3 | 60,000 | 60,0579 | 60,0579 | 0,0 |
| 4 | 35,000 | 34,9336 | 34,9336 | 0,0 |
| 5 | 45,000 | 44,7089 | 44,7089 | 0,0 |
| 6 | 48,000 | 47,4377 | 47,4377 | 0,0 |
| 7 | 65,000 | 64,7815 | 64,7815 | 0,0 |
| 8 | 23,000 | 22,7322 | 22,7322 | 0,0 |
| 9 | 25,000 | 23,9380 | 23,9380 | 0,0 |
| 10 | 37,000 | 36,0721 | 36,0721 | 0,0 |
| 11 | 23,103 | 22,7884 | 22,7884 | 0,0 |

Continua na próxima página...

**Tabela 8 – Continuação**

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 12 | 22,722 | 20,4495 | 20,4495 | 0,0 |
| 13 | 22,713 | 21,5452 | 21,5452 | 0,0 |
| 14 | 60,054 | 60,0535 | 60,0535 | 0,0 |
| 15 | 59,959 | 57,2886 | 57,2886 | 0,0 |
| 16 | 32,250 | 33,6880 | 33,6880 | 0,0 |
| 17 | 32,448 | 31,7185 | 31,7185 | 0,0 |
| 18 | 32,154 | 30,6143 | 30,6143 | 0,0 |
| 19 | 60,996 | 62,9324 | 62,9324 | 0,0 |
| 20 | 60,684 | 60,3428 | 60,3428 | 0,0 |
| 21 | 60,360 | 58,3135 | 58,3135 | 0,0 |
| 22 | 40,636 | 41,4952 | 41,4952 | 0,0 |
| 23 | 40,733 | 39,3944 | 39,3944 | 0,0 |
| 24 | 40,524 | 39,4233 | 39,4233 | 0,0 |
| 25 | 64,597 | 66,6126 | 66,6126 | 0,0 |
| 26 | 65,522 | 65,2090 | 65,2090 | 0,0 |
| 27 | 44,707 | 43,2840 | 43,2840 | 0,0 |
| 28 | 44,716 | 43,3947 | 43,3947 | 0,0 |
| 29 | 30,022 | 30,3226 | 30,3226 | 0,0 |
| 30 | 30,177 | 28,6635 | 28,6635 | 0,0 |
| 31 | 29,926 | 28,1745 | 28,1745 | 0,0 |
| 32 | 44,926 | 46,1783 | 46,1783 | 0,0 |
| 33 | 44,837 | 43,8530 | 43,8530 | 0,0 |
| 34 | 44,837 | 43,7524 | 43,7524 | 0,0 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *Bagging* no analito Cromo (Cr).

# 10 CONCLUSÃO

Este trabalho apresentou a migração e a modernização de um sistema legado de análise química multivariada, transicionando de uma arquitetura *Desktop* monolítica em Java para uma aplicação *Web* modular em Python e Streamlit. A reescrita foi conduzida sob o rigor da engenharia de *software*, com arquitetura desacoplada em três camadas e validação por meio de uma suíte de 92 testes automatizados.

No eixo quimiométrico, investigou-se a aplicação das técnicas de *bagging* e *subagging* incorporadas à Regressão Linear Múltipla (RLM) na calibração de cinco analitos (Mn, Mo, Cr, Ni e Fe) em amostras de aço-ligas. Os resultados demonstraram que o comportamento das técnicas de reamostragem varia conforme o analito considerado. A RLM clássica obteve menor RMSEP para o Molibdênio e o Cromo, enquanto o RLM-*bagging* destacou-se para o Ferro.

Para o Níquel (Ni), o RLM-*subagging* superou o RLM-*bagging*, porém apresentou desempenho numericamente equivalente ao da RLM clássica (diferença de 0,25% no RMSEP), não se atribuindo superioridade estatística ao método frente ao conjunto de teste avaliado. Além disso, o RLM-*subagging* comprovou maior eficiência computacional em relação ao RLM-*bagging*, reduzindo o tempo de processamento devido ao menor tamanho das subamostras.

No eixo de engenharia de *software*, a suíte com 21 testes unitários, 60 testes de caixa-preta e 11 testes de estresse assegurou a equivalência numérica estrita para os métodos determinísticos e a estabilidade da aplicação sob volumes de dados sintetizados de alta dimensão, comprovando a viabilidade do fluxo de reescrita sem degradação das regras de negócio.

## 10.1 Limitações e Trabalhos Futuros

Este trabalho possui escopo direcionado na migração de plataforma e avaliação experimental inicial. Constituem limitações desta pesquisa:

a) A não realização de testes estatísticos formais de hipótese para a decomposição rigorosa de viés e variância nos espectros;

b) A ausência de medição quantitativa da taxa de latência economizada pelo uso do decorador de *cache* em ambiente multiusuário de produção;

c) A limitação do conjunto de calibração ao número de amostras reais de aço-ligas disponibilizadas no protocolo de referência.

Como trabalhos futuros, propõe-se a integração de um agente de inteligência artificial especializado na interpretação automatizada dos gráficos gerados. Essa ferramenta auxiliará na análise dos resultados visuais em modelos estatísticos como RLM, *bagging* e *subagging*, cuja precisão na síntese dos dados científicos deverá ser avaliada por comparação com a interpretação de especialistas. Adicionalmente, recomenda-se a expansão do conjunto amostral com novos

padrões de aço-ligas e a realização de testes de carga em ambiente de nuvem sob acessos concorrentes simultâneos.

## REFERÊNCIAS

1 SKOOG, D. A.; HOLLER, F. J.; NIEMAN, T. A. *Principles of Instrumental Analysis*. 5. ed. USA: Brooks Cole, 1997. 960 p. Citado 2 vezes nas páginas 14 e 32.

2 HÄRDLE, W.; SIMAR, L. *Applied Multivariate Statistical Analysis*. 2. ed. Berlim, Germany: Springer-Verlag, 2007. 455 p. Citado 3 vezes nas páginas 14, 18 e 26.

3 TIMM, N. H. *Applied Multivariate Analysis*. Berlim, Germany: Springer-Verlag, 2002. 693 p. (Springer Texts in Statistics). Citado 3 vezes nas páginas 14, 17 e 18.

4 Streamlit Inc. *Streamlit: A faster way to build and share data apps*. 2026. Página oficial. Disponível em: https://streamlit.io/. Acesso em: 10 mar. 2026. Citado na página 14.

5 Streamlit Inc. *Streamlit Documentation: Open-source Python Framework for Data Apps*. 2026. Documentação oficial. Disponível em: https://docs.streamlit.io/. Acesso em: 10 mar. 2026. Citado na página 14.

6 CANTY, A. J.; DAVISON, A. C. Resampling-based variance estimation for labour force surveys. 2001. DOI: https://doi.org/10.1111/1467-9884.00196. Disponível em: https://rss.onlinelibrary.wiley.com/doi/10.1111/1467-9884.00196. Acesso em: 29 mar. 2025. Citado 2 vezes nas páginas 15 e 24.

7 RATHNAYAKE, S.; OLIVE, D. Bagged prediction accuracy in linear regression. *DiVA Portal*, n. 1669623, 2022. Disponível em: https://www.diva-portal.org/smash/record.jsf?pid=diva2:1669623. Acesso em: 3 out. 2025. Citado 2 vezes nas páginas 15 e 26.

8 DRAPER, N. R.; SMITH, H. *Applied Regression Analysis*. 3. ed. USA: John Willey & Sons, Inc, 1998. 697 p. (Wiley Series in Probability and Statistics). Disponível em: https://onlinelibrary.wiley.com/doi/book/10.1002/9781118625590. Acesso em: 29 mar. 2025. Citado 2 vezes nas páginas 17 e 18.

9 ALIN, A.; SERNEELS, S. Robust iteratively reweighted simpls. *Journal of Chemometrics*, v. 31, n. 3, p. e2881, 2017. DOI: https://doi.org/10.1002/cem.2881. Disponível em: https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/abs/10.1002/cem.2881. Acesso em: 3 jan. 2026. Citado na página 18.

10 FILZMOSER, P.; NORDHAUSEN, K. On robust testing for normality in chemometrics. *Chemometrics and Intelligent Laboratory Systems*, v. 130, p. 98–100, 2013. DOI: https://doi.org/10.1016/j.chemolab.2013.10.010. Disponível em: https://www.sciencedirect.com/science/article/abs/pii/S0169743913001913. Acesso em: 3 jan. 2026. Citado na página 18.

11 JOHNSON, R. A.; WICHERN, D. W. *Applied Multivariate Statistical Analysis*. 5. ed. USA: Prentice Hall, 2001. 767 p. Citado na página 18.

12 CHATTERJEE, S.; HADI, A. S. *Regression Analysis by Example*. 4. ed. USA: John Wiley & Sons, Inc, 2006. 371 p. Disponível em: https://onlinelibrary.wiley.com/doi/book/10.1002/0470055464. Acesso em: 29 mar. 2025. Citado na página 18.

13 YEON, H.; DAI, X.; NORDMAN, D. J. Bootstrap inference in functional linear regression models with scalar response. *Bernoulli*, Bernoulli Society for Mathematical Statistics and Probability, v. 29, n. 4, p. 2599–2626, 2023. DOI: https://doi.org/10.3150/22-BEJ1554. Disponível em: https://projecteuclid.org/journals/bernoulli/volume-29/issue-4/Bootstrap-inference-in-functional-linear-regression-models-with-scalar-response/10.3150/22-BEJ1554.short. Acesso em: 1 abr. 2025. Citado na página 18.

14 ALFONS, A.; ATEÅŸ, N. Y.; GROENEN, P. J. F. A robust bootstrap test for mediation analysis. *Organizational Research Methods*, v. 25, n. 3, p. 591–617, 2022. DOI: https://doi.org/10.1177/1094428121999096. Disponível em: https://journals.sagepub.com/doi/10.1177/1094428121999096. Acesso em: 1 abr. 2025. Citado na página 18.

15 WAN, Y. et al. Multiplier bootstrap-based exploration. *Proceedings of Machine Learning Research*, v. 202, p. 1–15, 2023. Disponível em: https://openreview.net/forum?id=M6n9VdvThQ. Acesso em: 3 out. 2025. Citado na página 18.

16 DU, J.-H. et al. Extrapolated Cross-Validation for Randomized Ensembles. *Journal of Computational and Graphical Statistics*, v. 33, n. 3, p. 1061–1072, jul 2024. ISSN 1061-8600. DOI: https://doi.org/10.1080/10618600.2023.2288194. Disponível em: https://www.tandfonline.com/doi/full/10.1080/10618600.2023.2288194. Acesso em: 3 out. 2025. Citado na página 18.

17 JENSEN, V.; BIANCHI, F. M.; ANFINSEN, S. N. Ensemble Conformalized Quantile Regression for Probabilistic Time Series Forecasting. *IEEE Transactions on Neural Networks and Learning Systems*, v. 35, n. 7, p. 9014–9025, jul 2024. ISSN 2162-237X. Disponível em: https://arxiv.org/abs/2202.08756https://ieeexplore.ieee.org/document/9940232/. Acesso em: 3 out. 2025. Citado 2 vezes nas páginas 18 e 35.

18 EFRON, B. *The Jackknife, the Bootstrap and Other Resampling Plans*. USA: Society for Industrial and Applied Mathematics, 1994. 91 p. (CBMS-NSF Regional Conference Series in Applied Mathematics). DOI: https://doi.org/10.1137/1.9781611970319. Disponível em: https://epubs.siam.org/doi/book/10.1137/1.9781611970319. Acesso em: 29 mar. 2025. Citado na página 19.

19 EFRON, B.; TIBSHIRANI, R. J. *An Introduction to the Bootstrap*. USA: Chapman & Hall/CRC, 1993. 430 p. DOI: https://doi.org/10.2307/2983304. Disponível em:

https://www.jstor.org/stable/2983304. Acesso em: 29 mar. 2025. Citado 2 vezes nas páginas 20 e 21.

20 EFRON, B.; TIBSHIRANI, R. J. Improvements on cross-validation: The .632+ bootstrap method. *Journal of the American Statistical Association*, USA, v. 92, n. 438, p. 548–560, jul 1997. DOI: https://doi.org/10.1080/01621459.1997.10474007. Disponível em: https://www.tandfonline.com/doi/abs/10.1080/01621459.1997.10474007. Acesso em: 29 mar. 2025. Citado na página 25.

21 FERNANDEZ, J. M. V.; MANTEIGA, W. G. Resampling for checking linear regression models via non-parametric regression estimation. *Computational Statistics & Data Analysis*, USA, v. 35, n. 2, p. 211–231, Dec 1999. DOI: https://doi.org/10.1016/S0167-9473(99)00117-6. Disponível em: https://www.sciencedirect.com/science/article/abs/pii/S0167947399001176?via%3Dihub. Acesso em: 29 mar. 2025. Citado na página 25.

22 OPITZ, D.; MACLIN, R. Popular ensemble methods: an empirical study. *Journal of Artificial Intelligence Research*, USA, v. 11, p. 169–198, 1999. DOI: https://doi.org/10.1613/jair.614. Disponível em: https://jair.org/index.php/jair/article/view/10239. Acesso em: 29 mar. 2025. Citado 2 vezes nas páginas 25 e 26.

23 GALVÃO, R. K. H. et al. An application of subagging for the improvement of prediction accuracy of multivariate calibration models. *Chemometrics and Intelligent Laboratory Systems*, v. 81, n. 1, p. 60–67, 2006. ISSN 0169-7439. Disponível em: https://www.sciencedirect.com/science/article/pii/S0169743905001528. Acesso em: 29 set. 2025. Citado 2 vezes nas páginas 25 e 27.

24 LIU, Y. et al. How are chemometric models validated? a systematic review of linear regression models for nirs data in food analysis. *Journal of Chemometrics*, v. 39, n. 5, p. e70036, 2025. DOI: https://doi.org/10.1002/cem.70036. Disponível em: https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/cem.70036. Acesso em: 3 out. 2025. Citado 2 vezes nas páginas 26 e 35.

25 MISHRA, P.; LILAND, K. H. Iterative re-weighted multilinear partial least squares modelling for robust predictive modelling. *Journal of Chemometrics*, 2023. DOI: https://doi.org/10.1002/cem.3527DigitalObjectIdentifier(DOI). Disponível em: https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/cem.3527. Acesso em: 3 out. 2025. Citado na página 26.

26 STYAWAN et al. The use of chemometrics in combination with molecular spectroscopy and chromatography methods for determining the levels of gingerol compounds in ginger (zingiber officinale): a review. *Food Research*, 2024. DOI: https://doi.org/10.26656/fr.2017.8(3).165. Disponível em: https://www.myfoodresearch.com/uploads/8/4/8/5/84855864/_39__fr-2022-165_styawan.pdf. Acesso em: 3 out. 2025. Citado na página 26.

27 BREIMAN, L. Bagging predictors. In: *Machine Learning, 24*. Boston, USA: Boston: Machine Learning, 1996. p. 123–140. Disponível em: https://link.springer.com/article/10.1007/BF00058655. Acesso em: 2 out. 2025. Citado na página 26.

28 SHABBIR, M.; CHAND, S.; IQBAL, F. Bagging-based ridge estimators for a linear regression model with non-normal and heteroscedastic errors. *Communications in Statistics - Simulation and Computation*, Taylor & Francis, v. 53, n. 8, p. 3653–3667, 2024. DOI: https://doi.org/10.1080/03610918.2022.2109675. Disponível em: https://www.tandfonline.com/doi/full/10.1080/03610918.2022.2109675. Acesso em: 1 abr. 2025. Citado na página 26.

29 HONG, H.; LIU, J.; ZHU, A.-X. Modeling landslide susceptibility using logitboost alternating decision trees and forest by penalizing attributes with the bagging ensemble. *Science of The Total Environment*, v. 718, p. 137231, 2020. ISSN 0048-9697. Disponível em: https://www.sciencedirect.com/science/article/pii/S0048969720307415. Acesso em: 1 abr. 2025. Citado na página 26.

30 ZHAO, N. et al. Pharmaceutical analysis model robustness from bagging-pls and pls using systematic tracking mapping. *Frontiers in Chemistry*, v. 6, p. 262, 2018. DOI: https://doi.org/10.3389/fchem.2018.00262. Disponível em: https://www.frontiersin.org/journals/chemistry/articles/10.3389/fchem.2018.00262/full. Acesso em: 3 out. 2025. Citado na página 26.

31 KUMARI, M. V. et al. A review on chemometric techniques. *International Journal of Pharmacy and Pharmaceutical Sciences*, v. 9, n. 4, 2024. Disponível em: https://www.pharmacyjournal.net/search?keyword=M%20Venkata%20Kumari. Acesso em: 3 out. 2025. Citado 3 vezes nas páginas 26, 35 e 36.

32 MARTINS, M. do N. *Transferência de calibração de instrumentos para análise espectrométrica empregando seleção de variáveis, reamostragem e combinação de modelos*. 114 p. Dissertação (Mestrado em Engenharia Eletrônica e Computação) — Programa de Pós-Graduação em Engenharia Eletrônica e Computação, ITA, São José dos Campos, 2006. Disponível em: https://www.dominiopublico.gov.br/pesquisa/DetalheObraForm.do?select_action=&co_obra=151850. Acesso em: 23 set. 2025. Citado na página 26.

33 BAUER, E.; KOHAVI, R. An empirical comparison of voting classification algorithms: Baggin, boosting e variants. *Machine Learning*, Boston, USA, v. 36, p. 105–142, 1999. DOI: https://doi.org/10.1023/A:1007515423169. Disponível em: https://link.springer.com/article/10.1023/A:1007515423169. Acesso em: 29 mar. 2025. Citado na página 27.

34 BREIMAN, L. Using iterated bagging to debias regressions. *Machine Learning*, Netherlands, v. 45, p. 261–277, 2001. DOI: https://doi.org/10.1023/A:1017934522171. Disponível em: https://link.springer.com/article/10.1023/A:1017934522171. Acesso em: 29 mar. 2025. Citado na página 27.

35 BUHLMANN, P. Bagging, subagging and bragging for improving some prediction algorithms. 2003. Disponível em: https://www.academia.edu/143294244/Bagging_Subagging_and_Bragging_for_Improving_some_Prediction_Algorithms. Acesso em: 31 mar. 2025. Citado 2 vezes nas páginas 27 e 28.

36 REVELAS, C.; BOLDEA, O.; WERKER, B. J. M. *When does Subagging Work?* 2024. Disponível em: https://arxiv.org/abs/2404.01832. Acesso em: 1 abr. 2025. Citado 2 vezes nas páginas 27 e 28.

37 QIN, T.; HUANG, W.-M. *On Subagging Boosted Probit Model Trees*. 2023. Disponível em: https://arxiv.org/abs/2311.02827. Acesso em: 1 abr. 2025. Citado na página 27.

38 ZHAO, Y.; AMEMIYA, Y.; HUNG, Y. Efficient gaussian process modeling using experimental design-based subagging. *Statistica Sinica*, v. 28, n. 3, p. 1459–1479, 2018. Disponível em: https://www.jstor.org/stable/26492953. Acesso em: 3 out. 2025. Citado na página 27.

39 SHI, C.; LU, W.; SONG, R. Breaking the curse of nonregularity with subagging: Inference of the mean outcome under optimal treatment regimes. *Journal of Machine Learning Research*, v. 22, n. 1, p. 1–35, 2021. Disponível em: https://www.jmlr.org/papers/v21/20-066.html. Acesso em: 3 out. 2025. Citado 2 vezes nas páginas 27 e 28.

40 ZHANG, C. et al. Subagging for the improvement of predictive stability of extreme learning machine for spectral quantitative analysis of complex samples. *Chemometrics and Intelligent Laboratory Systems*, v. 161, p. 43–48, feb 2017. ISSN 01697439. Disponível em: https://linkinghub.elsevier.com/retrieve/pii/S0169743916302295. Acesso em: 3 out. 2025. Citado na página 27.

41 TAO, X. et al. Gated recurrent unit-based parallel network traffic anomaly detection using subagging. *IEEE Transactions on Dependable and Secure Computing*, v. 19, n. 4, p. 2456–2468, 2021. DOI: https://doi.org/10.1016/j.adhoc.2021.102465. Disponível em: https://www.sciencedirect.com/science/article/abs/pii/S1570870521000342?via%3Dihub. Acesso em: 3 out. 2025. Citado na página 27.

42 PALEOLOGO, G.; ELISSEEFF, A.; ANTONINI, G. Subagging for credit scoring models [updated 2020]. *European Journal of Operational Research*, v. 201, n. 2, p. 490–499, 2020. DOI: https://doi.org/10.1016/j.ejor.2009.03.008. Disponível em: https://www.sciencedirect.com/science/article/abs/pii/S0377221709001532?via%3Dihub. Acesso em: 3 out. 2025. Citado na página 27.

43 WEI, Y. Quantifying variation of subsampling-based ensemble methods under a u-statistic framework. *Wellesley College Thesis*, 2021. Disponível em: https://repository.wellesley.edu/object/ir1637. Acesso em: 3 out. 2025. Citado na página 27.

44 ING, C.-K.; LAI, T. L. Randomized maximum-contrast selection: Subagging for large-scale regression. *Electronic Journal of Statistics*, v. 10, n. 1, p. 121–170, 2016. DOI: https://doi.org/10.1214/15-EJS1085. Disponível em: https://projecteuclid.org/journals/electronic-journal-of-statistics/volume-10/issue-1/Randomized-maximum-contrast-selection-Subagging-for-large-scale-regression/10.1214/15-EJS1085.full. Acesso em: 3 jan. 2026. Citado na página 28.

45 SOMMERVILLE, I. *Engenharia de Software*. 9. ed. São Paulo: Pearson Education Brasil, 2011. 504 p. ISBN 978-8581430157. Citado 3 vezes nas páginas 29, 58 e 66.

46 SADANGI, S. *Building Data Apps with Streamlit: Create Dynamic Dashboards, Machine Learning Workflows, and AI Chatbots with Streamlit and Python* . Orange Education Pvt Ltd, 2026. ISBN 978-9349887848. Disponível em: https://bookshop.org/p/books/building-data-apps-with-streamlit-create-dynamic-dashboards-machine-learning-workflows-and-ai-chatbots-with-streamlit-and-python-english-edition-s/9861ec986aa21894. Acesso em: 28 maio. 2026. Citado na página 29.

47 PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. Porto Alegre: Artmed, 2016. ISBN 978-8582603659. Citado 2 vezes nas páginas 29 e 65.

48 JORGENSEN, P. C. *Software Testing: A Craftsman's Approach*. 4. ed. Boca Raton: CRC Press, 2014. Citado na página 29.

49 Pytest Development Team. *pytest: helps you write better programs*. 2024. Disponível em: https://docs.pytest.org/en/stable/. Acesso em: 20 jun. 2026. Citado 2 vezes nas páginas 29 e 65.

50 ABELES, P. *EJML: Efficient Java Matrix Library*. 2026. Página oficial e documentação. Disponível em: https://ejml.org/. Acesso em: 21 out. 2026. Citado na página 30.

51 CLOUD, A. *Qwen3.7-Plus: Modelo de linguagem de inteligência artificial*. Hangzhou, China: Alibaba Cloud, 2026. Disponível em: https://chat.qwen.ai/. Acesso em: 16 ago. 2026. Citado na página 30.

52 KOPKA, H.; DALY, P. W. *Guide to LaTeX*. 4th. ed. Boston, MA: Addison-Wesley Professional, 2004. (Tools and Techniques for Computer Typesetting). ISBN 0-321-17385-6. Citado na página 30.

53 PIMENTEL, M. F. et al. Convertion of a sequential inductively coupled plasma emission spectrometer into a multichannel simultaneous system using a photodiode array detector. *Journal of Automatic Chemistry*, USA, v. 20, n. 3, p. 69–75, 1998. DOI: https://doi.org/10.1155/S146392469800008X. Disponível em: https://onlinelibrary.wiley.com/doi/10.1155/S146392469800008X. Acesso em: 29 mar. 2025. Citado 2 vezes nas páginas 32 e 34.

54 KRAMER, R. *Chemometric Techniques for Quantitative Analysis*. USA: CRC Press, 1998. 220 p. Citado 2 vezes nas páginas 32 e 34.

55 MARTENS, H.; NAES, T. *Multivariate Calibration*. USA: John Wiley & Sons Ltd, 1989. 399 p. Citado na página 32.

56 HARRIS, C. R. et al. Array programming with NumPy. *Nature*, v. 585, p. 357–362, 2020. Citado na página 33.

57 NumPy Developers. *numpy.linalg.lstsq: Return the least-squares solution to a linear matrix equation*. 2026. Documentação oficial. Disponível em: https://numpy.org/doc/stable/reference/generated/numpy.linalg.lstsq.html. Acesso em: 21 out. 2026. Citado na página 33.

58 ANDERSON, E. et al. *LAPACK Users' Guide*. 3. ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 1999. Citado na página 33.

59 WESTAD, F.; FLÅTEN, G. R. A retrospective view on non-linear methods in chemometrics, and future directions. *Frontiers in Analytical Science*, v. 4, 2024. DOI: https://doi.org/10.3389/frans.2024.1393222. Disponível em: https://www.frontiersin.org/journals/analytical-science/articles/10.3389/frans.2024.1393222/full. Acesso em: 3 out. 2025. Citado 2 vezes nas páginas 35 e 36.

60 PERIS-DIAZ, M. D.; KREZEL, A. A guide to good practice in chemometric methods for vibrational spectroscopy, electrochemistry, and hyphenated mass spectrometry. *TrAC Trends in Analytical Chemistry*, v. 135, p. 116157, 2021. DOI: https://doi.org/10.1016/j.trac.2020.116157. Disponível em: https://www.sciencedirect.com/science/article/pii/S0165993620303861?via%3Dihub. Acesso em: 3 out. 2025. Citado 2 vezes nas páginas 35 e 36.

61 HODSON, T. O. Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not. *Geoscientific Model Development*, Copernicus Publications, v. 15, n. 14, p. 5481–5487, 2022. DOI: doi.org/10.5194/gmd-15-5481-2022. Disponível em: https://pubs.usgs.gov/publication/70236308. Acesso em: 3 mar. 2026. Citado na página 35.

62 EZENARRO, J.; SCHORN-GARCÍA, D. How are chemometric models validated? A systematic review of linear regression models for NIRS data in food analysis. *Journal of Chemometrics*, Wiley, v. 39, n. 6, p. e70036, 2025. DOI: https://doi.org/10.1002/cem.70036. Disponível em: https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/cem.70036. Acesso em: 5 mar. 2026. Citado na página 36.

63 TAKAHASHI, T.; THORNTON, B. Quantitative methods for compensation of matrix effects and self-absorption in laser induced breakdown spectroscopy signals of solids. *Spectrochimica Acta Part B: Atomic Spectroscopy*, v. 138, p. 31–42, 2017. ISSN 0584-8547. Disponível em: https://www.sciencedirect.com/science/article/pii/S0584854716303299. Acesso em: 27 set. 2026. Citado na página 48.

64 CLEGG, D. *MoSCoW Method: Must, Should, Could, Won't have*. 1994. Developed at Oracle UK. Subsequently formalized and hosted by the Agile Business Consortium. Disponível em: https://www.agilebusiness.org/resource/what-is-moscow-prioritization/. Acesso em: 2 out. 2026. Citado na página 52.

65 KNIME. *KNIME: Open for Innovation*. 2026. Página oficial. Plataforma de código aberto para fluxos de dados, ETL, análise de dados e IA preditiva. Acessado em: 10 de outubro de 2026. Disponível em: https://www.knime.com/. Acesso em: 10 out. 2026. Citado na página 53.

66 World Wide Web Consortium (W3C). *Diretrizes de Acessibilidade para Conteúdo Web (WCAG) 2.1*. 2018. Tradução oficial autorizada. Disponível em: https://www.w3c.br/traducoes/wcag/wcag21-pt-BR/. Acesso em: 10 out. 2026. Citado na página 56.

67 World Wide Web Consortium (W3C). *W3C Internationalization (i18n) Activity: Making the World Wide Web worldwide!* 2026. Página oficial. Disponível em: https://www.w3.org/International/. Acesso em: 10 out. 2026. Citado na página 57.

68 SCHWABER, K.; SUTHERLAND, J. *O Guia do Scrum*. 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Acesso em: 20 jun. 2026. Citado na página 64.

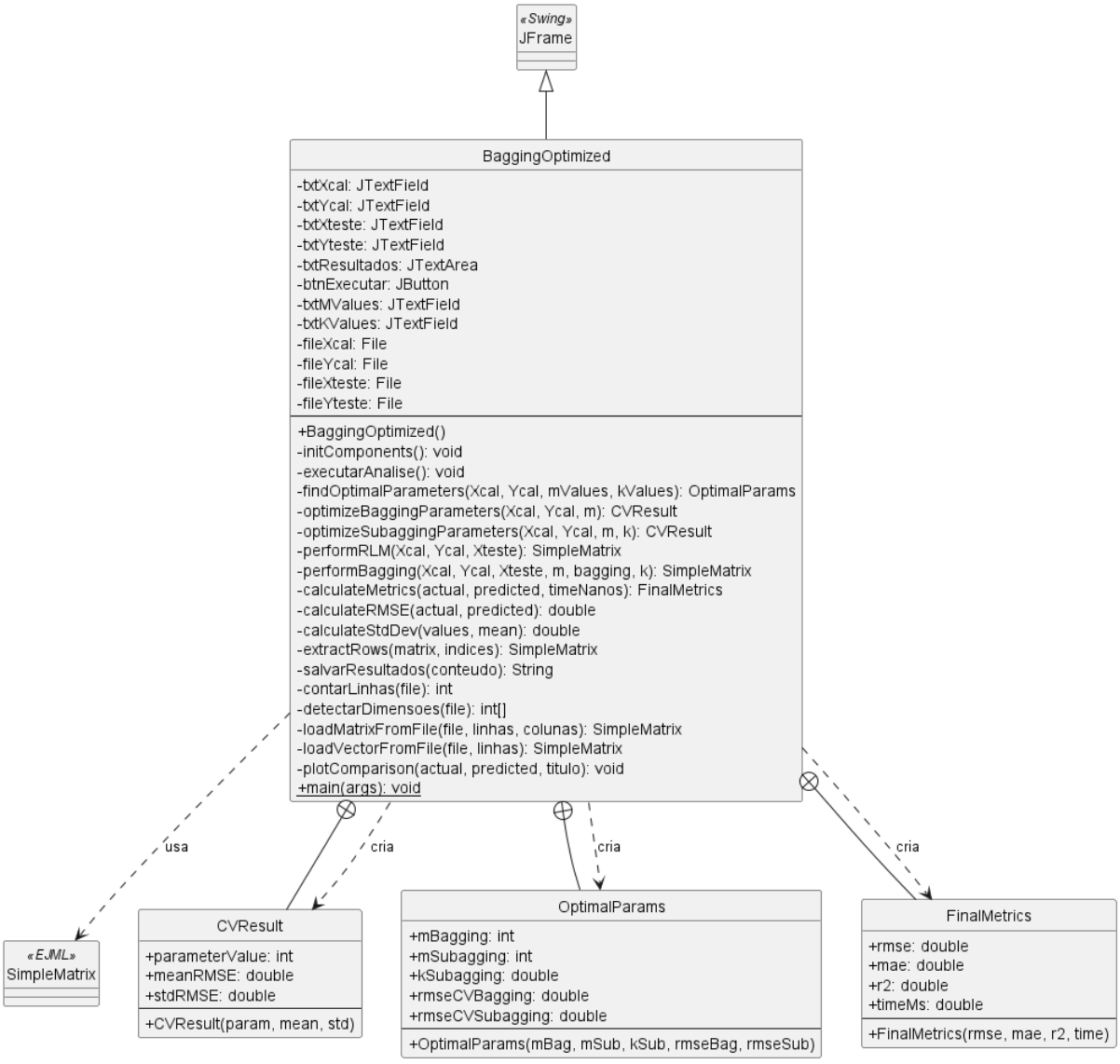
# Apêndices

# APÊNDICE A – DIAGRAMAS UML

Este apêndice reúne os diagramas UML elaborados, com o auxílio da ferramenta PlantUML, ao longo da modernização do sistema, desde a base algorítmica em Java (*Desktop*) até a reescrita final em Python/Streamlit. Os diagramas de classes, de atividades e de sequência de cada versão complementam a descrição apresentada no corpo do texto. Cada diagrama aparece em uma figura individual, com seção e legenda próprias, o que permite a referência cruzada nas demais seções do trabalho.

## A.1 Diagrama de Classes — Java

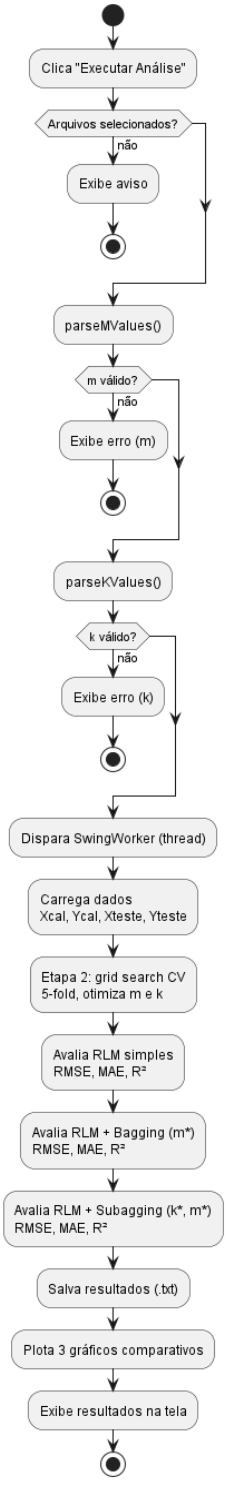
Figura 19 – Diagrama de classes.




Fonte: Wagner Oliveira de Araujo.

## A.2 Diagrama de Atividade — Java

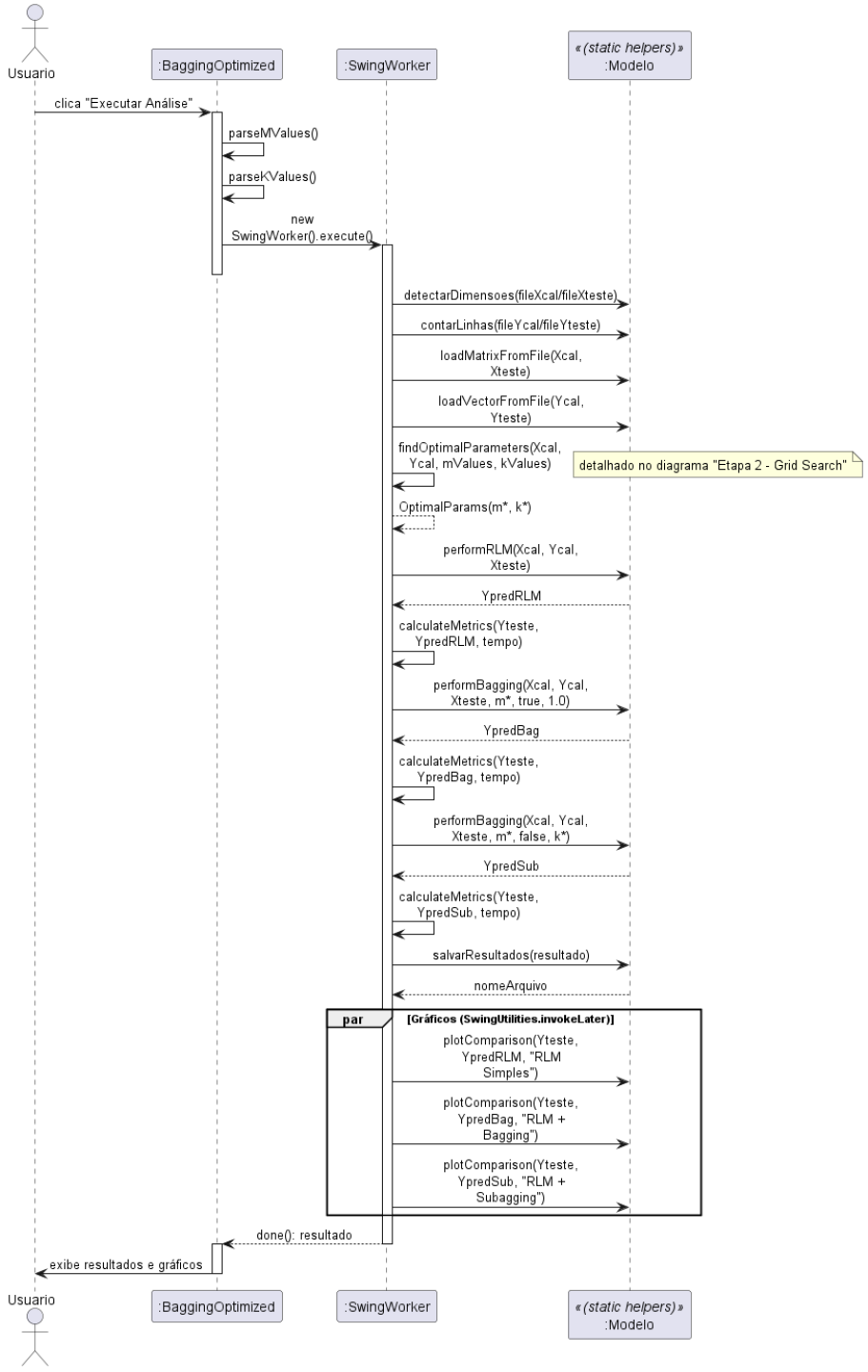
Figura 20 – Diagrama de atividades.




Fonte: Wagner Oliveira de Araujo.

## A.3 Diagrama de Sequência — Java

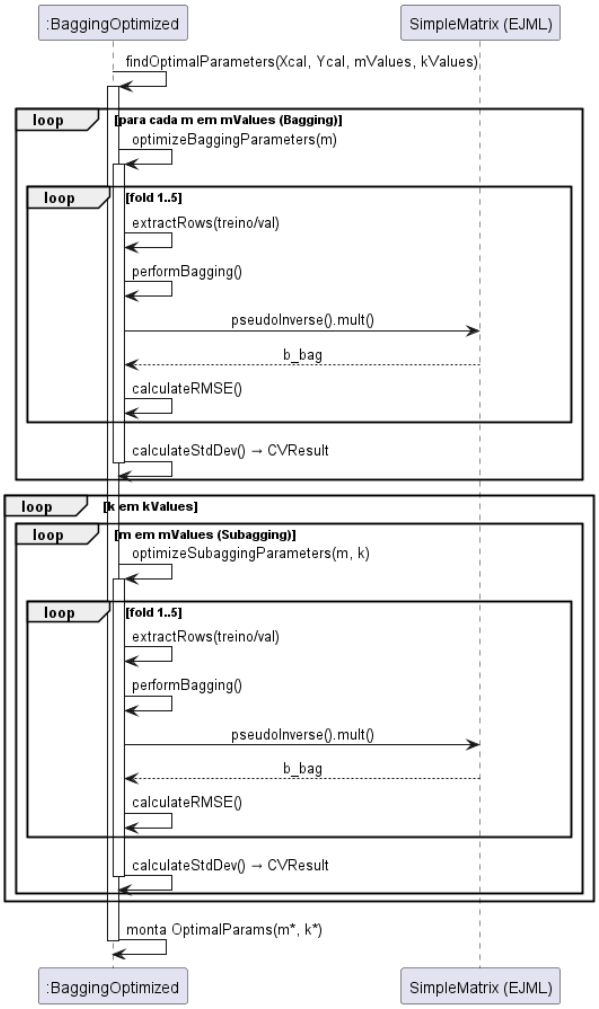
Figura 21 – Diagrama de sequência.




Fonte: Wagner Oliveira de Araujo.

## A.4 Diagrama de Atividades — Java (PlantUML)

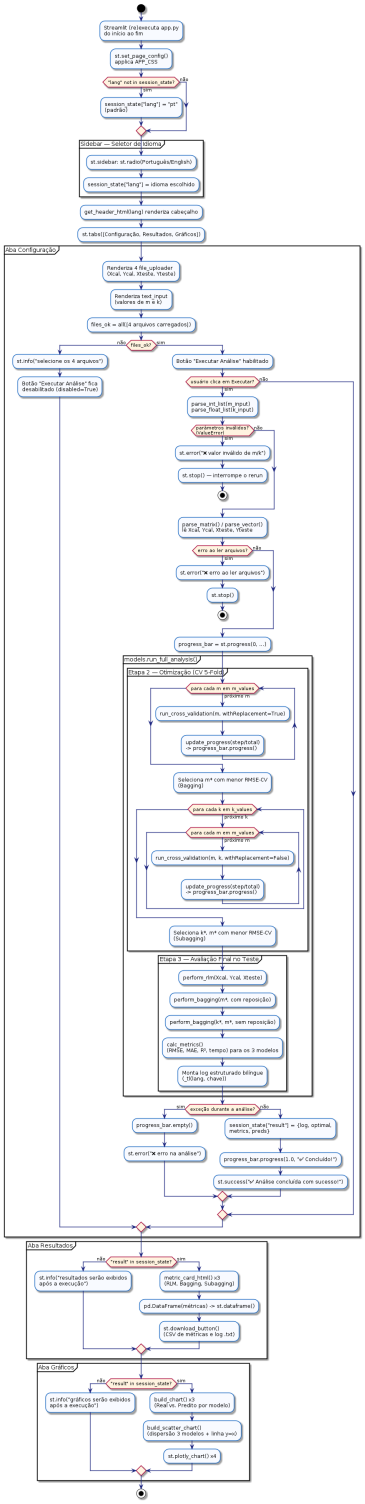
Figura 22 – Diagrama de atividade Java PlantUML .




Fonte: Wagner Oliveira de Araujo.

## A.5 Diagrama de Atividades — Streamlit

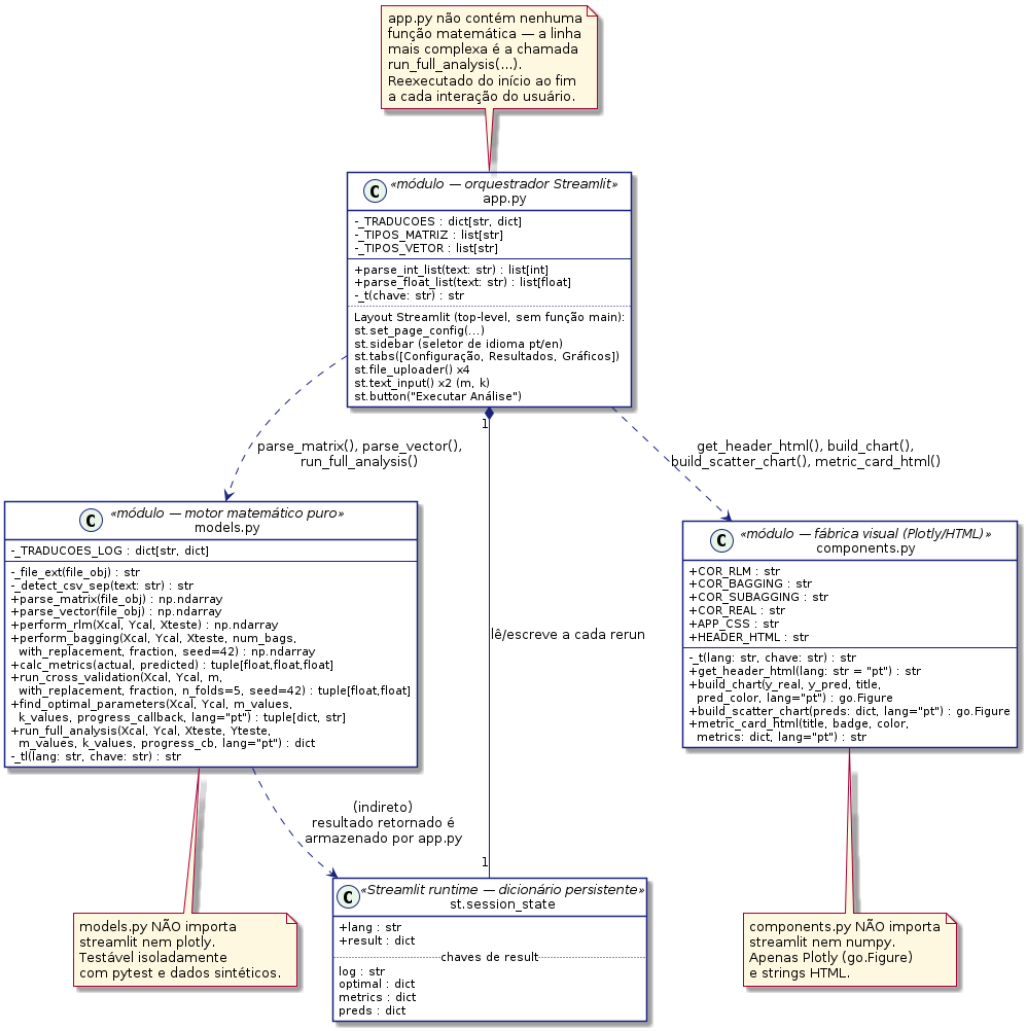
Figura 23 – Diagrama de atividade Python PlantUML.




Fonte: Wagner Oliveira de Araujo.

## A.6 Diagrama de Classes (Módulos) — Streamlit

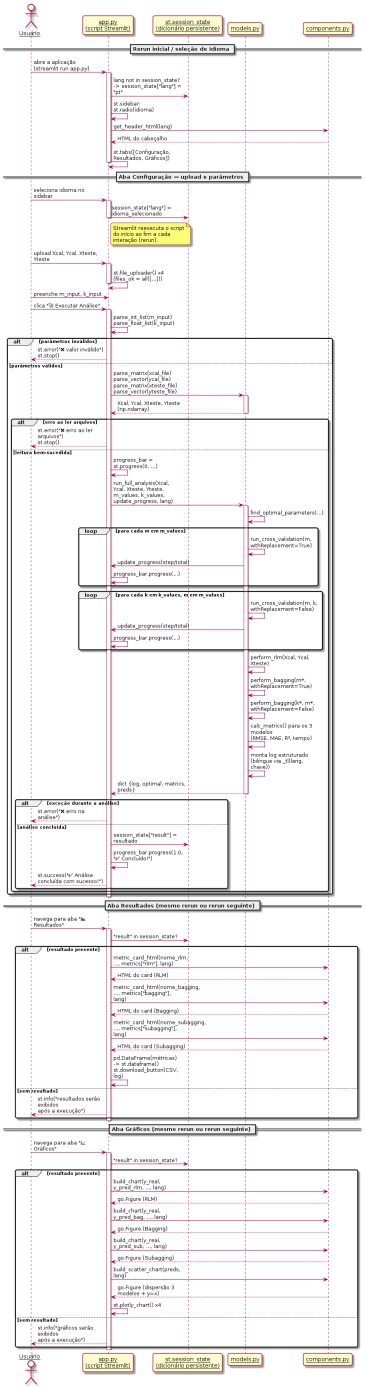
Figura 24 – Diagrama de classes (módulos) PlantUML.




Fonte: Wagner Oliveira de Araujo.

## A.7 Diagrama de Sequência — Streamlit

Figura 25 – Diagrama de sequência PlantUML.




Fonte: Wagner Oliveira de Araujo.

# APÊNDICE B – TABELAS DE PARIDADE AMOSTRA-A-AMOSTRA — JAVA × PYTHON

Este apêndice reúne, para os cinco analitos avaliados (Manganês, Molibdênio, Cromo, Níquel e Ferro) e para as três técnicas de regressão implementadas (RLM, *Bagging* e *Subagging*), a comparação amostra-a-amostra entre o valor predito pelo sistema Java legado e o valor predito pelo sistema Python/Streamlit modernizado, frente ao valor de referência (Y real) do conjunto de teste. Essas tabelas complementam os testes de caixa-preta descritos na Seção 9.2, que verificam a equivalência por meio de métricas agregadas (RMSEP, MAE, $R^2$): aqui, a equivalência é demonstrada em nível de cada predição individual, evidenciando que a migração de plataforma não introduziu perdas de acurácia amostra a amostra.

Tabela 9 – Paridade *Bagging* – Fe.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 200,000 | 201,2347 | 201,2347 | 0,0000 |
| 2 | 170,000 | 171,1741 | 171,1741 | 0,0000 |
| 3 | 160,000 | 162,5467 | 162,5467 | 0,0000 |
| 4 | 160,000 | 160,2073 | 160,2073 | 0,0000 |
| 5 | 150,000 | 151,4182 | 151,4182 | 0,0000 |
| 6 | 130,000 | 131,4313 | 131,4313 | 0,0000 |
| 7 | 140,000 | 140,8831 | 140,8831 | 0,0000 |
| 8 | 170,000 | 171,2164 | 171,2164 | 0,0000 |
| 9 | 200,000 | 199,1764 | 199,1764 | 0,0000 |
| 10 | 160,000 | 159,3144 | 159,3144 | 0,0000 |
| 11 | 226,190 | 231,0960 | 231,0960 | 0,0000 |
| 12 | 222,460 | 222,6728 | 222,6728 | 0,0000 |
| 13 | 222,370 | 226,7032 | 226,7032 | 0,0000 |
| 14 | 187,090 | 188,5555 | 188,5555 | 0,0000 |
| 15 | 186,790 | 184,3893 | 184,3893 | 0,0000 |
| 16 | 185,180 | 191,2621 | 191,2621 | 0,0000 |
| 17 | 186,320 | 187,1853 | 187,1853 | 0,0000 |
| 18 | 184,630 | 182,3263 | 182,3263 | 0,0000 |
| 19 | 189,600 | 196,2627 | 196,2627 | 0,0000 |
| 20 | 188,630 | 192,6022 | 192,6022 | 0,0000 |
| 21 | 187,620 | 186,6603 | 186,6603 | 0,0000 |
| 22 | 201,920 | 204,7633 | 204,7633 | 0,0000 |
| 23 | 202,400 | 201,5573 | 201,5573 | 0,0000 |
| 24 | 201,360 | 201,6813 | 201,6813 | 0,0000 |
| 25 | 130,950 | 134,2735 | 134,2735 | 0,0000 |
| 26 | 132,830 | 133,0646 | 133,0646 | 0,0000 |
| 27 | 177,820 | 178,4886 | 178,4886 | 0,0000 |
| 28 | 177,850 | 178,7110 | 178,7110 | 0,0000 |
| 29 | 217,680 | 221,7617 | 221,7617 | 0,0000 |
| 30 | 218,800 | 221,0220 | 221,0220 | 0,0000 |
| 31 | 216,990 | 218,1427 | 218,1427 | 0,0000 |
| 32 | 170,520 | 175,7446 | 175,7446 | 0,0000 |
| 33 | 170,180 | 171,4528 | 171,4528 | 0,0000 |
| 34 | 170,180 | 171,6698 | 171,6698 | 0,0000 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *Bagging* no analito Ferro (Fe).

Tabela 10 – Paridade *Bagging* – Mn.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 2,000 | 2,0153 | 2,0153 | 0,0000 |
| 2 | 4,000 | 3,9981 | 3,9981 | 0,0000 |
| 3 | 5,000 | 4,9809 | 4,9809 | 0,0000 |
| 4 | 2,500 | 2,5041 | 2,5041 | 0,0000 |
| 5 | 3,500 | 3,4813 | 3,4813 | 0,0000 |
| 6 | 4,500 | 4,4478 | 4,4478 | 0,0000 |
| 7 | 1,500 | 1,5056 | 1,5056 | 0,0000 |
| 8 | 3,700 | 3,6941 | 3,6941 | 0,0000 |
| 9 | 2,200 | 2,0499 | 2,0499 | 0,0000 |
| 10 | 3,200 | 3,1082 | 3,1082 | 0,0000 |
| 11 | 1,250 | 1,2605 | 1,2605 | 0,0000 |
| 12 | 1,229 | 1,0868 | 1,0868 | 0,0000 |
| 13 | 1,228 | 1,0790 | 1,0790 | 0,0000 |
| 14 | 1,050 | 1,1285 | 1,1285 | 0,0000 |
| 15 | 1,048 | 0,9638 | 0,9638 | 0,0000 |
| 16 | 2,016 | 2,1255 | 2,1255 | 0,0000 |
| 17 | 2,028 | 1,9454 | 1,9454 | 0,0000 |
| 18 | 2,010 | 1,8625 | 1,8625 | 0,0000 |
| 19 | 1,093 | 1,1769 | 1,1769 | 0,0000 |
| 20 | 1,087 | 1,0067 | 1,0067 | 0,0000 |
| 21 | 1,081 | 0,9659 | 0,9659 | 0,0000 |
| 22 | 2,297 | 2,3751 | 2,3751 | 0,0000 |
| 23 | 2,302 | 2,1683 | 2,1683 | 0,0000 |
| 24 | 2,290 | 2,1728 | 2,1728 | 0,0000 |
| 25 | 1,935 | 2,0812 | 2,0812 | 0,0000 |
| 26 | 1,963 | 1,9299 | 1,9299 | 0,0000 |
| 27 | 1,768 | 1,6882 | 1,6882 | 0,0000 |
| 28 | 1,768 | 1,6861 | 1,6861 | 0,0000 |
| 29 | 1,505 | 1,4854 | 1,4854 | 0,0000 |
| 30 | 1,513 | 1,3303 | 1,3303 | 0,0000 |
| 31 | 1,500 | 1,3175 | 1,3175 | 0,0000 |
| 32 | 3,783 | 3,9578 | 3,9578 | 0,0000 |
| 33 | 3,776 | 3,6993 | 3,6993 | 0,0000 |
| 34 | 3,776 | 3,6902 | 3,6902 | 0,0000 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *Bagging* no analito Manganês (Mn).

Tabela 11 – Paridade *Bagging* – Ni.

| Amostra | Y real | Y predito (Java) | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 15,000 | 15,7904 | 15,7904 | 0,0000 |
| 2 | 30,000 | 30,4864 | 30,4864 | 0,0000 |
| 3 | 20,000 | 20,6410 | 20,6410 | 0,0000 |
| 4 | 50,000 | 50,7132 | 50,7132 | 0,0000 |
| 5 | 45,000 | 46,0191 | 46,0191 | 0,0000 |
| 6 | 60,000 | 60,3324 | 60,3324 | 0,0000 |
| 7 | 35,000 | 35,6279 | 35,6279 | 0,0000 |
| 8 | 48,000 | 48,7008 | 48,7008 | 0,0000 |
| 9 | 25,000 | 25,0248 | 25,0248 | 0,0000 |
| 10 | 38,000 | 38,1906 | 38,1906 | 0,0000 |
| 11 | 31,242 | 33,7431 | 33,7431 | 0,0000 |
| 12 | 31,434 | 33,4756 | 33,4756 | 0,0000 |
| 13 | 31,149 | 32,2768 | 32,2768 | 0,0000 |
| 14 | 52,029 | 55,5627 | 55,5627 | 0,0000 |
| 15 | 52,775 | 56,3382 | 56,3382 | 0,0000 |
| 16 | 21,809 | 23,4771 | 23,4771 | 0,0000 |
| 17 | 21,813 | 23,2773 | 23,2773 | 0,0000 |
| 18 | 25,048 | 27,0453 | 27,0453 | 0,0000 |
| 19 | 24,999 | 26,5735 | 26,5735 | 0,0000 |
| 20 | 24,999 | 26,5728 | 26,5728 | 0,0000 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *Bagging* no analito Níquel (Ni).

Tabela 12 – Paridade *Bagging* – Mo.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 3,000 | 3,2844 | 3,2844 | 0,0000 |
| 2 | 8,000 | 8,1148 | 8,1148 | 0,0000 |
| 3 | 5,000 | 5,2699 | 5,2699 | 0,0000 |
| 4 | 3,600 | 3,6603 | 3,6603 | 0,0000 |
| 5 | 4,000 | 4,2607 | 4,2607 | 0,0000 |
| 6 | 5,000 | 5,1603 | 5,1603 | 0,0000 |
| 7 | 9,000 | 9,0995 | 9,0995 | 0,0000 |
| 8 | 7,000 | 7,0161 | 7,0161 | 0,0000 |
| 9 | 2,800 | 2,6208 | 2,6208 | 0,0000 |
| 10 | 10,000 | 10,0238 | 10,0238 | 0,0000 |
| 11 | 2,422 | 2,5660 | 2,5660 | 0,0000 |
| 12 | 2,383 | 2,1786 | 2,1786 | 0,0000 |
| 13 | 2,382 | 2,5483 | 2,5483 | 0,0000 |
| 14 | 1,742 | 2,1182 | 2,1182 | 0,0000 |
| 15 | 1,746 | 1,9412 | 1,9412 | 0,0000 |
| 16 | 1,737 | 2,0297 | 2,0297 | 0,0000 |
| 17 | 5,547 | 5,8904 | 5,8904 | 0,0000 |
| 18 | 5,548 | 6,0116 | 6,0116 | 0,0000 |
| 19 | 6,407 | 7,2260 | 7,2260 | 0,0000 |
| 20 | 6,394 | 6,9924 | 6,9924 | 0,0000 |
| 21 | 6,394 | 6,9658 | 6,9658 | 0,0000 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *Bagging* no analito Molibdênio (Mo).

Tabela 13 – Paridade *RLM* – Cr.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 30,000 | 30,4401 | 30,4401 | 0,0 |
| 2 | 40,000 | 40,6027 | 40,6027 | 0,0 |
| 3 | 60,000 | 60,5929 | 60,5929 | 0,0 |
| 4 | 35,000 | 35,2358 | 35,2358 | 0,0 |
| 5 | 45,000 | 45,2398 | 45,2398 | 0,0 |
| 6 | 48,000 | 47,8566 | 47,8566 | 0,0 |
| 7 | 65,000 | 65,1068 | 65,1068 | 0,0 |
| 8 | 23,000 | 22,9505 | 22,9505 | 0,0 |
| 9 | 25,000 | 24,1470 | 24,1470 | 0,0 |
| 10 | 37,000 | 36,5572 | 36,5572 | 0,0 |
| 11 | 23,103 | 22,9549 | 22,9549 | 0,0 |
| 12 | 22,722 | 20,7987 | 20,7987 | 0,0 |
| 13 | 22,713 | 22,0081 | 22,0081 | 0,0 |
| 14 | 60,054 | 60,4145 | 60,4145 | 0,0 |
| 15 | 59,959 | 57,6637 | 57,6637 | 0,0 |
| 16 | 32,250 | 33,9133 | 33,9133 | 0,0 |
| 17 | 32,448 | 31,8925 | 31,8925 | 0,0 |
| 18 | 32,154 | 30,9060 | 30,9060 | 0,0 |
| 19 | 60,996 | 63,0862 | 63,0862 | 0,0 |
| 20 | 60,684 | 60,5654 | 60,5654 | 0,0 |
| 21 | 60,360 | 58,6691 | 58,6691 | 0,0 |
| 22 | 40,636 | 41,9072 | 41,9072 | 0,0 |
| 23 | 40,733 | 39,7794 | 39,7794 | 0,0 |
| 24 | 40,524 | 39,8836 | 39,8836 | 0,0 |
| 25 | 64,597 | 66,6507 | 66,6507 | 0,0 |
| 26 | 65,522 | 65,1239 | 65,1239 | 0,0 |
| 27 | 44,707 | 43,7105 | 43,7105 | 0,0 |
| 28 | 44,716 | 43,8560 | 43,8560 | 0,0 |
| 29 | 30,022 | 30,5536 | 30,5536 | 0,0 |
| 30 | 30,177 | 29,0491 | 29,0491 | 0,0 |
| 31 | 29,926 | 28,6567 | 28,6567 | 0,0 |
| 32 | 44,926 | 46,8013 | 46,8013 | 0,0 |
| 33 | 44,837 | 44,3839 | 44,3839 | 0,0 |
| 34 | 44,837 | 44,3501 | 44,3501 | 0,0 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *RLM* no analito Cromo (Cr).

Tabela 14 – Paridade *RLM* – Mo.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 3,000 | 3,2312 | 3,2312 | 0,0 |
| 2 | 8,000 | 8,0690 | 8,0690 | 0,0 |
| 3 | 5,000 | 5,2252 | 5,2252 | 0,0 |
| 4 | 3,600 | 3,6276 | 3,6276 | 0,0 |
| 5 | 4,000 | 4,1362 | 4,1362 | 0,0 |
| 6 | 5,000 | 5,0582 | 5,0582 | 0,0 |
| 7 | 9,000 | 9,0103 | 9,0103 | 0,0 |
| 8 | 7,000 | 7,0129 | 7,0129 | 0,0 |
| 9 | 2,800 | 2,6673 | 2,6673 | 0,0 |
| 10 | 10,000 | 9,9628 | 9,9628 | 0,0 |
| 11 | 2,422 | 2,4751 | 2,4751 | 0,0 |
| 12 | 2,383 | 2,1994 | 2,1994 | 0,0 |
| 13 | 2,382 | 2,4692 | 2,4692 | 0,0 |
| 14 | 1,742 | 2,0321 | 2,0321 | 0,0 |
| 15 | 1,746 | 1,8651 | 1,8651 | 0,0 |
| 16 | 1,737 | 1,9493 | 1,9493 | 0,0 |
| 17 | 5,547 | 5,7680 | 5,7680 | 0,0 |
| 18 | 5,548 | 5,9010 | 5,9010 | 0,0 |
| 19 | 6,407 | 7,1323 | 7,1323 | 0,0 |
| 20 | 6,394 | 6,8709 | 6,8709 | 0,0 |
| 21 | 6,394 | 6,8308 | 6,8308 | 0,0 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *RLM* no analito Molibdênio (Mo).

Tabela 15 – Paridade *RLM* – Fe.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 200,000 | 201,3675 | 201,3675 | 0,0 |
| 2 | 170,000 | 171,6284 | 171,6284 | 0,0 |
| 3 | 160,000 | 162,8586 | 162,8586 | 0,0 |
| 4 | 160,000 | 160,3428 | 160,3428 | 0,0 |
| 5 | 150,000 | 151,4730 | 151,4730 | 0,0 |
| 6 | 130,000 | 131,2829 | 131,2829 | 0,0 |
| 7 | 140,000 | 140,9123 | 140,9123 | 0,0 |
| 8 | 170,000 | 172,1247 | 172,1247 | 0,0 |
| 9 | 200,000 | 200,0870 | 200,0870 | 0,0 |
| 10 | 160,000 | 159,7659 | 159,7659 | 0,0 |
| 11 | 226,190 | 230,8332 | 230,8332 | 0,0 |
| 12 | 222,460 | 223,5541 | 223,5541 | 0,0 |
| 13 | 222,370 | 227,3945 | 227,3945 | 0,0 |
| 14 | 187,090 | 188,8822 | 188,8822 | 0,0 |
| 15 | 186,790 | 184,4505 | 184,4505 | 0,0 |
| 16 | 185,180 | 190,9301 | 190,9301 | 0,0 |
| 17 | 186,320 | 187,3890 | 187,3890 | 0,0 |
| 18 | 184,630 | 182,9002 | 182,9002 | 0,0 |
| 19 | 189,600 | 196,1038 | 196,1038 | 0,0 |
| 20 | 188,630 | 192,7129 | 192,7129 | 0,0 |
| 21 | 187,620 | 186,7261 | 186,7261 | 0,0 |
| 22 | 201,920 | 204,5577 | 204,5577 | 0,0 |
| 23 | 202,400 | 201,9671 | 201,9671 | 0,0 |
| 24 | 201,360 | 202,1703 | 202,1703 | 0,0 |
| 25 | 130,950 | 134,1479 | 134,1479 | 0,0 |
| 26 | 132,830 | 132,1917 | 132,1917 | 0,0 |
| 27 | 177,820 | 178,5687 | 178,5687 | 0,0 |
| 28 | 177,850 | 178,9440 | 178,9440 | 0,0 |
| 29 | 217,680 | 221,3581 | 221,3581 | 0,0 |
| 30 | 218,800 | 221,6059 | 221,6059 | 0,0 |
| 31 | 216,990 | 219,0625 | 219,0625 | 0,0 |
| 32 | 170,520 | 175,9362 | 175,9362 | 0,0 |
| 33 | 170,180 | 171,7899 | 171,7899 | 0,0 |
| 34 | 170,180 | 172,1441 | 172,1441 | 0,0 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *RLM* no analito Ferro (Fe).

Tabela 16 – Paridade *RLM* – Ni.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 15,000 | 15,6181 | 15,6181 | 0,0 |
| 2 | 30,000 | 30,5664 | 30,5664 | 0,0 |
| 3 | 20,000 | 20,6828 | 20,6828 | 0,0 |
| 4 | 50,000 | 50,5499 | 50,5499 | 0,0 |
| 5 | 45,000 | 45,9700 | 45,9700 | 0,0 |
| 6 | 60,000 | 60,3713 | 60,3713 | 0,0 |
| 7 | 35,000 | 35,6232 | 35,6232 | 0,0 |
| 8 | 48,000 | 48,8117 | 48,8117 | 0,0 |
| 9 | 25,000 | 25,3044 | 25,3044 | 0,0 |
| 10 | 38,000 | 38,2514 | 38,2514 | 0,0 |
| 11 | 31,242 | 33,4020 | 33,4020 | 0,0 |
| 12 | 31,434 | 33,4244 | 33,4244 | 0,0 |
| 13 | 31,149 | 32,4378 | 32,4378 | 0,0 |
| 14 | 52,029 | 55,4537 | 55,4537 | 0,0 |
| 15 | 52,775 | 56,0699 | 56,0699 | 0,0 |
| 16 | 21,809 | 23,4975 | 23,4975 | 0,0 |
| 17 | 21,813 | 23,3956 | 23,3956 | 0,0 |
| 18 | 25,048 | 26,8478 | 26,8478 | 0,0 |
| 19 | 24,999 | 26,6769 | 26,6769 | 0,0 |
| 20 | 24,999 | 26,8538 | 26,8538 | 0,0 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *RLM* no analito Níquel (Ni).

Tabela 17 – Paridade *RLM* – Mn.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 2,000 | 2,0441 | 2,0441 | 0,0 |
| 2 | 4,000 | 4,0317 | 4,0317 | 0,0 |
| 3 | 5,000 | 4,9755 | 4,9755 | 0,0 |
| 4 | 2,500 | 2,5277 | 2,5277 | 0,0 |
| 5 | 3,500 | 3,5166 | 3,5166 | 0,0 |
| 6 | 4,500 | 4,4755 | 4,4755 | 0,0 |
| 7 | 1,500 | 1,5306 | 1,5306 | 0,0 |
| 8 | 3,700 | 3,7306 | 3,7306 | 0,0 |
| 9 | 2,200 | 2,0483 | 2,0483 | 0,0 |
| 10 | 3,200 | 3,1410 | 3,1410 | 0,0 |
| 11 | 1,250 | 1,3246 | 1,3246 | 0,0 |
| 12 | 1,229 | 1,1092 | 1,1092 | 0,0 |
| 13 | 1,228 | 1,1021 | 1,1021 | 0,0 |
| 14 | 1,050 | 1,1513 | 1,1513 | 0,0 |
| 15 | 1,048 | 0,9547 | 0,9547 | 0,0 |
| 16 | 2,016 | 2,1754 | 2,1754 | 0,0 |
| 17 | 2,028 | 1,9594 | 1,9594 | 0,0 |
| 18 | 2,010 | 1,8714 | 1,8714 | 0,0 |
| 19 | 1,093 | 1,1930 | 1,1930 | 0,0 |
| 20 | 1,087 | 0,9912 | 0,9912 | 0,0 |
| 21 | 1,081 | 0,9568 | 0,9568 | 0,0 |
| 22 | 2,297 | 2,4229 | 2,4229 | 0,0 |
| 23 | 2,302 | 2,1744 | 2,1744 | 0,0 |
| 24 | 2,290 | 2,1834 | 2,1834 | 0,0 |
| 25 | 1,935 | 2,1058 | 2,1058 | 0,0 |
| 26 | 1,963 | 1,9624 | 1,9624 | 0,0 |
| 27 | 1,768 | 1,7019 | 1,7019 | 0,0 |
| 28 | 1,768 | 1,6913 | 1,6913 | 0,0 |
| 29 | 1,505 | 1,5456 | 1,5456 | 0,0 |
| 30 | 1,513 | 1,3417 | 1,3417 | 0,0 |
| 31 | 1,500 | 1,3242 | 1,3242 | 0,0 |
| 32 | 3,783 | 4,0186 | 4,0186 | 0,0 |
| 33 | 3,776 | 3,7449 | 3,7449 | 0,0 |
| 34 | 3,776 | 3,7318 | 3,7318 | 0,0 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *RLM* no analito Manganês (Mn).

Tabela 18 – Paridade *Subagging* – Cr.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 30,000 | 30,1547 | 30,1547 | 0,0 |
| 2 | 40,000 | 39,9189 | 39,9189 | 0,0 |
| 3 | 60,000 | 59,9876 | 59,9876 | 0,0 |
| 4 | 35,000 | 34,8894 | 34,8894 | 0,0 |
| 5 | 45,000 | 44,6394 | 44,6394 | 0,0 |
| 6 | 48,000 | 47,3280 | 47,3280 | 0,0 |
| 7 | 65,000 | 64,7895 | 64,7895 | 0,0 |
| 8 | 23,000 | 22,7179 | 22,7179 | 0,0 |
| 9 | 25,000 | 23,9207 | 23,9207 | 0,0 |
| 10 | 37,000 | 36,0523 | 36,0523 | 0,0 |
| 11 | 23,103 | 22,8577 | 22,8577 | 0,0 |
| 12 | 22,722 | 20,5236 | 20,5236 | 0,0 |
| 13 | 22,713 | 21,6248 | 21,6248 | 0,0 |
| 14 | 60,054 | 60,0747 | 60,0747 | 0,0 |
| 15 | 59,959 | 57,2228 | 57,2228 | 0,0 |
| 16 | 32,250 | 33,6311 | 33,6311 | 0,0 |
| 17 | 32,448 | 31,6253 | 31,6253 | 0,0 |
| 18 | 32,154 | 30,5525 | 30,5525 | 0,0 |
| 19 | 60,996 | 62,9301 | 62,9301 | 0,0 |
| 20 | 60,684 | 60,3038 | 60,3038 | 0,0 |
| 21 | 60,360 | 58,2638 | 58,2638 | 0,0 |
| 22 | 40,636 | 41,4631 | 41,4631 | 0,0 |
| 23 | 40,733 | 39,3369 | 39,3369 | 0,0 |
| 24 | 40,524 | 39,3585 | 39,3585 | 0,0 |
| 25 | 64,597 | 66,5127 | 66,5127 | 0,0 |
| 26 | 65,522 | 65,1206 | 65,1206 | 0,0 |
| 27 | 44,707 | 43,1986 | 43,1986 | 0,0 |
| 28 | 44,716 | 43,3023 | 43,3023 | 0,0 |
| 29 | 30,022 | 30,3558 | 30,3558 | 0,0 |
| 30 | 30,177 | 28,6899 | 28,6899 | 0,0 |
| 31 | 29,926 | 28,1980 | 28,1980 | 0,0 |
| 32 | 44,926 | 46,1193 | 46,1193 | 0,0 |
| 33 | 44,837 | 43,7828 | 43,7828 | 0,0 |
| 34 | 44,837 | 43,6620 | 43,6620 | 0,0 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *Subagging* no analito Cromo (Cr).

Tabela 19 – Paridade *Subagging* – Fe.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 200,000 | 201,0091 | 201,0091 | 0,0 |
| 2 | 170,000 | 170,8332 | 170,8332 | 0,0 |
| 3 | 160,000 | 162,3268 | 162,3268 | 0,0 |
| 4 | 160,000 | 160,0403 | 160,0403 | 0,0 |
| 5 | 150,000 | 151,2045 | 151,2045 | 0,0 |
| 6 | 130,000 | 131,2211 | 131,2211 | 0,0 |
| 7 | 140,000 | 140,8559 | 140,8559 | 0,0 |
| 8 | 170,000 | 171,1367 | 171,1367 | 0,0 |
| 9 | 200,000 | 199,1951 | 199,1951 | 0,0 |
| 10 | 160,000 | 159,0988 | 159,0988 | 0,0 |
| 11 | 226,190 | 231,0943 | 231,0943 | 0,0 |
| 12 | 222,460 | 222,8628 | 222,8628 | 0,0 |
| 13 | 222,370 | 226,8653 | 226,8653 | 0,0 |
| 14 | 187,090 | 188,4710 | 188,4710 | 0,0 |
| 15 | 186,790 | 184,3402 | 184,3402 | 0,0 |
| 16 | 185,180 | 190,9941 | 190,9941 | 0,0 |
| 17 | 186,320 | 187,1093 | 187,1093 | 0,0 |
| 18 | 184,630 | 182,3256 | 182,3256 | 0,0 |
| 19 | 189,600 | 196,1556 | 196,1556 | 0,0 |
| 20 | 188,630 | 192,6410 | 192,6410 | 0,0 |
| 21 | 187,620 | 186,6622 | 186,6622 | 0,0 |
| 22 | 201,920 | 204,5022 | 204,5022 | 0,0 |
| 23 | 202,400 | 201,5124 | 201,5124 | 0,0 |
| 24 | 201,360 | 201,6347 | 201,6347 | 0,0 |
| 25 | 130,950 | 134,1539 | 134,1539 | 0,0 |
| 26 | 132,830 | 133,0960 | 133,0960 | 0,0 |
| 27 | 177,820 | 178,3030 | 178,3030 | 0,0 |
| 28 | 177,850 | 178,5385 | 178,5385 | 0,0 |
| 29 | 217,680 | 221,6976 | 221,6976 | 0,0 |
| 30 | 218,800 | 221,1559 | 221,1559 | 0,0 |
| 31 | 216,990 | 218,2779 | 218,2779 | 0,0 |
| 32 | 170,520 | 175,3388 | 175,3388 | 0,0 |
| 33 | 170,180 | 171,2162 | 171,2162 | 0,0 |
| 34 | 170,180 | 171,4580 | 171,4580 | 0,0 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *Subagging* no analito Ferro (Fe).

Tabela 20 – Paridade *Subagging* – Mo.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 3,000 | 3,2461 | 3,2461 | 0,0 |
| 2 | 8,000 | 8,0772 | 8,0772 | 0,0 |
| 3 | 5,000 | 5,2196 | 5,2196 | 0,0 |
| 4 | 3,600 | 3,6313 | 3,6313 | 0,0 |
| 5 | 4,000 | 4,1708 | 4,1708 | 0,0 |
| 6 | 5,000 | 5,0867 | 5,0867 | 0,0 |
| 7 | 9,000 | 9,0419 | 9,0419 | 0,0 |
| 8 | 7,000 | 7,0083 | 7,0083 | 0,0 |
| 9 | 2,800 | 2,6193 | 2,6193 | 0,0 |
| 10 | 10,000 | 9,9699 | 9,9699 | 0,0 |
| 11 | 2,422 | 2,5253 | 2,5253 | 0,0 |
| 12 | 2,383 | 2,1609 | 2,1609 | 0,0 |
| 13 | 2,382 | 2,4665 | 2,4665 | 0,0 |
| 14 | 1,742 | 2,0690 | 2,0690 | 0,0 |
| 15 | 1,746 | 1,8521 | 1,8521 | 0,0 |
| 16 | 1,737 | 1,9387 | 1,9387 | 0,0 |
| 17 | 5,547 | 5,7878 | 5,7878 | 0,0 |
| 18 | 5,548 | 5,9076 | 5,9076 | 0,0 |
| 19 | 6,407 | 7,1674 | 7,1674 | 0,0 |
| 20 | 6,394 | 6,8991 | 6,8991 | 0,0 |
| 21 | 6,394 | 6,8579 | 6,8579 | 0,0 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *Subagging* no analito Molibdênio (Mo).

Tabela 21 – Paridade *Subagging* – Mn.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 2,000 | 2,0088 | 2,0088 | 0,0 |
| 2 | 4,000 | 3,9880 | 3,9880 | 0,0 |
| 3 | 5,000 | 4,9797 | 4,9797 | 0,0 |
| 4 | 2,500 | 2,4954 | 2,4954 | 0,0 |
| 5 | 3,500 | 3,4684 | 3,4684 | 0,0 |
| 6 | 4,500 | 4,4303 | 4,4303 | 0,0 |
| 7 | 1,500 | 1,5007 | 1,5007 | 0,0 |
| 8 | 3,700 | 3,6739 | 3,6739 | 0,0 |
| 9 | 2,200 | 2,0319 | 2,0319 | 0,0 |
| 10 | 3,200 | 3,0903 | 3,0903 | 0,0 |
| 11 | 1,250 | 1,2602 | 1,2602 | 0,0 |
| 12 | 1,229 | 1,0783 | 1,0783 | 0,0 |
| 13 | 1,228 | 1,0738 | 1,0738 | 0,0 |
| 14 | 1,050 | 1,1324 | 1,1324 | 0,0 |
| 15 | 1,048 | 0,9554 | 0,9554 | 0,0 |
| 16 | 2,016 | 2,1216 | 2,1216 | 0,0 |
| 17 | 2,028 | 1,9282 | 1,9282 | 0,0 |
| 18 | 2,010 | 1,8453 | 1,8453 | 0,0 |
| 19 | 1,093 | 1,1846 | 1,1846 | 0,0 |
| 20 | 1,087 | 0,9996 | 0,9996 | 0,0 |
| 21 | 1,081 | 0,9579 | 0,9579 | 0,0 |
| 22 | 2,297 | 2,3711 | 2,3711 | 0,0 |
| 23 | 2,302 | 2,1490 | 2,1490 | 0,0 |
| 24 | 2,290 | 2,1537 | 2,1537 | 0,0 |
| 25 | 1,935 | 2,0800 | 2,0800 | 0,0 |
| 26 | 1,963 | 1,9192 | 1,9192 | 0,0 |
| 27 | 1,768 | 1,6718 | 1,6718 | 0,0 |
| 28 | 1,768 | 1,6675 | 1,6675 | 0,0 |
| 29 | 1,505 | 1,4858 | 1,4858 | 0,0 |
| 30 | 1,513 | 1,3205 | 1,3205 | 0,0 |
| 31 | 1,500 | 1,3057 | 1,3057 | 0,0 |
| 32 | 3,783 | 3,9483 | 3,9483 | 0,0 |
| 33 | 3,776 | 3,6782 | 3,6782 | 0,0 |
| 34 | 3,776 | 3,6669 | 3,6669 | 0,0 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *Subagging* no analito Manganês (Mn).

Tabela 22 – Paridade *Subagging* – Ni.

| Amostra | Y real | Y predito (Java)[a] | Y predito (Python) | Δ |
|---|---|---|---|---|
| 1 | 15,000 | 15,7160 | 15,7160 | 0,0 |
| 2 | 30,000 | 30,4808 | 30,4808 | 0,0 |
| 3 | 20,000 | 20,5533 | 20,5533 | 0,0 |
| 4 | 50,000 | 50,6579 | 50,6579 | 0,0 |
| 5 | 45,000 | 45,9194 | 45,9194 | 0,0 |
| 6 | 60,000 | 60,2658 | 60,2658 | 0,0 |
| 7 | 35,000 | 35,6042 | 35,6042 | 0,0 |
| 8 | 48,000 | 48,6265 | 48,6265 | 0,0 |
| 9 | 25,000 | 24,9836 | 24,9836 | 0,0 |
| 10 | 38,000 | 38,1508 | 38,1508 | 0,0 |
| 11 | 31,242 | 33,6217 | 33,6217 | 0,0 |
| 12 | 31,434 | 33,2719 | 33,2719 | 0,0 |
| 13 | 31,149 | 32,1552 | 32,1552 | 0,0 |
| 14 | 52,029 | 55,5191 | 55,5191 | 0,0 |
| 15 | 52,775 | 56,1578 | 56,1578 | 0,0 |
| 16 | 21,809 | 23,3643 | 23,3643 | 0,0 |
| 17 | 21,813 | 23,1637 | 23,1637 | 0,0 |
| 18 | 25,048 | 26,9728 | 26,9728 | 0,0 |
| 19 | 24,999 | 26,4848 | 26,4848 | 0,0 |
| 20 | 24,999 | 26,4882 | 26,4882 | 0,0 |

Fonte: Wagner Oliveira de Araujo.

[a] Tipo de dado utilizado em Java foi o *double*; no Python, o equivalente é o *float64* do NumPy — ambos representações IEEE 754 de precisão dupla (64 bits), o que garante paridade ao nível da precisão exibida (quatro casas decimais) entre as duas implementações.

**Nota:** $\Delta = |\mathrm{Y}_{\mathrm{Python}} - \mathrm{Y}_{\mathrm{Java}}|$. Todos os valores de $\Delta$ são iguais a zero, indicando paridade perfeita entre as implementações Java e Python para a técnica *Subagging* no analito Níquel (Ni).

# APÊNDICE C – RESULTADO DOS TESTES

O Código 1 apresenta a saída de uma sessão do *pytest* 9.1.0 (Python 3.12.10, Windows) que executou o `test_suite_5.py`, com 92 testes: 21 unitários (leitura de matrizes TXT/CSV), 60 de caixa preta (RLM, *Bagging* e *Subagging*, com RMSEP, MAE e $R^2$ para Cr, Ni e Fe) e 11 de estresse (tempo, RAM e escalabilidade). Conforme o Código 1, todos os testes passaram, sem falhas, em 75,25 segundos.

Código 1 – `Teste Consolidado`

```
============================= test session starts ==============================
platform win32 -- Python 3.12.10, pytest-9.1.0, pluggy-1.6.0
C:\Users\wagner.araujo\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Pyt⌋
↪  hon.3.12_qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
rootdir: L:\TCCI\D_Programacao\streamlit-app\tests
plugins: anyio-4.13.0
collecting ... collected 92 items

test_suite_5.py::TestUnitario::test_parse_matrix_txt_tab PASSED          [  1%]
test_suite_5.py::TestUnitario::test_parse_matrix_txt_virgula_br PASSED   [  2%]
test_suite_5.py::TestUnitario::test_parse_matrix_csv_ponto_virgula PASSED [  3%]
test_suite_5.py::TestUnitario::test_parse_matrix_csv_virgula PASSED      [  4%]
test_suite_5.py::TestUnitario::test_parse_matrix_ignora_linhas_vazias PASSED [  5%]
test_suite_5.py::TestUnitario::test_parse_vector_txt PASSED              [  6%]
test_suite_5.py::TestUnitario::test_parse_vector_csv PASSED              [  7%]
test_suite_5.py::TestUnitario::test_parse_vector_virgula_br PASSED       [  8%]
test_suite_5.py::TestUnitario::test_perform_rlm_shape PASSED             [  9%]
test_suite_5.py::TestUnitario::test_perform_rlm_sistema_exato PASSED     [ 10%]
test_suite_5.py::TestUnitario::test_perform_rlm_r2_alto PASSED           [ 11%]
test_suite_5.py::TestUnitario::test_bagging_shape PASSED                 [ 13%]
test_suite_5.py::TestUnitario::test_subagging_shape PASSED               [ 14%]
test_suite_5.py::TestUnitario::test_bagging_reprodutivel PASSED          [ 15%]
test_suite_5.py::TestUnitario::test_bagging_seeds_diferentes PASSED      [ 16%]
test_suite_5.py::TestUnitario::test_calc_metrics_perfeito PASSED         [ 17%]
test_suite_5.py::TestUnitario::test_calc_metrics_valores_conhecidos PASSED [ 18%]
test_suite_5.py::TestUnitario::test_calc_metrics_r2_negativo PASSED      [ 19%]
test_suite_5.py::TestUnitario::test_calc_metrics_retorna_floats PASSED   [ 20%]
test_suite_5.py::TestUnitario::test_cv_retorna_media_e_std PASSED        [ 21%]
test_suite_5.py::TestUnitario::test_cv_subagging_fracao PASSED           [ 22%]
test_suite_5.py::TestCaixaPreta::test_rlm_rmse[Mn] PASSED                [ 23%]
test_suite_5.py::TestCaixaPreta::test_rlm_mae[Mn] PASSED                 [ 25%]
test_suite_5.py::TestCaixaPreta::test_rlm_r2[Mn] PASSED                  [ 26%]
test_suite_5.py::TestCaixaPreta::test_bagging_rmse[Mn] PASSED            [ 27%]
test_suite_5.py::TestCaixaPreta::test_bagging_mae[Mn] PASSED             [ 28%]
test_suite_5.py::TestCaixaPreta::test_bagging_r2[Mn] PASSED              [ 29%]
test_suite_5.py::TestCaixaPreta::test_bagging_m_otimo[Mn] PASSED         [ 30%]
test_suite_5.py::TestCaixaPreta::test_subagging_rmse[Mn] PASSED          [ 31%]
test_suite_5.py::TestCaixaPreta::test_subagging_mae[Mn] PASSED           [ 32%]
test_suite_5.py::TestCaixaPreta::test_subagging_r2[Mn] PASSED            [ 33%]
test_suite_5.py::TestCaixaPreta::test_subagging_k_m_otimos[Mn] PASSED    [ 34%]
test_suite_5.py::TestCaixaPreta::test_estrutura_resultado[Mn] PASSED     [ 35%]
test_suite_5.py::TestCaixaPreta::test_rlm_rmse[Mo] PASSED                [ 36%]
test_suite_5.py::TestCaixaPreta::test_rlm_mae[Mo] PASSED                 [ 38%]
test_suite_5.py::TestCaixaPreta::test_rlm_r2[Mo] PASSED                  [ 39%]
test_suite_5.py::TestCaixaPreta::test_bagging_rmse[Mo] PASSED            [ 40%]
test_suite_5.py::TestCaixaPreta::test_bagging_mae[Mo] PASSED             [ 41%]
test_suite_5.py::TestCaixaPreta::test_bagging_r2[Mo] PASSED              [ 42%]
```

```
test_suite_5.py::TestCaixaPreta::test_bagging_m_otimo[Mo] PASSED         [ 43%]
test_suite_5.py::TestCaixaPreta::test_subagging_rmse[Mo] PASSED          [ 44%]
test_suite_5.py::TestCaixaPreta::test_subagging_mae[Mo] PASSED           [ 45%]
test_suite_5.py::TestCaixaPreta::test_subagging_r2[Mo] PASSED            [ 46%]
test_suite_5.py::TestCaixaPreta::test_subagging_k_m_otimos[Mo] PASSED    [ 47%]
test_suite_5.py::TestCaixaPreta::test_estrutura_resultado[Mo] PASSED     [ 48%]
test_suite_5.py::TestCaixaPreta::test_rlm_rmse[Cr] PASSED                [ 50%]
test_suite_5.py::TestCaixaPreta::test_rlm_mae[Cr] PASSED                 [ 51%]
test_suite_5.py::TestCaixaPreta::test_rlm_r2[Cr] PASSED                  [ 52%]
test_suite_5.py::TestCaixaPreta::test_bagging_rmse[Cr] PASSED            [ 53%]
test_suite_5.py::TestCaixaPreta::test_bagging_mae[Cr] PASSED             [ 54%]
test_suite_5.py::TestCaixaPreta::test_bagging_r2[Cr] PASSED              [ 55%]
test_suite_5.py::TestCaixaPreta::test_bagging_m_otimo[Cr] PASSED         [ 56%]
test_suite_5.py::TestCaixaPreta::test_subagging_rmse[Cr] PASSED          [ 57%]
test_suite_5.py::TestCaixaPreta::test_subagging_mae[Cr] PASSED           [ 58%]
test_suite_5.py::TestCaixaPreta::test_subagging_r2[Cr] PASSED            [ 59%]
test_suite_5.py::TestCaixaPreta::test_subagging_k_m_otimos[Cr] PASSED    [ 60%]
test_suite_5.py::TestCaixaPreta::test_estrutura_resultado[Cr] PASSED     [ 61%]
test_suite_5.py::TestCaixaPreta::test_rlm_rmse[Ni] PASSED                [ 63%]
test_suite_5.py::TestCaixaPreta::test_rlm_mae[Ni] PASSED                 [ 64%]
test_suite_5.py::TestCaixaPreta::test_rlm_r2[Ni] PASSED                  [ 65%]
test_suite_5.py::TestCaixaPreta::test_bagging_rmse[Ni] PASSED            [ 66%]
test_suite_5.py::TestCaixaPreta::test_bagging_mae[Ni] PASSED             [ 67%]
test_suite_5.py::TestCaixaPreta::test_bagging_r2[Ni] PASSED              [ 68%]
test_suite_5.py::TestCaixaPreta::test_bagging_m_otimo[Ni] PASSED         [ 69%]
test_suite_5.py::TestCaixaPreta::test_subagging_rmse[Ni] PASSED          [ 70%]
test_suite_5.py::TestCaixaPreta::test_subagging_mae[Ni] PASSED           [ 71%]
test_suite_5.py::TestCaixaPreta::test_subagging_r2[Ni] PASSED            [ 72%]
test_suite_5.py::TestCaixaPreta::test_subagging_k_m_otimos[Ni] PASSED    [ 73%]
test_suite_5.py::TestCaixaPreta::test_estrutura_resultado[Ni] PASSED     [ 75%]
test_suite_5.py::TestCaixaPreta::test_rlm_rmse[Fe] PASSED                [ 76%]
test_suite_5.py::TestCaixaPreta::test_rlm_mae[Fe] PASSED                 [ 77%]
test_suite_5.py::TestCaixaPreta::test_rlm_r2[Fe] PASSED                  [ 78%]
test_suite_5.py::TestCaixaPreta::test_bagging_rmse[Fe] PASSED            [ 79%]
test_suite_5.py::TestCaixaPreta::test_bagging_mae[Fe] PASSED             [ 80%]
test_suite_5.py::TestCaixaPreta::test_bagging_r2[Fe] PASSED              [ 81%]
test_suite_5.py::TestCaixaPreta::test_bagging_m_otimo[Fe] PASSED         [ 82%]
test_suite_5.py::TestCaixaPreta::test_subagging_rmse[Fe] PASSED          [ 83%]
test_suite_5.py::TestCaixaPreta::test_subagging_mae[Fe] PASSED           [ 84%]
test_suite_5.py::TestCaixaPreta::test_subagging_r2[Fe] PASSED            [ 85%]
test_suite_5.py::TestCaixaPreta::test_subagging_k_m_otimos[Fe] PASSED    [ 86%]
test_suite_5.py::TestCaixaPreta::test_estrutura_resultado[Fe] PASSED     [ 88%]
test_suite_5.py::TestStress::test_rlm_tempo_matriz_grande PASSED         [ 89%]
test_suite_5.py::TestStress::test_bagging_tempo_10bags PASSED            [ 90%]
test_suite_5.py::TestStress::test_rlm_tempo_stress PASSED                [ 91%]
test_suite_5.py::TestStress::test_ram_rlm_matriz_grande PASSED           [ 92%]
test_suite_5.py::TestStress::test_ram_bagging_matriz_grande PASSED       [ 93%]
test_suite_5.py::TestStress::test_ram_stress_registra_limite PASSED      [ 94%]
test_suite_5.py::TestStress::test_bagging_escalabilidade_bags[5] PASSED  [ 95%]
test_suite_5.py::TestStress::test_bagging_escalabilidade_bags[10] PASSED [ 96%]
test_suite_5.py::TestStress::test_bagging_escalabilidade_bags[25] PASSED [ 97%]
test_suite_5.py::TestStress::test_bagging_escalabilidade_bags[50] PASSED [ 98%]
test_suite_5.py::TestStress::test_parse_matrix_arquivo_grande PASSED     [100%]

======================== 92 passed in 75.25s (0:01:15) ========================
```